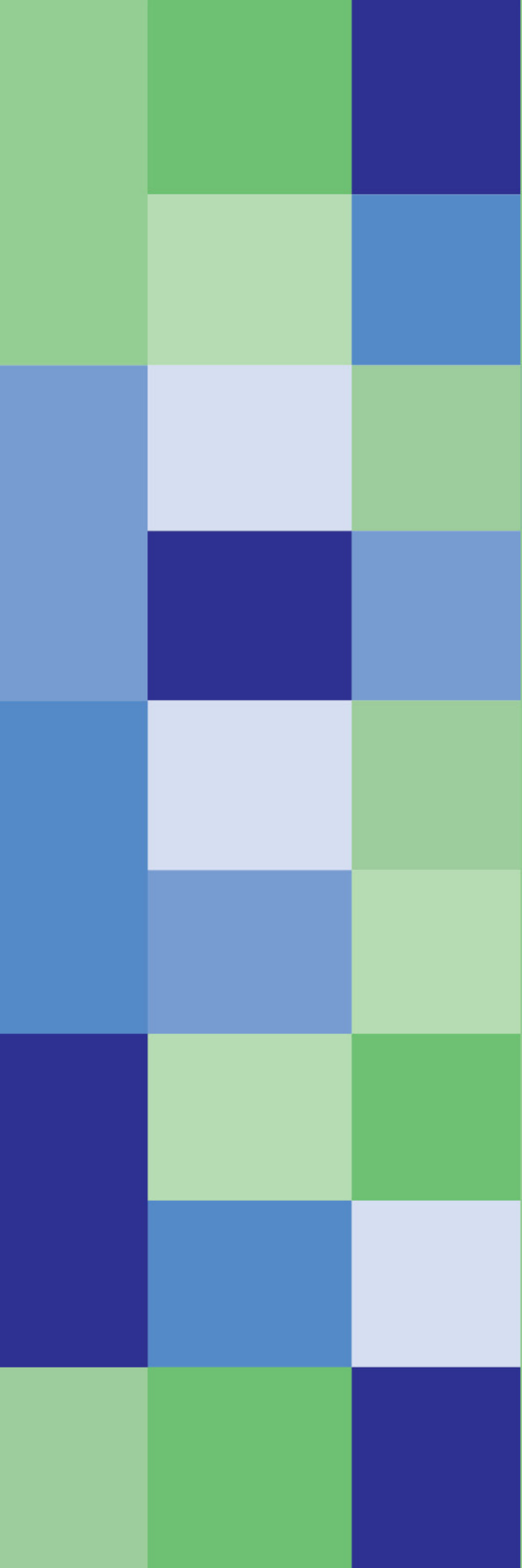


A decorative vertical strip on the left side of the cover, composed of a grid of squares in various shades of green, blue, and dark blue. The main background is a solid light green.

A₂ ADVANCED

IT FRONTIERS

Advanced Series

Teacher's Edition

A small, solid pink square located in the bottom right corner of the cover.

Author Wael I. Nasr
Copyright © 2025 Chapters & Co.
First Edition

All rights reserved. This edition is prepared exclusively for internal circulation and is not for commercial sale or public distribution. No part of this publication may be reproduced, stored, or transmitted in any form without prior written permission of the publisher.

Published by Chapters & Co.

A₂ ADVANCED

IT FRONTIERS

Advanced Series

Teacher's Edition

AUTHOR'S NOTE

This series reflects my conviction that computer science education must return to its scientific foundations. In an era of rapidly evolving technologies, the focus should remain on enduring concepts, presented here through clear structure, practical examples, and sound pedagogy that encourage curiosity and critical thinking. Digital literacy emerges naturally through the activities, yet it should not be confined to computer science alone — it is a shared responsibility across all disciplines. While programming is important, the greater value lies in cultivating algorithmic thinking — the ability to analyze, design, and reason about processes. To support this, I developed Jargon, a pseudocode framework that minimizes the distractions of syntax and semantics, enabling learners to focus on conceptual clarity and problem-solving. My aim is to make computer science accessible, rigorous, and relevant for young learners, while preparing them to adapt to technologies that will continue to change.

DIGITAL LITERACY IN EDUCATION

Digital literacy is not the sole responsibility of computer science education but a shared responsibility across all disciplines. Each subject carries its own digital tools, applications, and nuances of use: from data analysis in science, to dynamic modeling in mathematics, to research and composition in language learning. While computer science provides the foundation for understanding these technologies, their most meaningful application emerges within the disciplines where they are best used.

GROUNDING IN RESEARCH

This program reflects a growing consensus in computer science education: that we must balance practical coding with the enduring foundations of the discipline. Researchers and educators emphasize that computer science is not only about learning tools, which change rapidly, but about cultivating computational thinking and algorithmic reasoning — skills that remain constant.

The K-12 Computer Science Framework highlights computational thinking as essential for all learners, while international assessments such as PISA now include these skills alongside mathematics and science. Scholars from MIT and the ACM community have cautioned against reducing computer science to vocational training, noting that its true power lies in analysis, problem-solving, and conceptual clarity.

This publication embraces these principles: programming is introduced as a valuable skill, but the greater emphasis is on nurturing algorithmic thinking and understanding the scientific concepts that endure beyond changing technologies.

This series seeks to make computer science accessible and enduring, grounded in science, informed by research, and integrated across education.

Table of Contents

Part A1

Chapter 1

Computers 9

Lesson 1: Introducing Computers	10
Lesson 2: Types of Computers	32
Lesson 3: Systems & Interfaces	54
Lesson 4: Behind the Interface	76
Lesson 5: Design Optimizations	102

Chapter 2

Computers & Beyond 121

Lesson 1: Human-Computer Interface	122
Lesson 2: Computer Interaction	140
Lesson 3: Clouds & Things	170
Lesson 4: Information Systems	192

Part A2

Chapter 3

Data & Information 7

Lesson 1: From Data to Information	8
Lesson 2: Data Structures	30
Lesson 3: Searching Linear Data	68
Lesson 4: Sorting Linear Data	94

Chapter 4

Artificial Intelligence 157

Lesson 1: Machine Learning	158
Lesson 2: Prediction	174
Lesson 3: Logic & Reasoning	194
Lesson 4: AI in the Real-World	216

3

DATA & INFORMATION

LESSON 1: FROM DATA TO INFORMATION

Most of our interaction with the world relies on our ability to form an understanding of it; we need to know about it. As we have previously learned, our interactions with the world around us have three basic phases: sensing, processing, and responding. The sensing phase provides us with very basic signals from our environment: light, sound, touch, smell, and others. These are merely signals that we are capable of acquiring and converting into electrical signals our brain can use. It is only after we process these signals that they become meaningful; they become information. Prior to that, they were merely data.

EXAMPLE 1.1

When we see something, it is initially light signals perceived as various colors and shapes. But once we process them, these colors and shapes become our best friend's face. The colors and shapes in the light signals now have meaning; the data was transformed into information.

DATA IS ANY FORM OF RAW INPUT THAT CAN BE PROCESSED INTO MEANINGFUL INFORMATION.

In this definition of 'data', the term 'meaningful' is added to emphasize that processing imparts meaning to the acquired data, thereby converting it into information. It is meaning that differentiates data from information. The same applies to computers. Regardless of whether a human or a computer is processing data, the act of processing is what converts data into information.

EXAMPLE 1.2

A sensor inputs the following stream of numbers into a computer: '29, 28, 33, 37, 38, 39, 38, 31, 29, 28'. As a list of numbers, this data is meaningless. Let's begin to process it.

1. Identify the sensor that captured the data: Thermometer.
2. Label the data according to the sensor that captured it: Temperature (degrees Celsius).

So far, we have determined that this list of numbers is actually a list of temperatures. One would then want to know, what are these temperatures of?

3. Location of sensor: Outdoor.

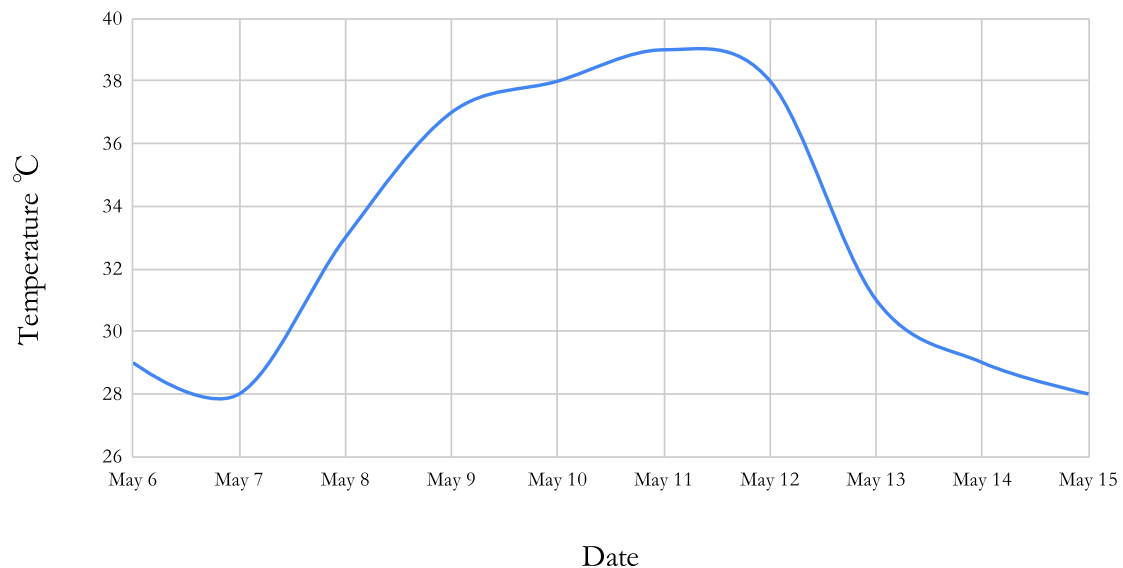
Now that we have gained more information, we know that these numbers are the outdoor temperatures. But when were these readings obtained? Typically, sensors would keep a date and time stamp with each reading. As part of our processing of this data, let us transform this list of temperature data into a table that contains the date and time each reading was taken.

4. Compile date, time and temperature data into a table.

Date	May 6	May 7	May 8	May 9	May 10	May 11	May 12	May 13	May 14	May 15
Time	12:00	12:00	12:00	12:00	12:00	12:00	12:00	12:00	12:00	12:00
Temp °C	29	28	33	37	38	39	38	31	29	28

Now we have realized that processing data by compiling it with other related data, in this case the date and time of each temperature reading, adds more information. We also learned that organizing data and presenting it in tabular form allows us to better visualize and understand the data. So, the presentation of data, which requires processing, is an added layer of information. Another method of presenting or visualizing data is charting.

Outdoor Temperature Reading in May



So we have gone from pure raw data of numbers to information; we have learned that between May 8 and May 12, there was a sudden rise and fall in temperature in the area from which the data was collected. This leads us to one last step, interpreting the data. A sudden rise in temperature could be interpreted as a heat wave, and therefore the information learned from our data is that: 'between May 8 and May 12, a heat wave occurred in the area from which the data was collected.' This information was derived through the processing of data, which included identifying, organizing, visualizing, and interpreting the data.

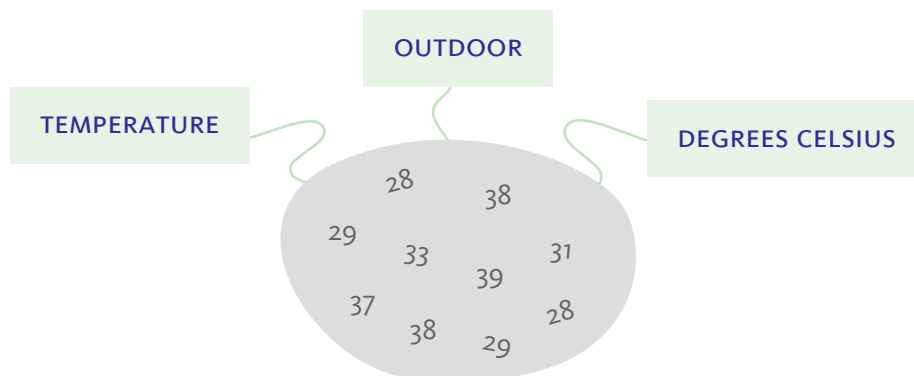
CONVERTING DATA TO INFORMATION INCLUDES IDENTIFYING,
ORGANIZING, VISUALIZING, AND INTERPRETING THE DATA.

IDENTIFYING DATA

One of the initial steps in processing data is identifying what it represents. The information extracted from data is significantly influenced by how it is identified. In the previous example, the list of numbers: `29, 28, 33, 37, 38, 39, 38, 31, 29, 28`, could represent the number of students going on a field trip, or the number of views someone received on their YouTube posting. Therefore, identifying the data by determining its source and what it represents is critical, as it sets the foundation for all subsequent steps.

LABELING DATA

Labeling data is an essential part of identifying it. This helps prevent mistakes and ensures that everyone examining the data understands its context and meaning. So in the case of Example 1.2, the labels that would be required are: 'temperature', 'outdoor', and 'degrees Celsius'.



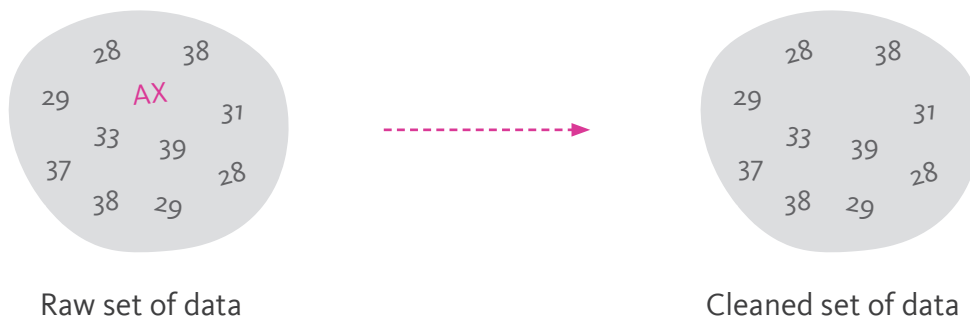
ORGANIZING DATA

Another important step in the process of extracting information is organizing the available data. There are several ways in which data can be organized: cleaning, structuring, and sorting.

CLEANING DATA

Cleaning data is the process of removing erroneous data. Errors can enter a dataset for several reasons: errors during input, such as a faulty sensor in an automated system or a mistake by a person entering the data manually. Other errors can arise while data is being stored, waiting to be processed. While there are other reasons errors can occur, the important point is that data should be screened and errors removed.

For example, what if our list of numbers read: `29, 28, 33, AX, 37, 38, 39, 38, 31, 29, 28` or `29, 28, 33, 300, 37, 38, 39, 38, 31, 29, 28`? In either case, the data items 'AX' and '300' do not fit for obvious reasons: an outdoor temperature must be a number, not letters, and 300°C is certainly too high a temperature to be recorded outdoors on Earth, unless maybe you are near a volcano or on a different planet, but that is not the situation.



STRUCTURING DATA

Data is generally stored in computers in a certain manner that determines how each item of data relates to the others. These are called data structures. We will explore them in greater detail later in this chapter, but for now, we will simply identify two data structures: lists and tables.

1. **List (Array):** An arrangement of data in a linear fashion; all data items are positioned next to each other in a line.

- 2. Table:** Well-suited to organize multiple sets of data. In Example 1.2, the initial organization of the temperature data was linear, with all the numbers presented in an array. But when other sets of data were included, such as the date and time, a table was used to organize the data.

Data structured as an array with one item following the other.

29, 28, 33, 37, 38, 39, 38, 31, 29, 28

Data structured as a table with rows and columns.

Date	May 6	May 7	May 8	May 9	May 10	May 11	May 12	May 13	May 14	May 15
Temp °C	29	28	33	37	38	39	38	31	29	28

SORTING DATA

Sorting plays an important role in data organization and processing. It is the process through which data is rearranged based on certain criteria, such as alphabetical order for alphanumeric data, increasing or decreasing value for numeric data, or by category for data that can be categorized. In the previous example, the data was already sorted. The sorting criterion was date, meaning that the temperature data was in chronological order based on the date it was acquired.

Data sorted chronologically by increasing dates.

Date	May 6	May 7	May 8	May 9	May 10	May 11	May 12	May 13	May 14	May 15
Temp °C	29	28	33	37	38	39	38	31	29	28

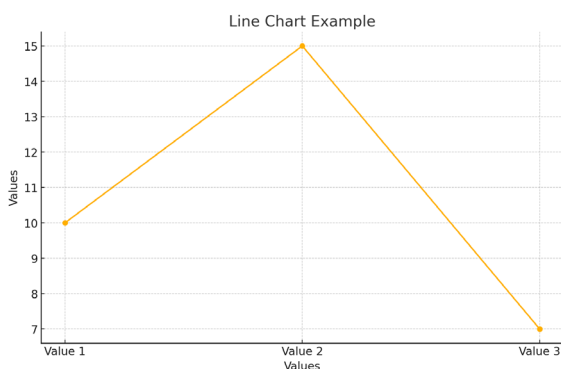
Data sorted by increasing temperature readings.

Date	May 7	May 15	May 6	May 14	May 13	May 8	May 9	May 10	May 12	May 11
Temp °C	28	28	29	29	31	33	37	38	38	39

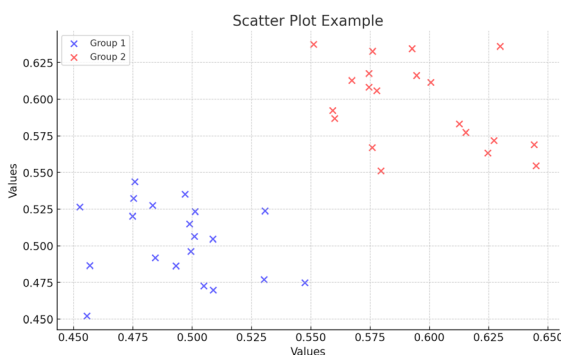
Each method of sorting allows us to better identify certain aspects of the data. Data sorted chronologically by increasing dates allows us to more easily identify trends over time, while data sorted by increasing temperature readings allows us to more easily identify the lowest and highest temperature readings collected in the dataset.

VISUALIZING DATA

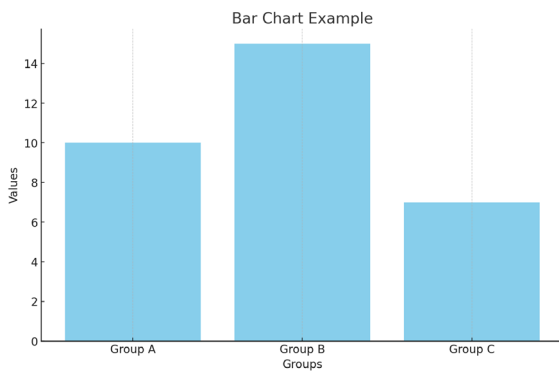
To visualize data means to present it in a way that makes it easy to visually identify trends and patterns. A common and effective method to visually present data is through charts. There are different types of charts, each presenting the data in a slightly different way. Four common types of charts are: line chart, scatter plot, bar chart, and pie chart.



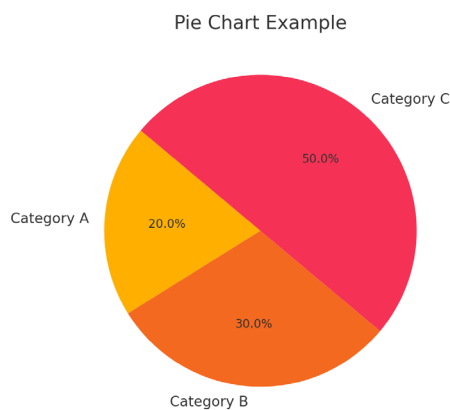
A **line chart** is a chart in which the value of each data point is represented by a point on the chart, and the points are then connected by a line. Line charts are useful for showing trends over time or continuous data.



A **scatter plot** is similar to a line chart in that the value of each data point is represented by a point on the chart. However, in a scatter plot, the points are not connected by a line. Instead, scatter plots are used to display the relationship between two continuous variables, making it easier to identify patterns, trends, and correlations.



A **bar chart** is a chart in which data is organized into groups or categories, with the value of each category represented by a bar. Bar charts can be either vertical or horizontal and are useful for comparing quantities across different categories.



A **pie chart** is a chart in which data is organized into groups or categories that are part of a whole. Each category is represented by a slice of the pie, which indicates the proportion of the whole that this category represents. Pie charts are useful for showing relative proportions and percentages of a whole.

The choice of the type of chart used to visualize data depends on the type of data itself. Generally, data can be broadly classified into two main types: quantitative and qualitative. Such classification influences the handling and presentation of the data.

Notes

Understanding the transformation of data into information is a crucial skill that extends beyond computer science, impacting fields like healthcare, business, finance, education, and environmental studies. This fosters critical thinking and analytical skills, enhancing our ability to interpret and act on complex data.

Quantitative Data

Quantitative data is numerical data that can be measured or counted, which means that it can either be continuous or discrete. Continuous data is essentially measurable data that can take any value within a range, such as temperature, height, and time. This type of data is best visualized using line charts and scatter plots. For example, in Example 1.2, a line chart was used to visually present the data because line charts are best suited for showing trends over time or continuous progression, such as the change in temperature over time.

QUANTITATIVE DATA IS NUMERICAL DATA THAT CAN BE MEASURED OR COUNTED.

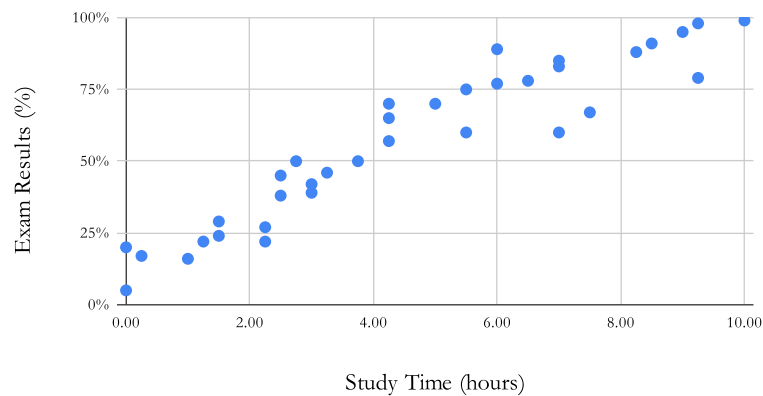
EXAMPLE 1.3

Data was collected from 35 students. Two sets of data were obtained: the hours of study for an exam and the results achieved on that exam. This is the raw data unprocessed in the form of a list, or more specifically, a paired list:

(3.00, 42%), (5.00, 70%), (7.00, 60%), (1.25, 22%), (2.50, 38%), (10.00, 99%), (6.00, 77%), (0.00, 20%), (8.50, 91%), (4.25, 70%), (0.00, 5%), (1.50, 24%), (6.50, 78%), (0.25, 17%), (2.75, 50%), (9.00, 95%), (9.25, 98%), (1.00, 16%), (2.25, 27%), (7.50, 67%), (9.25, 79%), (1.50, 29%), (2.50, 45%), (2.25, 22%), (7.00, 83%), (3.25, 46%), (6.00, 89%), (4.25, 65%), (3.00, 39%), (5.50, 75%), (3.75, 50%), (8.25, 88%), (4.25, 57%), (5.50, 60%), (7.00, 85%)

Looking at this data, it is unclear what information it holds. However, if it is processed and charted in the form of a scatter plot, the information it holds becomes better visualized.

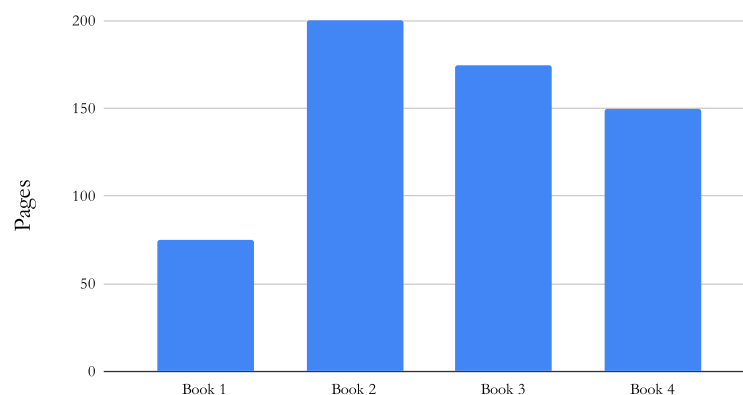
Exam Results (%) vs. Study Time (hours)



The scatter plot of exam results versus study time reveals a pattern: as the hours of studying increase, so do the exam results. While it is not a perfect straight line, it does reveal a trend that is not noticeable in the raw data.

Discrete data, on the other hand, is typically countable data that can only take distinct values, such as the number of pages in a book or the number of students in a class. It is not continuous, meaning it cannot be in the form of fractions or decimals. For example, you cannot have $10\frac{1}{2}$ pages in a book or 20.25 students in a class. The values of discrete data can only be whole numbers. In general, bar charts are a good choice for visually representing discrete data.

Number of Pages in Books



The bar chart allows for comparison of the number of pages in each book.

Qualitative Data

Qualitative data is descriptive data that can be observed but not measured. It is also referred to as categorical data because it can be categorized into groups and labeled, such as types of fruits, names of cities, colors of cars, and yes/no responses in a survey. This type of data is best visualized using bar charts and pie charts, which allow for a clear representation of categories and their frequencies. However, before it can be represented in charts, this type of data must be converted into quantitative data. The most common method of making this conversion is by counting the data items that fall within each category.

QUALITATIVE DATA IS DESCRIPTIVE DATA THAT CAN BE OBSERVED BUT NOT MEASURED.

EXAMPLE 1.4

The school's information management system is backed by a database that contains data about students attending the school, including their names, ages, grades, gender, and other pertinent information. The school has 618 students enrolled in Grades 1 through 12, which is too many to show in this book. Here are the first 6 entries in the database table:

Name	Family	Age	Grade	Gender
Dana	Lankart	6	1	Girl
Ahmed	Dandash	6	1	Boy
Ruba	Najem	8	3	Girl
Tarek	Najem	10	5	Boy
Thomas	Johnson	9	4	Boy
Ghassan	Mekki	9	4	Boy

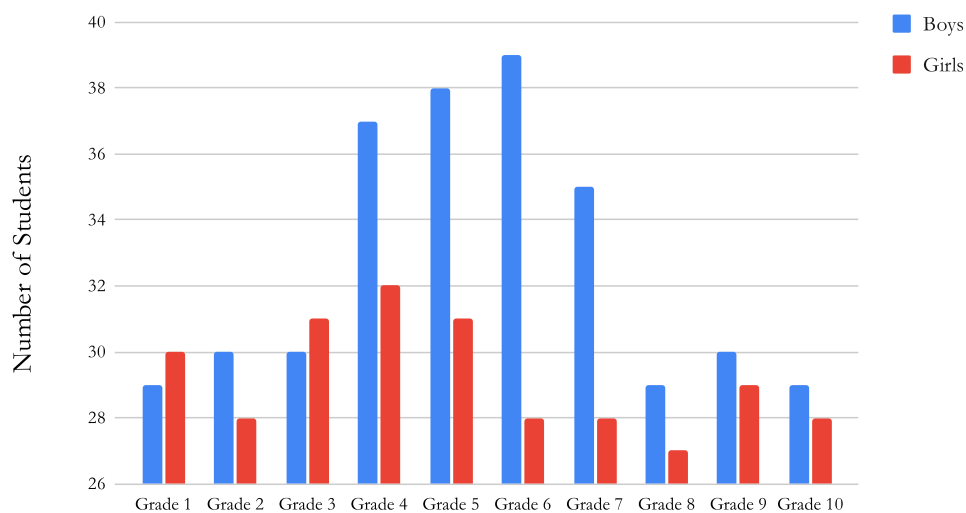
Based on this table, grade levels and gender are considered categorical qualitative data. The data items of grade levels are the names of each grade, such as 'Grade 1', 'Grade 2', and so on up to 'Grade 12'. The data items under gender are categorized as 'Boy' and 'Girl'. None of these data items are measurable or numerical in nature.

Assuming we want to learn the number of boys and girls in each grade level, the first step would be to query the database to count the number of occurrences of the data items 'Boy' and 'Girl' for each grade. This query results in the conversion of qualitative data into discrete quantitative data, from which we can extract the following information:

	Grade 1	Grade 2	Grade 3	Grade 4	Grade 5	Grade 6	Grade 7	Grade 8	Grade 9	Grade 10
Boys	29	30	30	37	38	39	35	29	30	29
Girls	30	28	31	32	31	28	28	27	29	28

Since the data can be grouped by grade levels (Grade 1 - Grade 10) and gender (boy, girl) it is better visualized using a bar chart. The resulting bar chart would look like this:

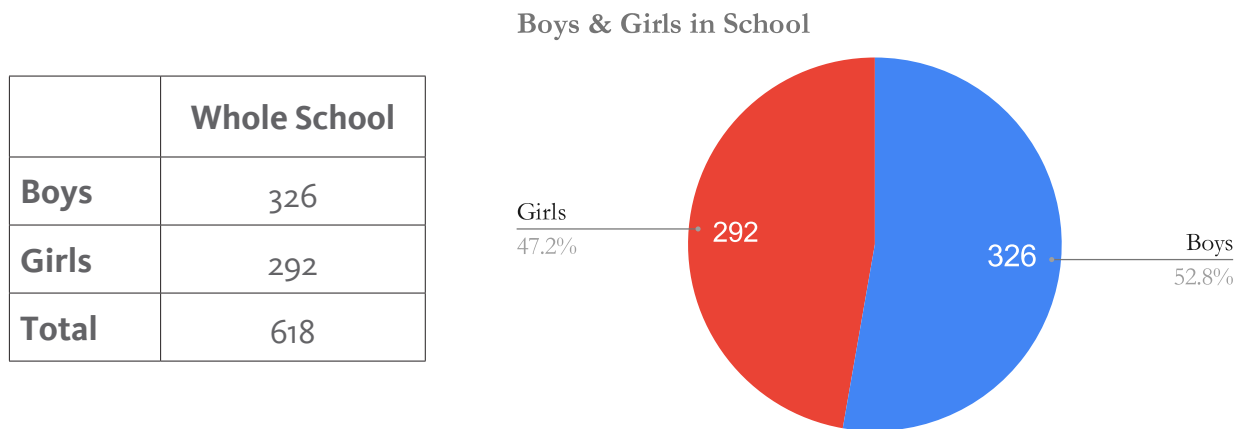
Boys & Girls in Grade Levels



The bar chart reveals that the number of boys exceeds the number of girls, particularly in grades 4, 5, 6, and 7.

EXAMPLE 1.5

If, for the same data, we wanted to visualize the proportion of boys and girls in the whole school, we could use a pie chart. Since we are now interested in the entire school rather than the details of each grade level, a pie chart is appropriate. To obtain this information, we would add the number of boys across all grades and the number of girls across all grades. The two categories, ‘Boys’ and ‘Girls,’ can then be represented in a pie chart, showing that the number of boys and girls is nearly the same.



INTERPRETING DATA

The final step in the process of converting raw data to actionable information is the interpretation of the findings. This step involves explaining what the data is presenting to us, whether it is a spike in outdoor temperature, a relationship between two sets of data such as study time and exam results, or a discrepancy between the proportion of boys and girls in a school. Interpreting these findings poses two crucial questions: “Why?” and “What should we do?”

The first question, “Why?” forces us to identify the factors underlying our findings. This step is essential for uncovering the reasons that have led to the

information we have discovered. It adds an extra layer of understanding that is necessary for a deeper comprehension of the world around us.

The second question, “What should we do?” prompts us to consider our next steps based on our findings. These next steps may involve further investigating the underlying causes or, if we are satisfied with our findings, learning from them and incorporating them into our decision-making process. This kind of information is referred to as ‘actionable information’, which is information that we can use to make informed decisions.

The process of converting raw data into information is fundamental to understanding and interpreting the world around us. By carefully following the basic stages of identifying, organizing, visualizing, and interpreting data, we can transform unprocessed data into actionable information regardless of the initial nature of the data, whether it is quantitative or qualitative. This process should guide our learning and decision-making, providing us with the analytical tools necessary to make informed decisions based on accurate and meaningful information.

Career Tips

Understanding the process of converting raw data into actionable information can open doors to a variety of rewarding careers, such as data analysts, data scientists, business intelligence developers, financial analysts, and market research analysts. All of these roles are in high demand in today’s data-driven world.

SUMMARY

The essential steps and processes for converting raw data into information are categorized into four main stages:

1. Identifying Data: This includes collecting, identifying, and labeling data.
2. Organizing Data: This includes cleaning, structuring, and sorting data.
3. Visualizing Data: Creating visual representations (charts, graphs) to highlight trends and patterns.
4. Interpreting Data: Analyzing visualized data to extract meaningful insights that lead to informed decision-making.

Data can be categorized as quantitative and qualitative.

Quantitative data is numerical data that can be measured (continuous) or counted (discrete).

Qualitative data is descriptive data that can be observed but not measured and is referred to as categorical data because it can be categorized into groups and labeled.

Four common charts used to visualize data are line charts, scatter plots, bar charts, and pie charts. Line charts and scatter plots are best for continuous quantitative data, while bar charts and pie charts are best for discrete and categorical data.

Activity 1.1

Read the following statements, and write on the line provided for each statement 'T' if it is true and 'F' if it is false.

T/F	STATEMENT
<u>F</u>	The process of converting raw data into information only includes organizing the data.
<u>T</u>	Labeling data helps prevent mistakes.
<u>F</u>	Continuous data can only take discrete values.
<u>T</u>	A line chart is best for showing trends over time.
<u>F</u>	Categorical data represents measurements that can take any value within a range.
<u>T</u>	A scatter plot is best suited for displaying continuous data.
<u>T</u>	Cleaning data involves removing erroneous data.
<u>F</u>	A pie chart is suitable for visualizing continuous data.
<u>T</u>	Sorting data arranges it for easy access and analysis.
<u>T</u>	Interpreting data involves explaining what the data is presenting.

Activity 1.2

Answer the questions below.

1. What is the first step in converting raw data into information?
 - a) Visualizing Data
 - b) Organizing Data
 - c) **Identifying Data**
 - d) Interpreting Data

2. Which of the following charts is best used for showing trends over time?
 - a) Bar Chart
 - b) **Line Chart**
 - c) Pie Chart
 - d) Scatter Plot

3. What is the primary purpose of cleaning data?
 - a) To identify data
 - b) To create charts
 - c) **To remove erroneous data**
 - d) To label data

4. Which type of data can take any value within a range?
 - a) Categorical Data
 - b) Discrete Data
 - c) **Continuous Data**
 - d) Nominal Data

5. What does a scatter plot best represent?
 - a) Proportions within a whole
 - b) Trends over time
 - c) Relationships between two continuous variables
 - d) Comparisons across categories

6. What type of chart would you use to compare quantities across different categories?
 - a) Bar Chart
 - b) Line Chart
 - c) Pie Chart
 - d) Scatter Plot

7. What type of chart is best for showing proportions of a whole?
 - a) Bar Chart
 - b) Line Chart
 - c) Pie Chart
 - d) Scatter Plot

8. Which of the following is a key aspect of data visualization?
 - a) Removing erroneous data
 - b) Arranging data in a list
 - c) Presenting data in a visual format
 - d) Labeling data points

9. Why is it important to interpret data after visualizing it?
 - a) To ensure data is labeled correctly
 - b) To derive meaningful insights and make decisions
 - c) To remove duplicate entries
 - d) To organize data in a table

Activity 1.3

Read the provided data examples. Classify each example as either quantitative or qualitative.

DATA EXAMPLES

1. *Types of Pets: Different kinds of pets owned by students.*
2. *Temperature: Daily temperatures recorded over a month.*
3. *Weight: The weight of different fruits*
4. *Brands of Cars: Different car brands owned by people.*
5. *Time: The time taken by athletes to complete a race.*
6. *Blood Types: Blood types of students.*

Quantitative : 2. Temperature, 3. Weight, 5. Time

Qualitative : 1. Types of Pets, 4. Brands of Cars, 6. Blood types

Activity 1.4

Identify 2 examples of quantitative data and 2 examples of qualitative data from your daily life.

Quantitative : Measure the time you spend on activities such as studying, exercising, and watching TV.

Qualitative : Record the different modes of transportation you use throughout the week such as bus, car, bike, and walking.

Activity 1.5

Examine the dataset provided in each question and determine the choice that best describes it.

1. List A = {12.5, 12.3, 11.7, 14.6, 15.1}
 - a) Qualitative Categorical Data
 - b) **Quantitative Continuous Data**
 - c) Quantitative Discrete Data

2. List B = {red, blue, red, red, yellow, blue, blue, red}
 - a) **Qualitative Categorical Data**
 - b) Quantitative Continuous Data
 - c) Quantitative Discrete Data

3. List C = {yes, yes, no, no, maybe, maybe, yes, yes, yes}
 - a) **Qualitative Categorical Data**
 - b) Quantitative Continuous Data
 - c) Quantitative Discrete Data

4. List D = {2, 3, 7, 2, 4, 1, 1, 1, 4, 5}
 - a) Qualitative Categorical Data
 - b) Quantitative Continuous Data
 - c) **Quantitative Discrete Data**

5. List E = { $\frac{1}{2}$, $8\frac{1}{2}$, 7, $8\frac{1}{4}$, $5\frac{3}{4}$ }
 - a) Qualitative Categorical Data
 - b) **Quantitative Continuous Data**
 - c) Quantitative Discrete Data

Activity 1.6

Examine the datasets provided and read the supporting narrative. Based on the information given, use a spreadsheet application to generate a chart that visually represents the data. Ensure that the data is appropriately organized, and that both your tabulated data and chart are clearly labeled. Identify patterns and draw conclusions from the data.

Refer to the spreadsheet '[IT Frontiers A Part 2 - Lesson 1 - Activity 1.6](#)'.

1. Age_Height_List = { (11, 143), (12, 145), (13, 153), (14, 162), (15, 170), (16, 175), (17, 178), (18, 180), (19, 181), (20, 181) }

Format: (age, height)

Omar's pediatrician has been keeping track of his height from the age of 11 to 20 years. Every year, she records his height in centimeters.

2. Rainfall_Weight_List = { (12, 150), (10, 120), (20, 180), (11, 140), (15, 160), (13, 137), (19, 195), (10, 135), (18, 180), (11, 115), (17, 180), (14, 142), (13, 155), (13, 130), (17, 165), (11, 134), (14, 158), (12, 141), (16, 180), (16, 162) }

Format: (rainfall, weight)

A farmer records the weekly rainfall in millimeters and the weight of the fruit picked from the tree in grams. The weekly rainfall ranged between 10 and 20 mm.

3. Item_Count_List = { (Laptops, 200), (Smartphones, 300), (Tablets, 170), (Desktops, 150) }

Format: (item, count)

An electronics store manager keeps a record of the items in his store. He wants to present this data to his administration. He queries the database for a count of certain items and receives this response.

4. Coats_Beachwear_List = { (January, 220, 50), (February, 200, 60), (March, 170, 100), (April, 130, 150), (May, 100, 200), (June, 50, 250), (July, 40, 230), (August, 30, 210) }

Format: (month, winter coats, beachwear)

A fashion retailer wants to compare the sales of winter coats and beachwear over the last 8 months.

Notes

A list can contain items in the form of doubles, triplets, or more, which are referred to as tuples. Tuples are similar to lists, but the items in a tuple have a fixed order and cannot be changed.

Activity 1.7

Examine the provided datasets and supporting narrative. Using the information given, use a spreadsheet application to generate a chart that visually represents the data. Organize the data appropriately, ensuring that both your tabulated data and chart are clearly labeled. Identify and clean any erroneous data in the dataset. Finally, identify patterns in the data and draw conclusions. [Refer to the spreadsheet 'IT Frontiers A Part 2 - Lesson 1 - Activity 1.7'.](#)

1. Climate_Data_List = { (20th, 310, 14.5), (19th, 290, 14.0), (18th, 280, 13.8), (17th, 275, 13.6), (16th, 270, 13.5), (15th, 265, 13.3), (14th, 260, 13.2), (13th, 255, 13.1), (12th, 250, 13.0), (11th, 245, 12.9), (10th, 240, 12.8), (9th, 235, 12.7), (8th, 230, 12.6), (7th, 225, 12.5), (6th, 220, 12.4), (5th, 215, 12.3), (4th, 210, 12.2), (3rd, 205, 12.1), (2nd, 200, 12.0), (1st, 195, 11.9) }

Format: (century, CO₂ level, temperature)

Sarah is a climatologist. She is studying global warming and is collecting ice core samples from Antarctica to measure the carbon dioxide levels trapped in the ice for each century. She is also measuring the ratios of gases trapped in the ice samples to estimate global temperatures at the time the ice was formed.

2. Coral_Reef_List = { (25.0, 8.2), (24.8, 8.3), (25.2, 8.1), (24.5, 8.5), (26.0, 7.8), (24.3, 8.7), (24.4, 8.6), (25.0, 8.0), (24.2, 8.8), (25.5, 8.0), (24.5, 8.4), (26.2, 7.5), (25.8, 7.6), (24.1, 8.9), (24.5, 19.1), (25.2, 7.9), (24.9, 8.1), (26.5, 7.4), (24.7, 8.5), (24.3, 8.6), (26.0, 7.7), (24.8, 8.3), (24.5, 8.5), (25.1, 8.0), (26.3, 7.5), (24.0, 9.0) }

Format: (temperature, O₂ level)

Jordan is a marine biologist studying the health of coral reefs. He uses underwater sensors to monitor the temperature of the water and the amount of dissolved oxygen at various depths.

3. Movie_Opinion_List = { Great, Good, Okay, Great, Good, Bad, Awful, Great, Good, Good, Great, Good, Okay, Great, Good, Bad, 1734, Good, Great, Good, Good, Okay, Bad, Good, Great, Okay, Good, Awful, Great, Good, Great, Good, Great, Good, Okay, Great, Good, Bad, Okay, Good, Great, Okay, Good, Great, Good, Good, Great, Good, Okay, Good, Great, Good, Great, Bad, Good, Great, Good, Good, Great, Okay, Great, Bad, !!!!, Good, Great, Okay, Good, Great, Awful, Good, Great, Good, Great, Okay, Good, Great, Good, Good, Great, Bad, Good, Great, Good, Okay, Great, Good, Good, Great, Good, Bad, Okay, Great }

A movie production company wants to gather audience feedback on their latest film. They categorize the feedback into five levels: Great, Good, Okay, Bad, and Awful. Note that this data is categorical and needs to be converted to discrete quantitative data before it is further processed.

LESSON 2: DATA STRUCTURES

In our current digital age, data is everywhere. From a simple image file on a smartphone to a complex database on a Google server, data surrounds us and is essential for the functioning of our apps and the generation of the information we rely on daily. However, data does not exist randomly in our computers; all stored data is organized in specific structures that allow us to organize, retrieve, modify, process, and store it efficiently. In this lesson, we will explore these data structures to gain a better understanding of their roles and importance.

DATA STRUCTURES ARE SPECIALIZED FORMATS FOR ORGANIZING, RETRIEVING, MODIFYING, PROCESSING, AND STORING DATA.

CLASSIFYING DATA STRUCTURES

Data structures are broadly classified based on the arrangement of data items within the structure. When data items are arranged sequentially (e.g., 1st, 2nd, 3rd, and so on) like books on a shelf, such structures are referred to as linear, such as lists or arrays.

EXAMPLE 2.1

Data in a linear structure are arranged sequentially.



"13, 47, 25, 28, 5, 32"

"Dana, Alex, Maya, Omar, Lana, Chen"

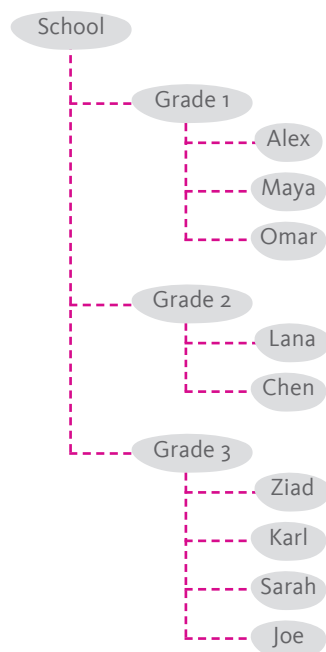
13	47	25	28	5	32
1 st	2 nd	3 rd	4 th	5 th	6 th

Dana	Alex	Maya	Omar	Lana	Chen
1 st	2 nd	3 rd	4 th	5 th	6 th

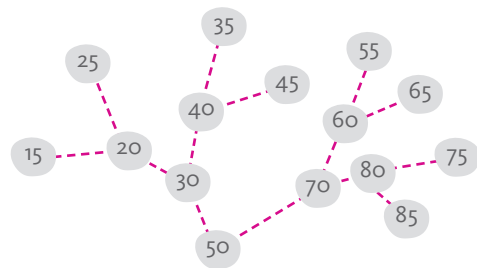
DATA STRUCTURES ARE CLASSIFIED AS LINEAR AND NON-LINEAR.

However, data items can be arranged in other ways. One such way is as a hierarchy, forming a structure similar to a tree, and therefore referred to as a tree data structure. In a tree, data is arranged along branches. Each branching point is referred to as a node, starting from the 'root node'. Each branching results in a new level in the hierarchy, with the root node existing at the first level.

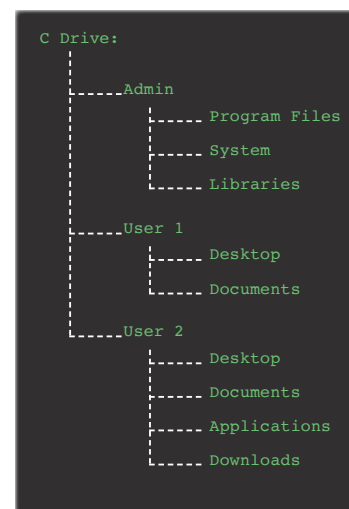
EXAMPLE 2.2



This organization chart for a school is a tree data structure with three hierarchical levels.



A binary search tree is used for efficiently searching databases and directories. This one has four levels. Can you count them?



This computer directory is a tree data structure with three hierarchical levels.

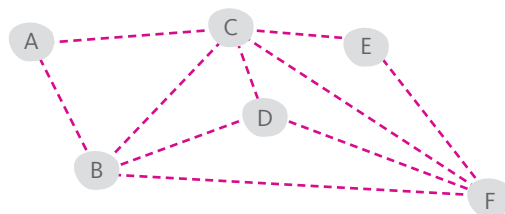
In the tree data structure, data items are connected along branches, and items in one branch cannot connect to items in another branch. However, sometimes it is necessary for data items to freely interconnect so that any data item can connect to any other data item, forming a structure known as a graph data structure.

In a graph, each data point acts as a node and can point to one or more other nodes in the structure. Information about the link between two nodes, referred to as an edge, is also stored as data information at each node. This information about the edge could represent the distance between the nodes, the time it takes to travel between the two nodes, or other relevant metrics.

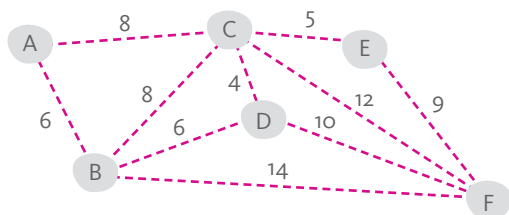
EXAMPLE 2.3



A map of flight routes between cities.



A graph data structure used to digitally represent the flight routes, with each city defined as a node.



The graph identifies the weight of each edge, with the number representing the hours of flight between two cities.

Adjacency List:

A: [(B, 6), (C, 8)]
 B: [(A, 6), (C, 8), (D, 6), (F, 14)]
 C: [(A, 8), (B, 8), (D, 4), (E, 5), (F, 12)]
 D: [(B, 6), (C, 4), (F, 10)]
 E: [(C, 5), (F, 9)]
 F: [(B, 14), (C, 12), (D, 10), (E, 9)]

This is an adjacency list to represent the graph. For each node, the nodes connected to it are listed along with their edge weights.

ALGORITHMS & DATA STRUCTURES

Everything inside a computer is determined by instructions, or algorithms—sets of instructions specific for particular tasks. One such group of tasks involves controlling data structures. Algorithms control data structures. These algorithms define the rules for size, data type, and methods for adding, removing, and accessing items in various data structures, whether linear (like lists), hierarchical (like trees), or interconnected (like graphs).

ALGORITHMS CONTROL DATA STRUCTURES.

LINEAR STRUCTURES

There are five common linear data structures: the array, list, tuple, queue, and stack. Being linear structures, all five maintain their items in a sequential manner. However, they vary depending on the rules that determine their size, the type of data they can accept, and how items are inserted or removed.

THE ARRAY

The array is unique because it is defined by a fixed size and a homogeneous set of data. Once an array is created, the number of positions it contains remains constant. While the data items within an array can be changed or swapped, no new positions can be added or removed, maintaining the array's fixed size. Additionally, an array can only hold one type of data, meaning all items must be of the same type, either numeric or alphanumeric.

	Array	List	Stack	Queue
Size	Fixed	Dynamic	Dynamic	Dynamic
Data type accepted	one type	mixed	mixed	mixed
Insert data	not allowed	any position	end	end
Remove data	not allowed	any position	end	beginning

In contrast, the list, stack, and queue are dynamic data structures. This means that their sizes can change as data is inserted or removed. They are also more flexible regarding the types of data they can contain, allowing for a mixture of data types.

EXAMPLE 2.4

Maya, Sarah, and Leen are three sisters. Maya is 12 years old, Sarah is 9, and Leen is 14. If we wanted to use an array as a data structure to store their names and ages, we would need to use two separate arrays: one for the alphanumeric (string) data, which would store their names, and another for the numeric (number) data, which would store their ages. This is because an array cannot hold more than one type of data simultaneously.



The three sisters: Maya, Sarah, and Leen.

```
name_array = ["Maya", "Sarah", "Leen"]  
age_array = [12, 7, 14]
```

Instructions for creating two arrays one for each type of data.

```
CREATE New Dataset  
  Structure = Linear  
  Size = Fixed  
  Data Type = Numeric  
  Elements = [12, 7, 14]  
  Name = "age_array"  
  Insert = Not Allowed  
  Remove = Not Allowed
```

A hypothetical algorithm in pseudocode that would operate behind the scenes to create the 'age_array'.

Let's assume there was an error with Sarah's age—she is actually 9 years old, not 7. Although we cannot insert or remove data in an array, as that would change the array's size, we can swap out or modify the existing data items. In this case, we would simply update the value at the position corresponding to Sarah's age, correcting it from 7 to 9.

```
age_array = [12, 7, 14]
    UPDATE index (1) = 9
    PRINT age_array
END
```

OUTPUT:
12, 9, 14

The pseudocode instructions and the output for changing the value of the data held at position (1) in the array.

Note: Positions in a linear data structure are indexed, meaning each position is assigned a sequential number. Indexing typically starts at 0, so the first position in the structure has an index of 0.

THE LIST

A list is another linear data structure that, unlike an array, has a dynamic size and can accept a mixture of data types—numeric, alphanumeric, or both. Being dynamic in size means that it can change by inserting or removing data items. In a list, this insertion and removal of data items can occur at any position.

EXAMPLE 2.5

Jamal is Maya, Sarah, and Leen's 11-year-old brother. To add his name to the 'name_array' or his age to the 'age_array' is not possible, as that would change the size of the arrays. However, if these arrays were lists, then adding Jamal and his age would be possible.



Jamal is 11 years old.

```

name_list = ["Maya", "Sarah", "Leen"]
    INSERT index (2) = "Jamal"
    PRINT name_list
END

```

OUTPUT:
Maya, Sarah, Jamal, Leen

```

age_list = [12, 9, 14]
    INSERT index (2) = 11
    PRINT age_list
END

```

OUTPUT:
12, 9, 11, 14

Two sets of pseudocode instructions, each creating a list, inserting a new item at the third position (2), and then printing the list. Inserting a new item shifts the current items up by one position.

Note that each list contains only one data type, even though a list can simultaneously accept multiple data types.

If we want to sort the children in ascending order from youngest to oldest, we would need to shift the data items “Maya” and her age, 12, from their current positions (0) to the third position (2), currently occupied by “Jamal” and his age, 11.

```

age_list = [12, 9, 11, 14]
    REMOVE index (0) = 12
    INSERT index (2) = 12
    PRINT age_list
END

```

OUTPUT:
9, 11, 12, 14

```

name_list = ["Maya", "Sarah", "Jamal", "Leen"]
    REMOVE index (0) = "Maya"
    INSERT index (2) = "Maya"
    PRINT name_list
END

```

OUTPUT:
Sarah, Jamal, Maya, Leen

The two sets of pseudocode instructions show that we first sort the ‘age_list’ by changing the position of data item 12 from index (0) to (2). Then, the second set of instructions does the same with the name “Maya.” Now, both lists are sorted.

The last example, while it demonstrates how easily a list lends itself to the insertion and removal of data items, also reveals a limitation: when dealing with two related datasets, a list does not allow for associating between related data items, such as the name and age of each sibling. Even if we add both names and ages to one list, the related data items cannot be inherently associated and manipulated together.

THE TUPLE

The tuple is yet another linear data structure. It is similar to an array in that it has a fixed size, but it can accept different data types. The more interesting feature of a tuple is that it allows each position to be associated with specific data, effectively allowing related data items to be associated together.

EXAMPLE 2.6

Tuples can be used to combine the name and age of each sibling into one data unit with the format (name, age). Using such tuples would make the list more organized and make data manipulation easier. We have used brackets '[']' to define arrays and lists, whereas a tuple will be defined using parentheses '(')'.

```
sibling_list = [("Maya", 12), ("Sarah", 9), ("Jamal", 11), ("Leen", 14)]
PRINT sibling_list
END
```

OUTPUT:

Maya, 12, Sarah, 9, Jamal, 11, Leen, 14

Using tuples to organize related datasets in a list ensures that the related data items, such as the name and age of each sibling, follow each other. This method helps maintain the association between data points and makes it easier to manage and manipulate the data within the list.

Now, if we want to sort the children in ascending order from youngest to oldest, we would still need to shift the data items “Maya” and her age, 12, from their current positions (0) to the third position (2), currently occupied by “Jamal” and his age, 11. However, since the data is now organized by tuples, we simply need to move the entire tuple containing Maya’s data from its current position (0) to occupy the third position (2), which currently holds the tuple with Jamal’s data.

This approach makes the sorting process more straightforward, as related data points remain together within each tuple, reducing the complexity of shifting multiple individual data items.

```
sibling_list = [("Maya", 12), ("Sarah", 9), ("Jamal", 11), ("Leen", 14)]
REMOVE index (0) = ("Maya", 12)
INSERT index (2) = ("Maya", 12)
PRINT sibling_list
END
```

OUTPUT:

Sarah, 9, Jamal, 11, Maya, 12, Leen, 14

These instructions create a list of tuples, each formatted as (name, age). The data items in each tuple are fixed together in the order in which they were placed, with the name at index (0) and the age at index (1) of the tuple. The tuples are then placed in a list, with each tuple occupying one position in the list as if it were a single data item. So when Maya’s data needs to be moved, it is moved together as a tuple from its position at index (0) to its new position at index (2).

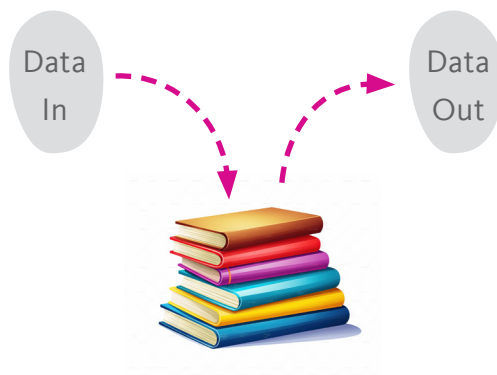
Notes

Creating a list of tuples in the same format is akin to creating a table, with each tuple acting as an entry on a row and the data items in each tuple placed according to their corresponding field or column.

THE STACK

The stack is similar to a list in that it has a dynamic size, meaning we can add to and remove items from it as needed, and it can accept data of different types, including both strings and numbers. The main difference is that in a stack, inserting and removing data items can only happen from the end of the stack. As a result, the last item added to the stack will be the first item to be removed; this is referred to as the Last In, First Out (LIFO) principle.

THE LIFO PRINCIPLE STATES THAT IN AN ORDERED SET OF ITEMS, THE LAST ITEM ADDED WILL BE THE FIRST ITEM REMOVED.



In a stack of books, the first book that can be removed is the last book that was added to the stack.

```
CREATE New Dataset
  Structure = Linear
  Size = Dynamic
  Data Type = Any
  Elements = ["A", "B", "C"]
  Name = "file_stack"
  Insert = To End
  Remove = From End
```

A hypothetical algorithm in pseudo-code that would operate behind the scenes to create a stack.

EXAMPLE 2.7

Lee works at a warehouse for an electronics store, where he maintains an up-to-date inventory of all products in stock. As products frequently move in and out of the warehouse, Lee must keep an accurate list of the current inventory.



One of the products in the warehouse is television sets. Each inbound TV is assigned a unique SKU code, which is entered into the warehouse computer system before the TV is placed on the shelf with other TVs waiting to be sold. The TVs are typically stacked, with the most recent arrival placed on top. When a client buys a TV, the one at the top of the stack is selected and shipped out to the client.

```
CREATE New Dataset
  Structure = Linear
  Size = Dynamic
  Data Type = Any
  Elements = []
  Name = "tv_stack"
  Insert = To End
  Remove = From End
```

OUTPUT:
tv_stack = []

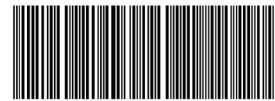
The hypothetical pseudocode instructions would create a stack, a linear data structure that can grow to any length and is assigned the name 'tv_stack'.

It is initially empty and does not contain any data elements. Insertion and removal of data is defined to occur only at the end of the stack.

```
IF tv = inbound THEN
  OPEN tv_stack
  READ tv_sku
  INSERT index (END) = tv_sku
  SEND tv TO shelf
END
```

OUTPUT:
tv_stack = [
 "TV-55S4K-BLK-342"
]

When the first TV arrives at the warehouse, the computer system checks to confirm it is inbound. It reads the SKU barcode, 'tv_sku', and adds it to the end of the stack, 'tv_stack', which in this case is the first position since no other TVs have entered the warehouse. The TV is then placed on a shelf.



SKU: TV-55S4K-BLK-342

OUTPUT:
tv_stack = [
 "TV-55S4K-BLK-342"
 "TV-55S4K-BLK-110"
 "TV-55S4K-BLK-729"
 "TV-55S4K-BLK-845"
]

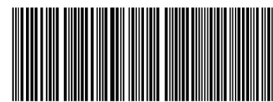
As more TVs arrive at the warehouse, the same process is repeated, and each new 'tv_sku' is added to the end of the stack 'tv_stack'.

Now that a client has purchased a TV, the warehouse needs to ship it out. The computer system receives the order, identifies the last TV set brought in, uses that TV's SKU to locate it on the shelf, and sends it for shipping to the client. This means the last TV set to arrive will be the first to go out, following the LIFO (Last In, First Out) principle.

OUTPUT:

```
tv_stack = [
    "TV-55S4K-BLK-342"
    "TV-55S4K-BLK-110"
    "TV-55S4K-BLK-729"
    "TV-55S4K-BLK-845"
]
```

The last 'tv_sku' is "TV-55S4K-BLK-845".



SKU: TV-55S4K-BLK-845

```
IF tv = outbound THEN
    OPEN tv_stack
    REMOVE index (END) = tv_sku
    FIND tv = tv_sku
    SEND tv TO client
END
```

OUTPUT:

```
tv_stack = [
    "TV-55S4K-BLK-342"
    "TV-55S4K-BLK-110"
    "TV-55S4K-BLK-729"
]
```

The computer system will open the 'tv_stack', check the last entry, remove that SKU, locate the TV with that SKU, and send it for delivery to the client.

Now, if another TV is sold, the next TV to leave the shelf will be the one with the SKU "TV-55S4K-BLK-729."

A data structure should be selected to match the nature of the data and the specific objectives for its use. When a real-world scenario operates under the Last In, First Out (LIFO) principle, a LIFO-based data structure, like a stack, is the most appropriate choice to effectively manage its data. In the electronics store warehouse, where the last product in is typically the first product out, the best data structure to manage the warehouse data is a stack.

THE QUEUE

The queue is similar to the stack and the list in that it has a dynamic size and can accept data of different types, including both strings and numbers. However, the difference lies in the rules that govern the insertion and removal of data. While in a list, these operations can occur at any position, and in a stack, they occur only at the end, the queue follows the First In, First Out (FIFO) principle. This means that the first item added to the queue will be the first item to be removed.

THE **FIFO PRINCIPLE** STATES THAT IN AN ORDERED SET OF ITEMS, THE FIRST ITEM ADDED WILL BE THE FIRST ITEM REMOVED.



In a queue of people at the grocery store, the first person to stand in line is the first person served at the counter. Similarly, in a queue of data, this implies that data can only be removed from the beginning and inserted at the end.

EXAMPLE 2.8

Jenna works at a warehouse for a grocery store. Unlike Lee, who works for an electronics store, many of the products Jenna handles are perishable and have a limited shelf life, such as apples, and should be shipped out as soon as possible before they go bad. Therefore, the first products that come in should be the first to go out.



A crate of apples

```
CREATE New Dataset
  Structure = Linear
  Size = Dynamic
  Data Type = Any
  Elements = [ ]
  Name = "apple_queue"
  Insert = To End
  Remove = From Beginning
```

OUTPUT:
apple_queue = []

```
IF apple_crate = inbound THEN
  OPEN apple_queue
  READ crate_sku
  INSERT index (END) = crate_sku
  SEND apple_crate TO shelf
END
```

OUTPUT:
apple_queue = [
 "APL-1025-20M-012"
]

OUTPUT:
apple_queue = [
 "APL-1025-20M-012"
 "APL-1025-20M-327"
 "APL-1025-20M-445"
 "APL-1025-20M-129"
]

The hypothetical pseudocode instructions would create a queue, a linear data structure that can grow to any length and is assigned the name `apple_queue`.

It is initially empty and does not contain any data elements. Insertion is defined to occur at the end, while removal occurs from the beginning of the queue.

When the first crate of apples arrives at the storage facility, the computer system checks to confirm it is inbound. It reads the SKU barcode on the crate, 'crate_sku', and adds it to the end of the queue, 'apple_queue', which in this case is the first position since this is the first inbound apple crate. The apple crate is then placed in a storage.



SKU: APL-1025-20M-012

As more apple crates arrive, the same process is repeated, and each new 'crate_sku' is added to the end of the queue 'apple_queue'.

Now that a client has purchased two crates of apples, they need to be shipped out. The computer system receives the order, identifies the first two apple crates that arrived, uses those crate SKUs to locate them in the storage area, and sends the crates for shipping to the client. This means the first two apple crates to arrive will be the first ones to go out, following the FIFO (First In, First Out) principle.

OUTPUT:

```
apple_queue = [
    "APL-1025-20M-012"
    "APL-1025-20M-327"
    "APL-1025-20M-445"
    "APL-1025-20M-129"
]
```

The first two 'crate_sku' codes are: "APL-1025-20M-012" and "APL-1025-20M-327".



SKU: APL-1025-20M-012



SKU: "APL-1025-20M-327"

```
IF apple_crate = outbound THEN
    OPEN apple_queue
    REMOVE index (END) = crate_sku
    FIND apple_crate = crate_sku
    SEND apple_crate TO client
END
```

OUTPUT:

```
apple_queue = [
    "APL-1025-20M-445"
    "APL-1025-20M-129"
]
```

The computer system will open the `apple\_queue`, check the first two entries, remove that SKU codes, locate the apple crates with those SKU codes, and send them for delivery to the client.

Now, if another apple crate is sold, the next apple crate to leave the storage will be the one with the SKU "APL-1025-20M-445".

At the electronics store, it was more efficient to remove the TV set on top of the stack to save time, rather than shifting all the TV sets to reach the bottom or the first one that came in. In this scenario, using a FIFO approach would waste time. However, with the apples, it was crucial to get the oldest apples out of storage first, as the longer they remain, the more likely they are to spoil; a LIFO approach would not have worked.

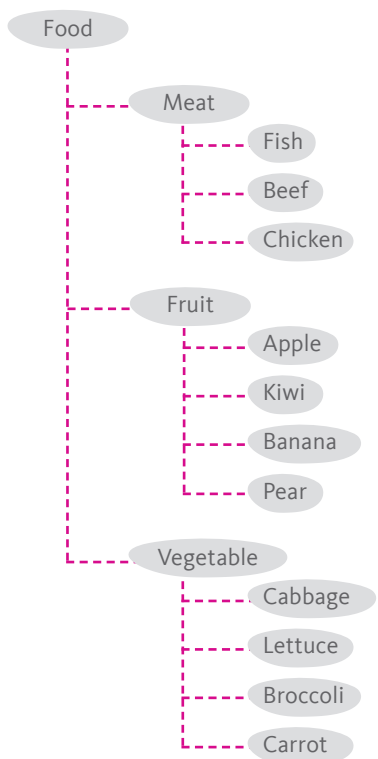
This illustrates that data structures are not just digital constructs but are essential tools designed to meet specific real-world needs. Choosing the right data structure depends on the context and the objectives, whether it's maximizing efficiency or ensuring the freshness of products.

TREE STRUCTURES

When data is structured using linear constructs, items are arranged in a sequence, with each item following the next. These structures are well-suited for representing real-world scenarios that require lists, stacks, and queues. However, the real world often presents more complex situations where data items need to be connected and organized in ways beyond a simple linear sequence.

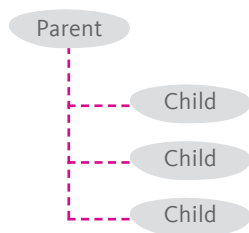
EXAMPLE 2.9

While the warehouse may rely on a stack or queue to efficiently manage inbound and outbound products, maintaining a full inventory of its stock requires a more complex data structure—one that can accommodate various categories and subcategories of products. The tree data structure is well-suited for this purpose, as it allows data to be organized hierarchically in levels that effectively support the categorization and subcategorization of items along branches.

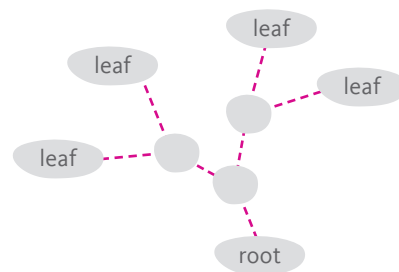


This is the organization chart for the food inventory at the warehouse where Jenna works. It is a tree structure with three hierarchical levels, each containing a category or subcategory of a food product.

In a tree structure, data items are referred to as nodes. A node functions as a data point that, in its most basic form, includes the data itself and additional information that points to other nodes to which this node is connected. Nodes that precede it are referred to as parent nodes, while nodes that follow are known as child nodes. The original parent node from which all other nodes originate is called the root node. Nodes at the very end of the tree structure, which do not have any child nodes, are referred to as leaf nodes.



A parent node with three child nodes.



A tree data structure with the root node and leaf nodes labeled.

A NODE IS A DATA CONSTRUCT THAT HOLDS BOTH DATA AND REFERENCES TO OTHER NODES.

Let's construct Jenna's warehouse food inventory as a tree data structure. The first step would be to create the file that would hold the dataset.

```

CREATE New Dataset
  Structure = Tree
  Size = Dynamic
  Data Type = Any
  Elements = []
  Node = (parent_index, data)
  Name = "food_inventory"
  Insert = any position
  Remove = any position
  
```

OUTPUT:
 food_inventory = []

These instructions create a dynamic file named food_inventory with no initial elements and allow data of any type to be inserted or removed from any position. The instructions also define the format of each node to contain the data and identify its parent nodes.

The instructions present a simplified method for defining a node. A tuple is used to contain the data, along with information about the parent nodes of the node being defined. It also employs the indexing method used in linear data structures to reference the position of each node within its hierarchical level and along its branch.

```
food_inventory = []
    INSERT node (R, "Food")
    PRINT food_inventory
END
```

OUTPUT:
Food

These instructions insert the root node, identified by the letter 'R', into an empty food_inventory tree and establish "Food" as the data associated with that node.

```
food_inventory = [(R, "Food")]
    INSERT node (R_0, "Meat")
    INSERT node (R_1, "Fruit")
    INSERT node (R_2, "Vegetable")
    PRINT food_inventory
END
```

OUTPUT:
Food
|____ Meat
|____ Fruit
|____ Vegetable

Three additional nodes are inserted as child nodes of the root, creating a new level in the hierarchy. Each node is indexed based on its position in this level, starting from 0.

```
food_inventory = [(R, "Food"),
    (R_0, "Meat"),
    (R_1, "Fruit"),
    (R_2, "Vegetable")
]
    INSERT node (R_1_0, "Apple")
    PRINT food_inventory
END
```

OUTPUT:
Food
|____ Meat
|____ Fruit
|____ |____ Apple
|____ Vegetable

These instructions further expand the food_inventory tree by adding a new child node under the existing "Fruit" node. The new node, labeled R_1_0 with the data "Apple," is inserted as a child of R_1 ("Fruit").

Removing data from a tree is straightforward when the data exists within a leaf node, meaning there are no other nodes dependent on it. However, if the data is within a parent node, removing it will likely result in all its dependent child nodes being deleted as well. This is because the child nodes rely on the parent node to maintain their position and relationship within the tree structure.

```
food_inventory = [(R,"Food"),
                  (R_0, "Meat"),
                  (R_1, "Fruit"),
                  (R_2, "Vegetable")
                  (R_1_0, "Apple"
                  ]
REMOVE node (R_1)
PRINT food_inventory
END
```

OUTPUT:

```
Food
|_____ Meat
|_____ Vegetable
```

The food_inventory tree initially contains nodes representing categories like “Meat,” “Fruit,” and “Vegetable.” When the instruction ‘REMOVE node (R_1)’ is executed, the “Fruit” node (R_1) is removed from the tree. As a result, all child nodes under “Fruit,” including “Apple” (R_1_o), are also deleted.

Nodes can change position in a tree as long as their child nodes are moved with them.

```
food_inventory = [(R,"Food"),
                  (R_0, "Meat"),
                  (R_1, "Fruit"),
                  (R_2, "Vegetable")
                  (R_1_0, "Apple"
                  ]
SWAP node (R_1),(R_2)
PRINT food_inventory
END
```

OUTPUT:

```
Food
|_____ Meat
|_____ Vegetable
|_____ Fruit
|           |_____ Apple
```

The instruction ‘SWAP node (R_1), (R_2)’ swaps the positions of the “Fruit” (R_1) and “Vegetable” (R_2) nodes. As a result, “Vegetable” will move into the position previously occupied by “Fruit,” and vice versa, while all their respective child nodes (such as “Apple” under “Fruit”) are moved along with them, maintaining the hierarchical integrity of the tree structure.

GRAPH STRUCTURES

In a tree, data is placed in nodes that are arranged along branches. Nodes can only connect to other nodes along their branch; they cannot directly connect to nodes in different branches. In contrast, in a graph, data is also placed in nodes, but the connections between nodes are not restricted. Nodes can interconnect freely, allowing for more complex relationships between data points.

In a tree data structure, we gave nodes a simplistic format that contains a reference to parent nodes and the data itself (`parent_index`, `data`). In a graph data structure, nodes have more complex connections, so a node will need to contain more information about linked nodes. It includes a list of all the indices of linked nodes, formatted as (`node_index`, `data`, `linked_node1`, `linked_node2`, ...).

EXAMPLE 2.10

Now that Jenna has used a queue to organize inbound and outbound food products, ensuring their freshness by minimizing shelf time, and has utilized a tree to create an inventory for all products in the warehouse, she now needs to manage the delivery process from the warehouse to her clients. For this task, she will need a graph to represent the various client destinations on the map.



Jenna's warehouse is in Seville, in the south of Spain. Her clients are in the cities of Madrid, Barcelona, and Valencia. Each of these four cities can be represented as a node in a graph data structure, and the roads connecting the cities can be represented as links between the nodes of the graph.

Just like with any data structure that uses nodes, nodes in a graph must be indexed. Typically, indexing is done using numbers, but that's not always the case—characters such as letters can also be used, as seen in the previous example where the letter R was used to refer to the root node. In a graph structure, there is no root node, and in fact, any node can act as a root. In this example, we will use letters to index the nodes. The letters A, B, C, and D will refer to the nodes representing the cities of Seville, Madrid, Barcelona, and Valencia, respectively.

One method to approach constructing a graph is by creating a list of each node and every node connected to it, known as an adjacency list. In this example, each node represents a city, and the cities are connected by roads. For the sake of this example, we will assume that the road between Madrid and Valencia is still under construction and cannot be used or included.

Adjacency List using City Names:

Seville is connected to: Madrid, Barcelona, and Valencia
Madrid is connected to: Seville and Barcelona
Barcelona is connected to: Seville, Madrid, and Valencia
Valencia is connected to: Seville and Barcelona

Adjacency List using Indices:

A: [B, C, D]
B: [A, C]
C: [A, B, D]
D: [A, C]

But graphs can carry more information—not just the data at each node, such as the name of the city, but also details about the links between nodes, referred to as ‘edges’. In this example, the edges between nodes could represent data such as the distance between cities. This additional information allows the graph to model real-world scenarios more accurately, where the relationships between entities (like cities) are as important as the entities themselves.

IN A GRAPH, AN **EDGE** IS A LINK BETWEEN TWO NODES THAT CARRIES ADDITIONAL INFORMATION ABOUT THEIR RELATIONSHIP.

Distances between cities:

Seville - Madrid: 538 km
 Seville - Barcelona: 1,031 km
 Seville - Valencia: 653 km
 Madrid - Barcelona: 621 km
 Valencia - Barcelona: 351 km

Numerical Value Assigned to Each Edge:

AB, BA = 538
 AC, CA = 1,031
 AD, DA = 653
 BC, CB = 621
 CD, DC = 351

This new data about the distances between cities will be included as additional information regarding the edge between each node. Accordingly, the adjacency list will be updated to include the edge data.

Updated Adjacency List:

A: [(B, 538), (C, 1,031), (D, 653)]
 B: [(A, 538), (C, 1,031)]
 C: [(A, 1,031), (B, 621), (D, 351)]
 D: [(A, 653), (C, 351)]

To construct Jenna's delivery map as a graph data structure, the first step is to create the file that will hold the dataset.

```
CREATE New Dataset
  Structure = Graph
  Size = Dynamic
  Data Type = Any
  Elements = []
  Node = (node_index, data, (linked_node, edge))
  Name = "delivery_map"
  Insert = any position
  Remove = any position
```

OUTPUT:
 delivery_map = []

These instructions create a dynamic file named delivery_map with no initial elements and allow data of any type to be inserted or removed from any position. The instructions also define the format of each node to contain the data and identify its connected nodes and their edges.

Once the graph has been created, the first node can be inserted. Since there are no other nodes to link to, the first node would be defined as (A, “Seville”), with A as the index of the node and “Seville” as the data it contains.

```
delivery_map = []
    INSERT node (A,“Seville”)
END
```

OUTPUT:
`delivery_map = [(A,“Seville”)]`

These instructions insert the first node, identified by the letter ‘A’, into an empty `delivery_map` graph and associate the city name “Seville” with that node.

Since there are no other nodes yet, no additional data, such as node indices or edge information, can be included at this stage.

The next step is to insert the remaining three nodes that represent the cities of Madrid, Valencia, and Barcelona, along with the distances between them as values for the edges connecting the nodes.

```
delivery_map = [(A,“Seville”)]
    INSERT node (B,“Madrid”,(A,538))
    INSERT node (C,“Barcelona”,(A,1031))
    INSERT node (D,“Valencia”,(A,653))
END
```

OUTPUT:
`delivery_map = [
 (A,“Seville”,(B,538),(C,1031),(D,653)),
 (B,“Madrid”,(A,538))
 (C,“Barcelona”,(A,1031))
 (D,“Valencia”,(A,653))
]`

With the insertion of the three nodes “Madrid” (B), “Barcelona” (C), and “Valencia” (D), the graph now includes all four nodes, with each node identifying the nodes it is linked to and their corresponding edges.

Note that the A node is automatically updated to include these connections, reflecting the distances to each city.

If Jenna wants to consider the distance her delivery truck needs to travel to deliver to her clients in Madrid, Barcelona, and Valencia, she can have the computer system perform the necessary calculations now that the dataset has been defined.

```

delivery_map = [
    (A, "Seville", (B, 538), (C, 1031), (D, 653)),
    (B, "Madrid", (A, 538))
    (C, "Barcelona", (A, 1031))
    (D, "Valencia", (A, 653))
]
Trip_1 = AB + BA
Trip_2 = AC + CA
Trip_3 = AD + DA
Trip_All = Trip_1 + Trip_2 + Trip_3
PRINT Trip_All
END

```

OUTPUT:
4444

These instructions calculate the total distance Jenna's delivery truck would travel to visit each client in Madrid, Barcelona, and Valencia, and then return to Seville.

The distances for each round trip (Seville to Madrid and back, Seville to Barcelona and back, and Seville to Valencia and back) are summed, resulting in a total trip distance of 4,444 km, which is then printed as the final output.

Jenna realized that the total distance of 4,444 km is quite long. She decided to consider an alternative delivery route that started in Seville, passed through each of the three delivery destinations—Madrid, Barcelona, and Valencia—before returning to the warehouse in Seville. To calculate this route, she would need to include the distances between Madrid and Barcelona, as well as between Barcelona and Valencia.

```

delivery_map = [
    (A, "Seville", (B, 538), (C, 1031), (D, 653)),
    (B, "Madrid", (A, 538))
    (C, "Barcelona", (A, 1031))
    (D, "Valencia", (A, 653))
]
INSERT edge (B:C, 621)
INSERT edge (C:D, 351)
PRINT delivery_map
END

```

OUTPUT:

```

delivery_map = [
    (A, "Seville", (B, 538), (C, 1031), (D, 653)),
    (B, "Madrid", (A, 538), (C, 621)),
    (C, "Barcelona", (A, 1031), (B, 621), (D, 351)),
    (D, "Valencia", (A, 653), (C, 351))
]

```

The instructions update the delivery_map by inserting edges between Madrid (B) and Barcelona (C) with a distance of 621 km, and between Barcelona (C) and Valencia (D) with a distance of 351 km.

Now that the required data has been entered into the graph, Jenna can have the distance for the alternate route calculated.

```
delivery_map = [  
    (A, "Seville", (B, 538), (C, 1031), (D, 653)),  
    (B, "Madrid", (A, 538), (C, 621)),  
    (C, "Barcelona", (A, 1031), (B, 621), (D, 351)),  
    (D, "Valencia", (A, 653), (C, 351))  
]  
Trip_All = AB + BC + CD + DA  
PRINT Trip_All  
END
```

OUTPUT:
2163

The instructions calculate the distance of the alternate route by summing the values of the edges between each of the four nodes: AB, BC, CD, and DA. The calculated distance for this route is 2,163 km, significantly less than the initial route, making it a more efficient option for deliveries.

The use of a graph helped organize the data in a way that allowed for the calculations Jenna needed to make an informed decision on which route to take. At the warehouse, Jenna used a linear queue to manage inbound and outbound products, a tree to manage inventory, and a graph to manage delivery routes. These simple examples illustrate how data structures are crucial in the process of converting data into actionable information.

CONCLUSION

In conclusion, this lesson and the examples provided emphasize that the selection of a data structure is not arbitrary; it must be carefully aligned with the nature of the data and the specific requirements of the task at hand. Linear structures, such as arrays, lists, and stacks, are straightforward and efficient for managing sequential data, while trees and graphs offer the complexity needed to handle more complex relationships, such as hierarchical or interconnected data. Therefore, it is crucial to select the most appropriate structure to accurately represent and manage the underlying data.

SUMMARY

Data structures are specialized formats for organizing, retrieving, modifying, processing, and storing data.

Algorithms control these data structures by defining the rules for size, data type, and methods for inserting, removing, and accessing items.

Data structures are broadly classified into linear (array, list, tuple, stack, and queue) and non-linear (tree and graph) categories.

In linear data structures, data is arranged sequentially.

- Array: Fixed size, accepts one data type, and does not allow the insertion or removal of data after creation.
- List: Dynamic size, accepts mixed data types, and allows insertion or removal of data from any position.
- Tuple: Fixed size and format, accepts mixed data types, but does not allow the insertion or removal of data after creation.
- Stack: Dynamic size, accepts mixed data types, and allows data to be added or removed only from the end, following the Last In, First Out (LIFO) principle.
- Queue: Dynamic size, accepts mixed data types, allows data to be added at the end and removed from the beginning, following the First In, First Out (FIFO) principle.

In non-linear data structures like trees and graphs, data is held in nodes that link data together.

- Tree: Nodes are linked to form a hierarchical structure of parent-child relationships arranged in branches.
- Graph: Nodes can connect freely, allowing more complex relationships between data points.

A node is a data construct that contains both data and references to other nodes, making it essential in organizing and navigating data in non-linear structures.

In a graph, an edge is a link between two nodes that carries additional information about their relationship.

Activity 2.1

Read the following statements, and write on the line provided for each statement 'T' if it is true and 'F' if it is false.

T/F	STATEMENT
<u>F</u>	An array can hold multiple data types within the same structure.
<u>T</u>	A list has a dynamic size, meaning you can add or remove items from it.
<u>F</u>	A stack follows the First In, First Out (FIFO) principle.
<u>T</u>	In a tree structure, the root node is the node from which all other nodes descend.
<u>F</u>	A queue allows data to be added at any position and removed from any position.
<u>T</u>	In a graph, an edge represents a connection between two nodes and can carry additional information.
<u>F</u>	Tuples can have a dynamic size and allow for the addition of new data after creation.
<u>F</u>	Nodes in a tree can connect directly to nodes in different branches.
<u>T</u>	The Last In, First Out (LIFO) principle is applied when using a stack data structure.
<u>F</u>	A graph structure is more suitable than a tree structure for representing hierarchical data.

Activity 2.2

Answer the questions below.

1. Which data structure follows the Last In, First Out (LIFO) principle?
 - a) Queue
 - b) **Stack**
 - c) List
 - d) Array
2. Which is a characteristic of an array?
 - a) It can hold multiple data types
 - b) It has a dynamic size
 - c) **It can only hold one data type**
 - d) It allows insertion and deletion of data at any position
3. In a tree structure, what is the topmost node called?
 - a) Leaf node
 - b) Child node
 - c) **Root node**
 - d) Parent node
4. Which data structure is most suitable for managing hierarchical data?
 - a) **Tree**
 - b) Graph
 - c) Array
 - d) Stack
5. What is the primary feature of a tuple?
 - a) It has a dynamic size
 - b) It follows the First In, First Out (FIFO) principle
 - c) It can be modified after creation
 - d) **It has a fixed format**

Activity 2.3

Answer the questions below.

1. Which data structure follows the First In, First Out (FIFO) principle?
 - a) Stack
 - b) Queue
 - c) Tuple
 - d) List
2. What does an edge represent in a graph?
 - a) The data stored in a node
 - b) The hierarchical level of a node
 - c) The connection between two nodes
 - d) The index of a node
3. Which of these data structures can freely connect nodes?
 - a) Tree
 - b) Graph
 - c) Stack
 - d) Queue
4. Which data structure is best suited for representing a subway network?
 - a) Tree
 - b) Array
 - c) Stack
 - d) Graph
5. What happens when a node with child nodes is removed from a tree structure?
 - a) The tree collapses.
 - b) The child nodes become root nodes.
 - c) The tree becomes a graph.
 - d) The child nodes are automatically removed.

Activity 2.4

Match each data structure with the statement that best describes it.

Queue	Graph	Stack
Tree	Tuple	List

1. A fixed-size, ordered collection of the same data type. List
2. A hierarchical structure where each element is a node. Tree
3. A structure that follows the FIFO principle. Queue
4. It has a fixed size and fixed format. Tuple
5. A structure that follows the LIFO principle. Stack
6. A flexible structure where nodes are connected by edges. Graph

Activity 2.5

Match each data structure with the task for which it is best optimized.

Tree	Array	Tuple
Queue	Stack	Graph

1. Handling tasks in the order they arrive, like print jobs. Queue
2. Modeling networks, like social connections. Graph
3. Organizing a file system, like a computer's directory. Tree
4. Storing the coordinates of a point in a 2D space as (x, y). Tuple
5. Storing a city's monthly average temperatures for a year. Array
6. Managing undo operations in a software application. Stack

Activity 2.6

Answer the questions below about the following dataset, which represents the average monthly temperatures (in °C) for a city over a year.

```
monthly_temp = [2.5, 3.0, 7.8, 12.5, 16.4, 20.1, 23.3, 22.8, 18.5, 13.2, 8.3, 4.1]
```

1. How many data items are in the array? 12
2. What type of data is in the array? numeric
3. What data item is in the fourth position? 12.5
4. What was the temperature in September? 18.5
5. What would be the output of this instruction? 8.3

```
PRINT index (10)
```

6. Assume there was an error with the average temperature for February—it is actually 3.2°C rather than 3.0°C. Review the following instructions and determine which ones can be implemented and which cannot. Explain.

```
REMOVE index (1) = 3.0  
INSERT index (1) = 3.2
```

```
UPDATE index (1) = 3.2
```

The remove and insert instructions cannot be implemented because they
would change the size of the array, which is not allowed. However, updating
the value held at a particular position is allowed as it does not affect
the size of the array.

Activity 2.7

Answer the questions below about the following dataset, which represents a grocery shopping list.

```
grocery_list = ["Apples", "Bread", "Milk", "Eggs", "Cheese", "Tomatoes"]
```

- 1. How many data items are in the list? 6
- 2. What type of data is in the list? alphanumeric
- 3. What data item is in the third position? Milk
- 4. What is the index of the data item "Cheese"? 4

5. Fill in the blanks with the data items from the list and write the index beneath each.

<u>Apples</u>	<u>Bread</u>	<u>Milk</u>	<u>Eggs</u>	<u>Cheese</u>	<u>Tomatoes</u>
<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	<u>5</u>

6. Assume the eggs were removed from the grocery cart. To modify the list, you must:
- a) Remove all data items before and including index (3).
 - b) Remove all data items after and including index (3).
 - c) Remove the data item at index (3).
- Update the list accordingly.

<u>Apples</u>	<u>Bread</u>	<u>Milk</u>	<u>Cheese</u>	<u>Tomatoes</u>
<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>

7. Pears were added to the grocery cart. Write a pseudocode instruction that will add "Pears" to the list to follow "Apples". Then update the list.

INSERT index (1) = "Pears"

<u>Apples</u>	<u>Pears</u>	<u>Bread</u>	<u>Milk</u>	<u>Cheese</u>	<u>Tomatoes</u>
<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	

Activity 2.8

Answer the questions below about the undo stack of a text editor.

```
undo_stack = [
    "Italicize 'World'",
    "Type 'World'",
    "Bold 'Hello'",
    "Type 'Hello'"
]
```

1. How many actions were taken by the user? 4
2. What was the first action taken by the user? Type 'Hello'
3. What was the last action taken by the user? Italicize 'World'
4. What action is held at index (1)? Type 'World'
5. To undo "Bold 'Hello'", which actions must be undone? Write them in the order in which they are removed from the stack.

The last action taken will be removed first: "Italicize 'World'".

The next action to be removed is: "Type 'World'".

Finally, after removing the previous two actions, you can undo "Bold 'Hello'".

6. How many are left in the stack? 1
7. What is the item left in the stack? Type 'Hello'
8. The user now performs the following actions in this order: italicizes 'Hello', types 'World', italicizes 'World', underlines 'Hello World'. Rewrite the items in the order in which they would appear in the stack. Indicate the index of each.

Index (0) - Underlines 'Hello World'

Index (1) - Italicizes 'World'

Index (2) - Types 'World'

Index (3) - Italicizes 'Hello'

Index (4) - Type 'Hello'

Activity 2.9

Answer the questions below about the queue of customers waiting to speak with a customer service representative.

```
customer_queue = ["Maya", "Lee", "Chen", "Sara", "Omar"]
```

1. How many customers are in the queue? 5
2. Who was the first customer to enter the queue? Maya
3. Who was the last customer to enter the queue? Omar
4. Which customer will be the first to be served? Maya
5. From which index will customer names leave the queue? 0
6. Write the updated queue after the next customer is served.
customer_queue = ["Lee", "Chen", "Sara", "Omar"]
7. Which customer will be served next? Lee
8. Write the updated queue after the Lee is served.
customer_queue = ["Chen", "Sara", "Omar"]
9. Jenna joins the queue. Write the updated queue.
customer_queue = ["Chen", "Sara", "Omar", "Jenna"]
10. At which index is "Jenna"? 3
11. Read the following instruction and determine how it would be handled.

```
INSERT index (1) = "Mike"
```

A queue operates based on the FIFO principle, so any new data must be added at the end. This instruction will result in an error and cannot be carried out.

Activity 2.10

Answer the questions about the following list of tuples.

```
student_grades = [
    ("Maya", 85),
    ("Lee", 92),
    ("Sara", 78),
]
```

1. How many tuples are in the list? 3
2. What format does each tuple have? (name, grade)
3. Which tuple is at index (0)? ("Maya", 85)
4. What is Sara's grade? 78
5. Read the following instruction and update the list accordingly.

```
INSERT index (1) = ("Mike", 83)
```

student_grades = [("Maya", 85), ("Mike", 83), ("Lee", 92), ("Sara", 78)]

6. Assume there was an error entering Lee's grade. To modify the data, you must:
 - a) Swap the incorrect grade with the correct one inside the tuple at index (2).
 - b) Remove the tuple at index (2) and insert a corrected tuple at index (2).
 - c) Remove all tuples before index (2) before inserting the correct tuple.

Lee's correct grade is 91. Write the pseudocode instructions that will carry out this correction.

REMOVE index (2) = ("Lee", 92)

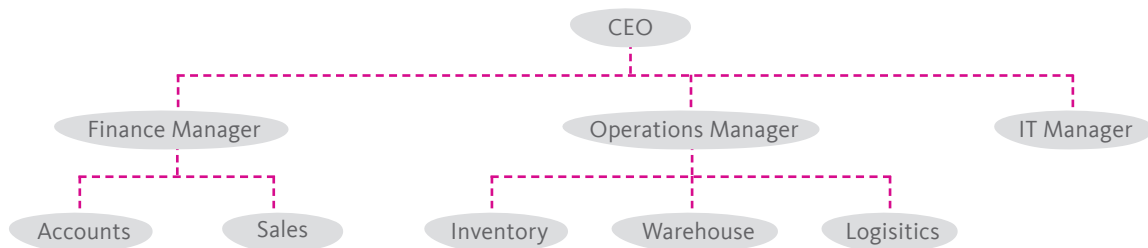
INSERT index (2) = ("Lee", 91)

Write the corrected list.

student_grades = [("Maya", 85), ("Mike", 83), ("Lee", 91), ("Sara", 78)]

Activity 2.11

Answer the questions below about the organization chart of the company where Jenna works.



1. Which data structure is best suited for this chart? Tree
2. How many hierarchical levels are in this chart? 3
3. Which data item is at the root node? CEO
4. How many child nodes does the root node have? 3
5. How many child nodes does the 'Finance Manager' have? 2
6. How many leaf nodes are there in total? 6
7. What are the leaf nodes under 'Finance Manager'? Accounts, Sales
8. Is 'IT Manager' a leaf node or a parent node? Leaf node
9. Using pseudocode, write the tree data structure for the first two levels of the hierarchy. Name it 'org_chart'.

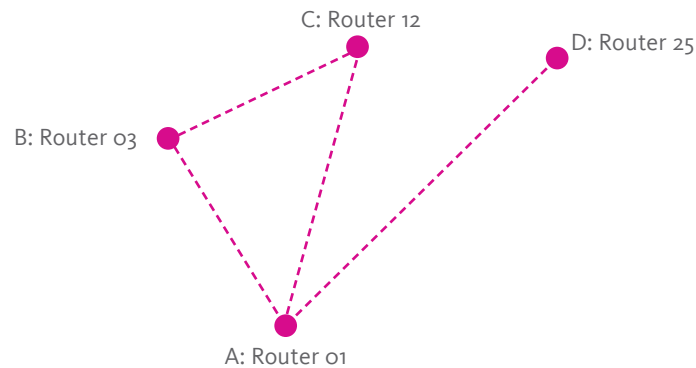
org_chart = [(R, "CEO"), (R_o, "Finance Manager"), (R_1, "Operations Manager"),
(R_2, "IT Manager")]

10. Write a pseudocode instruction that will add 'Marketing' to follow 'Sales' under 'Finance Manager'.

INSERT node (R_o_2, "Marketing")

Activity 2.12

Answer the questions about the following network of routers.



1. What type of data structure best represents this network? Graph
2. How many nodes are in this graph? 4
3. What is the name of the router designated as node B? Router 03
4. What is the node index of "Router 12"? C
5. Using node indices, write an adjacency list for this graph.

A: [B, C, D]
B: [A, C]
C: [A, B]
D: [A]
6. If the time it takes for a signal to travel between nodes (latency) is: A to B 12ms, A to C 18ms, A to D 25ms, and B to C 9ms, rewrite the adjacency list to include latency data as edges.

A: [(B, 12ms), (C, 18ms), (D, 25ms)]
B: [(A, 12ms), (C, 9ms)]
C: [(A, 18ms), (B, 9ms)]
D: [(A, 25ms)]

LESSON 3: SEARCHING LINEAR DATA

One of the fundamental operations on data is searching for a specific item within a dataset. Before we can modify, extract, relocate, or process any data item, we must first locate it accurately. Searching, like all computer operations, is defined and guided by algorithms, each with its own performance characteristics.

In this lesson, we will explore the mechanics and applications of basic search algorithms—linear search and binary search—and compare their performance. Through this comparison, we'll understand how choosing the right algorithm can make a significant difference in efficiency, particularly when dealing with large datasets organized in a linear format.

LINEAR SEARCH

One of the simplest strategies is known as linear search. It is a straightforward approach applied to linear data structures, where the algorithm starts at the first position in the dataset and moves sequentially from one position to the next, checking each data item for a match. If a data item matches the search item, the search ends. Otherwise, it continues until the end of the dataset.

EXAMPLE 3.1

Let's go back to the example of books on a shelf. To search for a particular title using a linear search approach, you would start at the beginning of the shelf and check each book title in order until you find the one you're looking for. However, if you reach the end of the shelf without finding the title, then the book is not on this shelf.

Using a linear search, books on a bookshelf are checked one by one from left to right across the shelf.

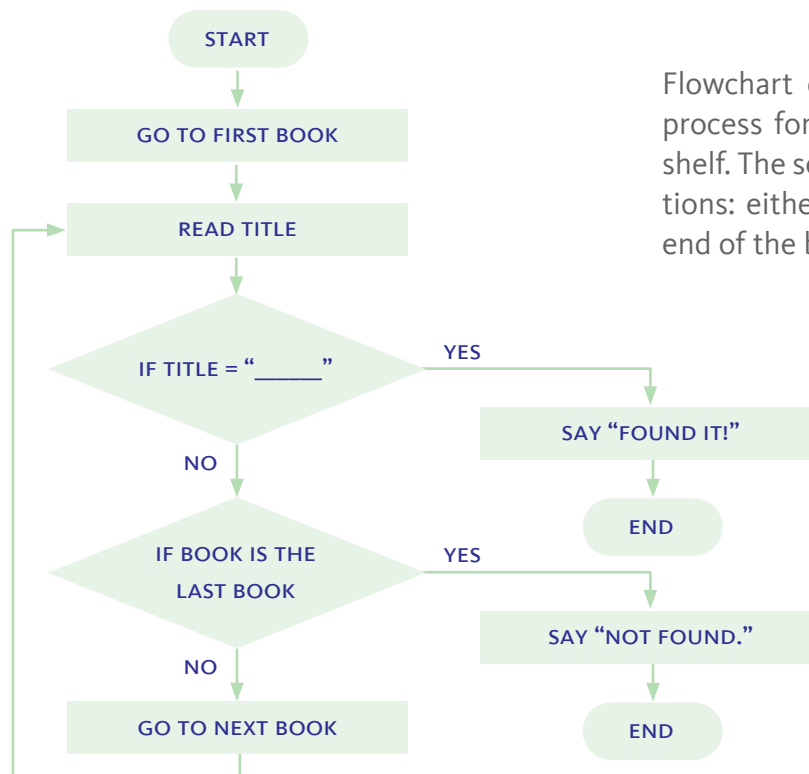


A LINEAR SEARCH ALGORITHM SEQUENTIALLY CHECKS EACH POSITION IN A DATASET.

THE LINEAR SEARCH PROCESS

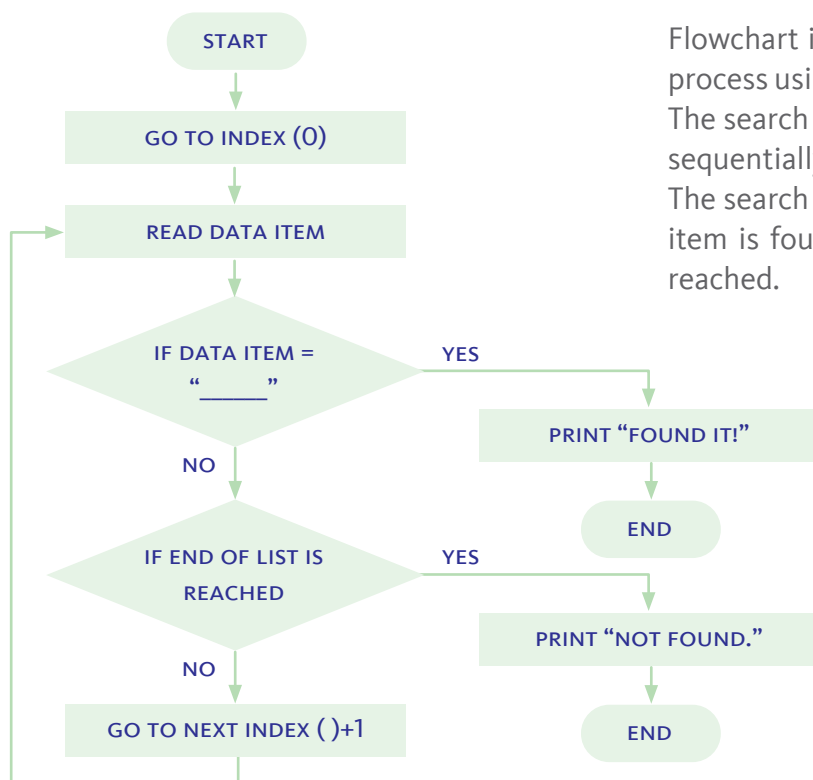
While the process of searching for an item, such as a book on a bookshelf or a toy in a toy basket, may come intuitively, breaking it down into discrete steps—including those required for decision-making—is essential. These steps include deciding to end the search once you’ve found what you’re looking for, or concluding that the item is not on the bookshelf or in the toy basket.

When dealing with any step-by-step process, it is best understood when visually depicted in the form of a flowchart. A well-structured flowchart includes each individual step of the process, a clear flow from one step to the next, and decision points that indicate the path of flow based on each condition that is met.



Flowchart depicting the linear search process for finding a book on a bookshelf. The search ends under two conditions: either the book is found, or the end of the bookshelf is reached.

Now that the basic process of the linear search strategy has been established—even if only for books—we can extrapolate the structure of an algorithm that can be applied to any list of items. Simply described, the algorithm must start at the beginning of the list, sequentially check each item, and end the search if one of two conditions is satisfied: the item is found, or the end of the list is reached.



Flowchart illustrating the linear search process using an index-based approach. The search begins at the first index and sequentially checks each data item. The search ends when either the target item is found, or the end of the list is reached.

Achieving a clear understanding of the process is essential before creating a program that a computer can execute, following the steps of the linear search algorithm. Breaking down the linear search into its fundamental components not only clarifies each action but also prepares us to translate these actions into code.

Next, we will present the linear search algorithm in pseudocode form. This pseudocode provides a step-by-step outline of the process. Each instruction

represents a specific action that the computer will execute, transforming the logical steps of the search process into a precise, executable program.

```
START LinearSearch
OPEN list
SET search_index (0)

REPEAT_UNTIL search_index reaches end of list
  READ data_item AT: search_index

  IF data_item = target THEN
    PRINT "Found it!"
  END LinearSearch
ELSE
  SET search_index (search_index + 1)

PRINT "Not found"
END LinearSearch
```

SET search_index (o): Initializes the starting position of the search to the first index (o), marking the beginning of the list.

REPEAT_UNTIL: Creates a loop that continues until the end of the list is reached. The condition for ending the search upon reaching the last position in the list is embedded within this instruction.

Within the loop, the algorithm **READs** each data item and ends the search **IF** the data item matches the target being searched for.

SET search_index (search_index + 1): Ensures that the search progresses by moving to the next position in the list, incrementing search_index by 1 with each iteration.

The strategy of searching sequentially through a list is straightforward and would work for any list. However, as the size of the list grows, challenges arise. When there are only 10 books on a shelf, checking each one is manageable, but what if the search involves an entire library with hundreds or even thousands of books?

PERFORMANCE

Let's explore the performance of a linear search algorithm. One simple measure of performance is execution time, which refers to the amount of time it takes for the program to complete its run.

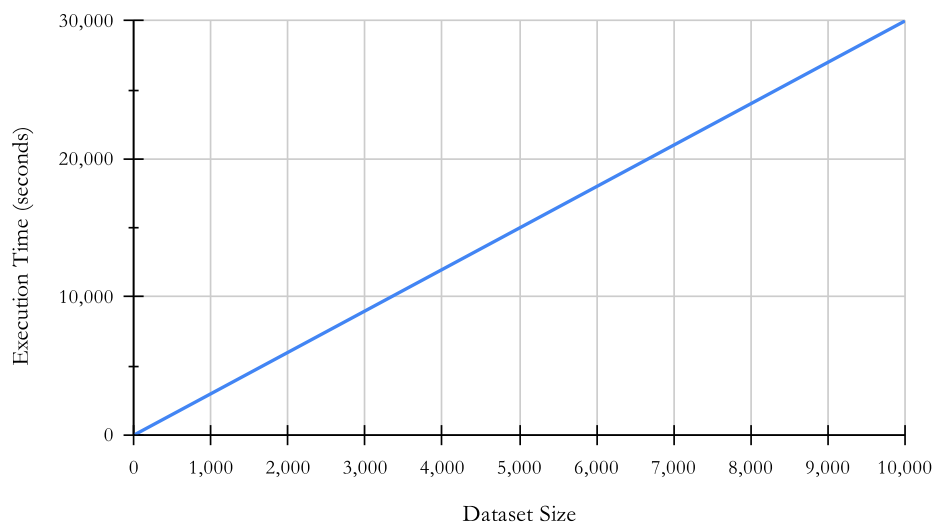
EXAMPLE 3.2

Referring back to the books on the bookshelf, let's assume it takes approximately 3 seconds for a person to check each book, decide whether they've found what they're looking for, or move on to the next book on the shelf. We'll also assume that this person can perform this task tirelessly, without breaks or errors. Since it takes 3 seconds to check each book, the time to search n books would be:

$$\text{Execution Time} = \text{Time per Attempt} \times \text{Number of Attempts}$$

Size of Dataset	Execution Time
10 books	$3 \times 10 = 30 \text{ seconds}$
100 books	$3 \times 100 = 300 \text{ seconds (5 minutes)}$
1,000 books	$3 \times 1,000 = 3,000 \text{ seconds (50 minutes)}$
10,000 books	$3 \times 10,000 = 30,000 \text{ seconds (8 hours 20 minutes)}$

Linear Search Algorithm: Execution Time vs. Dataset Size



The graph illustrates how execution time increases in direct proportion to the size of the dataset when using a linear search algorithm. Each additional book requires a constant amount of time (3 seconds) to check, resulting in a straight-line growth pattern.

WORST-CASE

To best understand the performance of an algorithm, we need to evaluate it in its worst-case scenario. For a search algorithm, this means assuming it has to examine every item in the list before finding the target (or concluding that the target isn't present). This contrasts with a best-case scenario, where the algorithm finds its target on the very first attempt. For a bookshelf, the best case would mean that the target book is the very first one checked.

A WORST-CASE SCENARIO IS THE SITUATION IN WHICH AN ALGORITHM TAKES THE LONGEST POSSIBLE TIME TO ACHIEVE ITS GOAL.

In the previous example, the execution time calculations are based on this worst-case scenario for our linear search, where the algorithm wasn't fortunate enough to find the book among the first few items and had to search all the way to the end of the shelf. This worst-case analysis helps us understand the maximum time our linear search algorithm might require under the least favorable conditions.

Notes

The concepts of algorithm performance in this section are approached with a focus solely on time complexity. This helps students concentrate on understanding how execution time scales with input size before introducing additional factors like space complexity.

Example 3.2 demonstrates that as the dataset size increases, the time required for the search also increases at a constant rate of 3 seconds per item. For instance, searching through 10 books takes about 30 seconds, while searching through 10,000 books takes 30,000 seconds. This reveals a key limitation of the linear search algorithm: for large datasets, the search time becomes impractically long.

EXAMPLE 3.3

Global telephone operators may manage databases with hundreds of millions of telephone numbers. Let's assume one such operator manages 100 million numbers. Further, let's assume its computer systems have a 50-nanosecond latency, meaning it takes the processor about 50 nanoseconds to access memory. A call comes in from a client, and the customer service representative wants to search for this client's account. The execution time for a linear search would be:

$$\text{Execution Time} = 50 \times 100,000,000 = 5,000,000,000 \text{ ns (5 seconds)}$$

A 5-second execution time for a single search in a call center could have significant repercussions for the operator's business. In a call center, where efficiency is crucial, this delay would slow down interactions, leading to longer wait times and potentially frustrating customers.

But the impact extends beyond customer service. Telecom operators rely on real-time systems to route calls, authenticate users, and manage billing. A 5-second delay in verifying or routing each call could disrupt these processes, resulting in dropped calls, misrouted calls, or costly billing errors. Moreover, as the operator's client base grows, so will the delays and related issues, emphasizing the need for more efficient search solutions to maintain smooth, reliable service.

BINARY SEARCH

What if we could use a strategy that reduces the size of the list as the search progresses? The binary search algorithm does exactly that. Instead of starting at the beginning of the list, binary search begins at the middle index. If the target is less than the value of the data item at the middle index, the algorithm knows the target must be in the first half of the list, so it disregards the second half. It then focuses on the middle index of the remaining portion of the list, checking the value there. Based on this comparison, it determines which half of the list to continue searching and which to disregard. This process repeats until the target is found or the list is fully divided, essentially halving the dataset with every iteration.

An important condition for the binary search algorithm to work is that the list must be sorted; otherwise, this strategy would not be effective. While a linear search algorithm works equally well on both sorted and unsorted lists, the binary search algorithm requires a sorted list to function properly. Without sorting, binary search would not be able to accurately determine which half of the list to focus on, making the algorithm ineffective.

A BINARY SEARCH ALGORITHM FINDS A TARGET VALUE IN A SORTED LIST BY REPEATEDLY DIVIDING THE SEARCH RANGE IN HALF.

EXAMPLE 3.4

Let's go back to our books on the bookshelf. There are 12 books on the shelf, and this time, we'll assume the books are sorted in alphabetical order. In a linear search, the worst-case scenario would require the algorithm to check all 12 books. Now, let's see what happens in the worst-case scenario for a binary search algorithm.

The 12 books on the shelf are sorted in alphabetical order.



In order to make the example more tangible, here is a list of all twelve book titles arranged in alphabetical order. In the worst-case scenario, the target book would be at the final possible location after all divisions, requiring multiple attempts to reach it. In this case, our target title is 'Zebra Crossing'. Here's how the binary search would proceed:

```
book_titles = [  
    "A Tale",  
    "Blue Moon",  
    "Crimson Sky",  
    "Deep Waters",  
    "Endless Night",  
    "Frost Fire",  
    "Golden Path",  
    "Hidden World",  
    "Iron Fist",  
    "Jungle King",  
    "Silver Light",  
    "Zebra Crossing"  
]
```

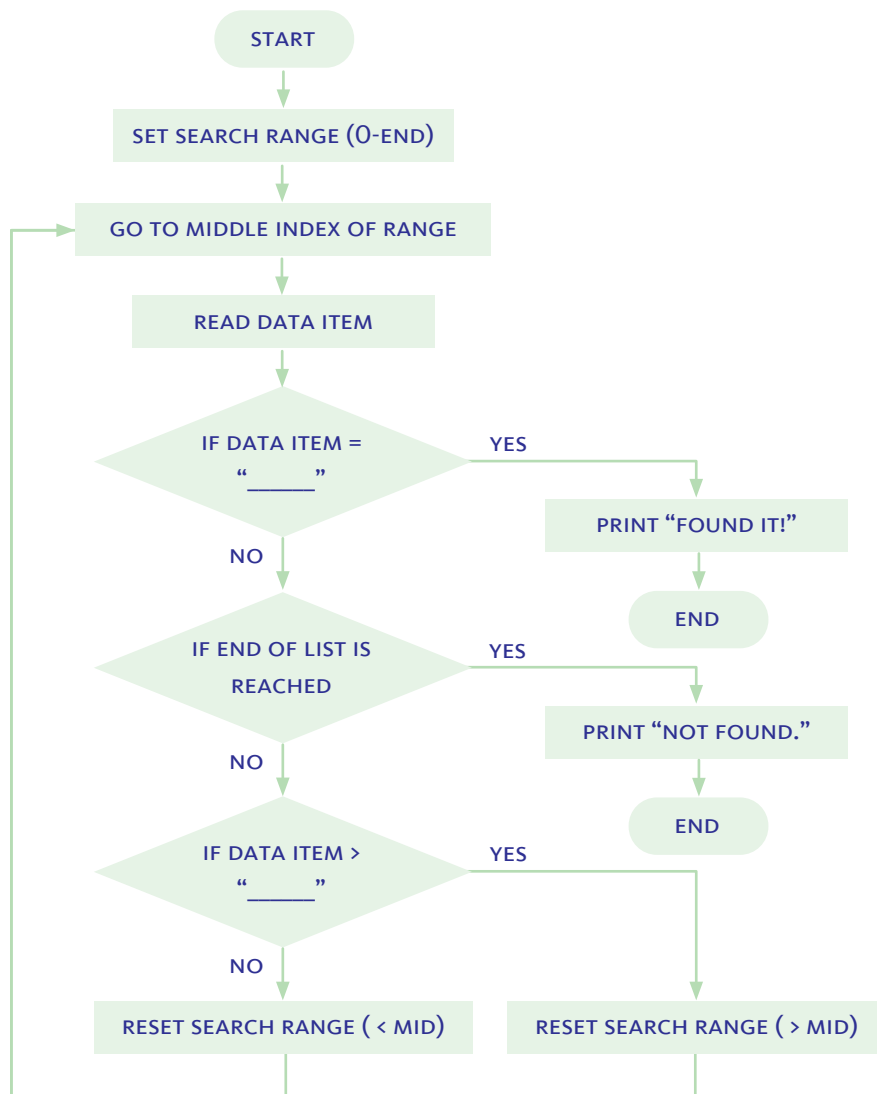
The list of 12 book titles in alphabetical order, which serves as the dataset for the binary search simulation.

The simulation illustrates the steps of a binary search to locate the book title 'Zebra Crossing' within the sorted list. Each attempt shows the search range, middle index, and the decision to disregard half of the list based on comparisons, continuing until the target is found at index 11.

Steps	Range (index)	Middle (index)	Comments
Attempt 1	0 - 11	5	Z > F, disregard 0 - 5
Attempt 2	6 - 11	8	Z > I, disregard 6 - 8
Attempt 3	8 - 11	10	Z > S, disregard 8 - 10
Attempt 4	11	11	Found it!

Had this search been conducted using a linear search algorithm, it would have taken 12 attempts. In comparison, the binary search algorithm's worst-case scenario required only 4 attempts, demonstrating the performance advantage of binary search over linear search when working with sorted lists.

Now that the basic concept of a binary search algorithm has been introduced, we can represent it in a flowchart that illustrates its essential steps: 1) setting the search range, 2) identifying the middle index of the search range, 3) evaluating the data item at the middle index to determine if the target has been found, and 4) if the target is not found, defining the next range of indices to continue the search.



This flowchart illustrates the steps of a binary search algorithm. Starting with an initial search range, the algorithm identifies the middle index, evaluates the data item at that index, and determines if it matches the target. If the target is not found, the algorithm narrows the search range by selecting the appropriate half and repeats the process until the target is located or the search range is exhausted.

Before writing the algorithm in pseudocode, we should explore its fundamental aspects. The first is setting the search range and identifying the search index, and the second is understanding the main loop of the algorithm and the conditions that repeat within it.

THE SEARCH RANGE AND INDEX

The search range in a binary search algorithm is always confined between its lower end, the **start_index**, and its upper end, the **end_index**. When the algorithm begins, these indices are set to the first and last positions of the list: **start_index (0)** and **end_index (last index of the dataset)**.

Once the search range has been set, the next step is to calculate the midpoint of the range to identify the index at which to search, the **search_index**. This midpoint is determined by halving the sum of the **start_index** and **end_index** of the search range.

$$\text{search_index} = \text{midpoint} = (\text{start_index} + \text{end_index}) / 2$$

With each iteration of the loop, the search range is redefined by adjusting either the upper end, **end_index**, or the lower end, **start_index**, depending on which half of the current range the target is determined to be in.

- If the target is in the lower half of the current range, **end_index** is adjusted to one position before the middle, calculated as **search_index - 1**.
- If the target is in the upper half of the current range, **start_index** is adjusted to one position after the middle, calculated as **search_index + 1**.

This approach continually narrows down the search range, recalculating the midpoint with each iteration that resets the range.

Notes

The midpoint must be an integer. If the calculation results in a fractional number, round down to the nearest integer.

THE LOOP AND CONDITIONS

The loop in the binary search algorithm ensures that a specific set of steps is repeated for each search index and that certain conditions are evaluated:

- Has the target been found at the search index: Is **data_item = target**?
- Is the target in the lower half of the search range: Is **data_item > target**?
- Is the target in the upper half of the search range: Is **data_item < target**?
In this algorithm, if **data_item** is neither equal to nor greater than the target, then it must be less than the target. This condition is therefore implied rather than explicitly stated.
- Has the whole list been searched? This is embedded within the **REPEAT_UNTIL** loop, which ensures that the loop continues until a specific condition is met—in this case, when the **search_index** has reached the end of the list.

```
START BinarySearch
OPEN list
SET start_index (0)
SET end_index (last index of list)

REPEAT_UNTIL search_index reaches end of list
  SET midpoint = (start_index + end_index)/2
  SET search_index (midpoint)
  READ data_item AT: search_index

  IF data_item = target THEN
    PRINT "Found it!"
    END BinarySearch
  ELSE IF data_item > target THEN
    SET end_index (search_index - 1)
  ELSE
    SET start_index (search_index + 1)

PRINT "Not found"
END BinarySearch
```

Pseudocode for a binary search algorithm that repeatedly narrows down the search range by adjusting the start and end indices until the target is found or the list is fully searched.

PERFORMANCE

Previously, in Example 3.4, we compared the performance of linear and binary search algorithms by counting the attempts required to find a target. In this worst-case scenario of searching through 12 book titles, the binary search algorithm located its target in significantly fewer attempts than the linear search—4 attempts versus 12.

We can also compare the two algorithms based on their execution time. For a linear search algorithm, the execution time is the product of the time required for a single attempt and the total number of attempts needed to identify the target. In contrast, while the execution time for a binary search algorithm also depends on the time required for each attempt, the number of possible attempts is halved with each step. To represent this reduction mathematically, we use the log-base 2 operation, which captures the exponential decrease in search range for each iteration.

If t is the time it takes to execute one attempt, and n is the number of items in the dataset, then the execution time for a binary search is:

$$\text{Execution Time} = t \times \log_2(n)$$

EXAMPLE 3.5

Referring back to the books on the shelf in Example 3.2, we will now use a binary search algorithm instead of a linear search. We'll compare the execution times for datasets of the same sizes to highlight the efficiency of binary search. The assumption that it takes 3 seconds to check each book will still hold.

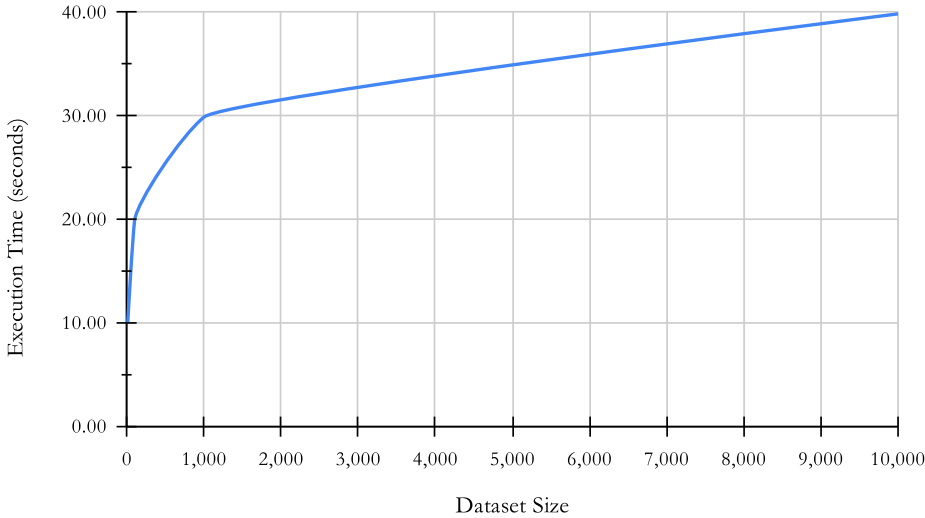
Notes

Students are not expected to understand the logarithmic function in depth at this stage. However, they should grasp that the log-base 2 function is used to mathematically represent the exponential decrease in the search range with each iteration of the binary search algorithm.

Size of Dataset	Execution Time (Binary Search)	Execution Time (Linear Search)
10 books	$3 \times \log_2(10) = 9.97 \text{ seconds}$	30 seconds
100 books	$3 \times \log_2(100) = 19.93 \text{ seconds}$	300 seconds (5 minutes)
1,000 books	$3 \times \log_2(1,000) = 29.90 \text{ seconds}$	3,000 seconds (50 minutes)
10,000 books	$3 \times \log_2(10,000) = 39.86 \text{ seconds}$	30,000 seconds (8 hours 20 minutes)

The table highlights a drastic difference in performance between linear search and binary search algorithms. For a dataset of 10,000 books, the execution time for a linear search grows linearly, reaching 8 hours and 20 minutes, while the execution time for a binary search grows logarithmically, reaching only 39.86 seconds. This substantial difference arises because binary search halves the search range with each attempt, making it far more efficient for larger datasets.

Binary Search Algorithm: Execution Time vs. Dataset Size



Execution time of a binary search algorithm across different dataset sizes, illustrating logarithmic growth and the algorithm’s efficiency as dataset size increases.

EXAMPLE 3.6

If the global telephone operator in Example 3.3 used a binary search algorithm instead of a linear search to locate a telephone number in a database of 100 million numbers, with a computer system latency of 50 nanoseconds per attempt, the execution time for the binary search would be:

$$\text{Execution Time} = 50 \times \log_2(100,000,000) = 1,330 \text{ nanoseconds}$$

Compared to the 5 seconds required for a linear search to accomplish this task, the 1,330 nanoseconds (1.33 microseconds) for a binary search is a significant improvement in performance.

CONCLUSION

In this lesson, we explored two fundamental search algorithms—linear search and binary search—and compared their performance across varying dataset sizes. While linear search is simple and works on unsorted data, its execution time grows linearly, making it inefficient for large datasets. In contrast, binary search, which requires sorted data, drastically reduces the search range with each attempt, resulting in logarithmic growth in execution time. Understanding these differences highlights the importance of choosing the right algorithm based on dataset size and performance requirements, a key concept in computer science.

SUMMARY

Searching is a fundamental operation needed to locate specific items in a dataset before performing further actions like modification, extraction, or processing.

Linear Search:

- Works by sequentially checking each item in the dataset from start to finish.
- Can be used on both sorted and unsorted datasets.
- Execution time grows linearly with the size of the dataset, making it inefficient for large datasets.

Binary Search:

- Requires the dataset to be sorted.
- Starts by checking the middle item of the search range and eliminates half of the range with each attempt.
- Execution time grows logarithmically with the size of the dataset, making it much more efficient than linear search for large datasets.

Comparison of Performance:

- Linear search checks every item, which results in a high execution time as the dataset grows.
- Binary search dramatically reduces the number of attempts by halving the search range, resulting in faster execution times for large, sorted datasets.
- Understanding the performance differences helps in choosing the right algorithm based on dataset size and structure.

Activity 3.1

Read the following statements, and write on the line provided for each statement 'T' if it is true and 'F' if it is false.

T/F	STATEMENT
<u>T</u>	A linear search algorithm can be used on both sorted and unsorted datasets.
<u>F</u>	A binary search algorithm can be used on both sorted and unsorted datasets.
<u>F</u>	A binary search algorithm begins by checking the first item in the dataset.
<u>F</u>	In a linear search algorithm, the search range is halved with each iteration.
<u>T</u>	Execution time for a linear search increases directly with the size of the dataset.
<u>T</u>	Execution time for a binary search increases logarithmically with the size of the dataset.
<u>F</u>	In the worst-case scenario, binary search requires more attempts than linear search.
<u>T</u>	In the best-case scenario, binary search and linear search are equally efficient.
<u>T</u>	In the worst-case scenario, binary search is more efficient than linear search, regardless of dataset size.
<u>F</u>	The choice of search algorithm does not impact the overall performance of a program.

Activity 3.2

Answer the questions below.

1. What is the primary characteristic of a linear search algorithm?
 - a) It requires a sorted dataset.
 - b) **It searches each item sequentially.**
 - c) It divides the dataset in half each time.
 - d) It starts at the middle of the dataset.
2. Which of the following is a limitation of linear search for large datasets?
 - a) It requires a sorted list.
 - b) **It can be very slow because it checks each item sequentially.**
 - c) It cannot be used with unique values.
 - d) It does not work on datasets with numbers.
3. What is the best-case scenario for a linear search algorithm?
 - a) The target is in the middle of the dataset.
 - b) The target is at the end of the dataset.
 - c) The dataset is already sorted.
 - d) **The target is the first item in the dataset.**
4. Which type of data structure can linear search be applied to?
 - a) Only sorted arrays
 - b) Only linked lists
 - c) **Any list, sorted or unsorted**
 - d) Only binary trees
5. What happens in a linear search if the target item is not in the dataset?
 - a) The search stops at the midpoint of the dataset.
 - b) **The search checks every item and then stops.**
 - c) The search requires the dataset to be reordered.
 - d) The search skips some items to save time.

Activity 3.3

Answer the questions below.

1. What is the main requirement for using a binary search algorithm on a dataset?
 - a) The dataset must contain unique values.
 - b) **The dataset must be sorted.**
 - c) The dataset must be large.
 - d) The dataset must be stored in a linked list.
2. What does binary search do with each iteration?
 - a) Moves one step closer to the target from the beginning of the list.
 - b) **Divides the search range in half.**
 - c) Checks each item sequentially.
 - d) Moves in reverse through the list.
3. In a binary search, if the target is less than the middle item, what should the algorithm do next?
 - a) **Focus on the lower half of the search range.**
 - b) Focus on the upper half of the search range.
 - c) Ignore the entire list.
 - d) Conclude that the target is not present.
4. What is the best-case scenario for a binary search algorithm?
 - a) The dataset is very large.
 - b) The dataset is unsorted.
 - c) The target is the first item in the dataset.
 - d) **The target is the middle item on the first check.**
5. What is the growth rate of a binary search algorithm?
 - a) Linear
 - b) Exponential
 - c) **Logarithmic**
 - d) Quadratic

Activity 3.4

Answer the questions below.

1. Which search algorithm is generally faster for large, sorted datasets?
 - a) **Binary search**
 - b) Linear search
 - c) Both are equally fast
 - d) It depends on the dataset size
2. What is a key difference in requirements between linear and binary search?
 - a) Linear search requires sorted data, while binary search does not.
 - b) **Binary search requires sorted data, while linear search does not.**
 - c) Both require sorted data.
 - d) Neither requires sorted data.
3. When would a linear search be preferred over a binary search?
 - a) When there are fewer than 10 items.
 - b) When the dataset is sorted and very large.
 - c) When binary search fails to find the target.
 - d) **When the dataset is unsorted.**
4. Why is binary search more efficient than linear search on large, sorted datasets?
 - a) It skips every other item in the list.
 - b) It checks each item twice.
 - c) **It divides the search range in half with each attempt.**
 - d) It starts at the end of the list.
5. If the target is the first item in a dataset, how does the performance of binary search compare to linear search?
 - a) Binary search will be slower.
 - b) Linear search will be slower.
 - c) **Both will perform the same.**
 - d) The result depends on dataset size.

Activity 3.5

Refer to the sorted list. Compare the number of attempts required for linear search and binary search to find specific numbers in the list.

List_A = [3, 7, 12, 19, 25, 31, 44, 52, 67, 72, 88, 93, 105, 117, 126, 139, 145, 159, 167, 178, 190]

- Find the number 3.

Linear Search Attempts: 1

Binary Search Attempts: 5

- Find the number 88.

Linear Search Attempts: 11

Binary Search Attempts: 1

- Find the number 190.

Linear Search Attempts: 21

Binary Search Attempts: 5

- Find the number 105.

Linear Search Attempts: 13

Binary Search Attempts: 3

- What is the number of worst-case scenario attempts?

Linear Search Attempts: 21

Binary Search Attempts: 5

- Which index in the list provides a best-case scenario for:

Linear Search: index (0) first

Binary Search: index (10) middle

 Activity 3.6

For each of the three scenarios provided below, determine the number of attempts that a linear and a binary search algorithm would require to find the target item in the worst-case scenario. Then, calculate the execution time for each algorithm's search.

1. A dataset contains 1,024 items, and each attempt takes 2 milliseconds. Calculate the worst-case number of attempts and the execution time for both a linear and a binary search on this dataset.

Linear Search: Attempts: $n = 1,024$ (all items are checked)

Execution time = $t \times n = 2 \times 1,024 = 2,048$ ms (about 2 seconds)

Binary Search: Attempts: $\log_2(n) = \log_2(1,024) = 10$

Execution time = $t \times \log_2(n) = 2 \times 10 = 20$ ms

2. A dataset of 20,000 items is searched, with each attempt taking 5 milliseconds. Calculate the worst-case number of attempts and the execution time for both a linear and a binary search.

Linear Search: Attempts: $n = 20,000$ (all items are checked)

Execution time = $t \times n = 5 \times 20,000 = 100,000$ ms (100 seconds)

Binary Search: Attempts: $\log_2(n) = \log_2(20,000) = 14.29 = 15$ (round up)

Execution time = $t \times \log_2(n) = 5 \times 15 = 75$ ms

3. A large dataset of 1,000,000 items is searched, and each attempt takes 10 milliseconds. Calculate the worst-case number of attempts and the execution time for both a linear and a binary search.

Linear Search: Attempts: $n = 1,000,000$ (all items are checked)

Execution time = $t \times n = 10 \times 1,000,000 = 10,000,000$ ms (10,000 seconds)

Binary Search: Attempts: $\log_2(n) = \log_2(1,000,000) = 19.93 = 20$ (round up)

Execution time = $t \times \log_2(n) = 10 \times 20 = 200$ ms

Activity 3.7

Use the linear search algorithm provided in pseudocode to answer the following questions.

```

START LinearSearch
OPEN list
SET search_index (0)

REPEAT_UNTIL search_index reaches end of list
  READ data_item AT: search_index

  IF data_item = target THEN
    PRINT "Found it!"
    END LinearSearch
  ELSE
    SET search_index (search_index + 1)

PRINT "Not found"
END LinearSearch

```

1. What is the purpose of the instruction: SET search_index (0) ?

Sets the starting position of the search at the first item in the list.

2. Which instruction increments the search index by 1 position?

SET search_index (search_index + 1)

3. What is the purpose of the instruction: REPEAT_UNTIL ?

It creates a loop to repeatedly check items in the list.

4. Which two conditions end the REPEAT_UNTIL loop?

search_index reaches end of list

data_item = target

Activity 3.8

Use the binary search algorithm provided in pseudocode to answer the following questions.

```

START BinarySearch
OPEN list
SET start_index (0)
SET end_index (last index of list)

REPEAT_UNTIL search_index reaches end of list
  SET midpoint = (start_index + end_index)/2
  SET search_index (midpoint)
  READ data_item AT: search_index

  IF data_item = target THEN
    PRINT "Found it!"
    END BinarySearch
  ELSE IF data_item > target THEN
    SET end_index (search_index - 1)
  ELSE
    SET start_index (search_index + 1)

PRINT "Not found"
END BinarySearch

```

1. Which two parameters define the search range?

The two parameters are the start_index and the end_index.

2. What is the purpose of the midpoint variable?

The midpoint variable calculates the middle index of the current search range.

3. What is the purpose of the instruction: SET end_index (search_index - 1) ?

It focuses the search on the lower half of the current search range by resetting the current end index to one position below the current midpoint.

4. Why is the instruction IF data_item < target THEN not included?

If data_item is neither equal to nor greater than the target, then it must be less than the target by exclusion.

Activity 3.9

Having gained an in-depth understanding of the mechanics, performance, and logic behind linear and binary search algorithms, it's time to explore them further. This activity allows students to examine linear and binary search algorithms in Python programs. Follow the links to download the three programs provided for this activity.

1. [Linear Search Program](#): This program performs a linear search on a user-defined, sorted range. It checks each item in the list sequentially until it finds the target or reaches the end. The program outputs the number of attempts taken, demonstrating the step-by-step process of linear search.
2. [Binary Search Program](#): This program performs a binary search on a user-defined, sorted range. It begins by checking the midpoint of the range and repeatedly halves the search space based on comparisons. The program reports the number of attempts, showing the efficiency of binary search on sorted data.
3. [Combined Search Program](#): This program includes both linear and binary search algorithms, operating on the same user-defined, sorted range. It allows users to compare the performance of linear and binary search directly by running both methods on the same target and range. This comparison highlights the efficiency differences between the two search methods.

Notes

Notice the similarity between the pseudocode and the Python programs provided for linear and binary search. This similarity highlights how pseudocode serves as an essential bridge between algorithmic logic and actual coding. Pseudocode provides students with a clear, language-agnostic structure for algorithms, helping them transition smoothly from planning to implementation in Python or other programming languages. Encourage students to use pseudocode as a guide to understand, develop, and refine their programming skills.

LESSON 4: SORTING LINEAR DATA

In earlier lessons, we worked with data in many forms: we tracked temperature changes over time, compared students' exam scores with their study hours, and even helped organize a warehouse of electronics and groceries. We learned how to label, clean, and organize data, and how to search through it efficiently using linear and binary search.

But what if you wanted to find the highest temperature, rank students by their exam scores, or organize your inventory from oldest to newest? Before you can do any of that, you need to sort your data.

**SORTING IS THE ARRANGEMENT OF DATA IN A SPECIFIC ORDER
BASED ON A CERTAIN CRITERION.**

Sorting means arranging data in a specific order based on a certain criterion. For example:

- Books can be sorted by their titles.
- Apple crates in a warehouse can be sorted by their arrival date.
- Students can be sorted by their exam scores.

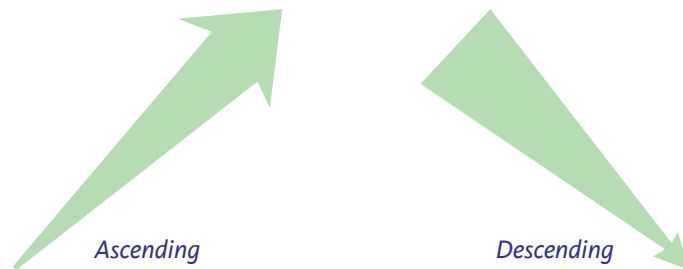
Sorting is a powerful approach to organizing data within any data structure. It typically makes the data more accessible and allows for easier extraction of information.

In this lesson, we will learn what sorting is, why it is useful, and explore two simple but powerful sorting strategies—Insertion Sort and Bubble Sort—and how they work.

ORDER & CRITERIA

Sorting is defined as the arrangement of data in a specific order based on a certain criterion. This definition combines two key ideas: “a specific order” and “a certain criterion.”

Order refers to the direction in which the data is arranged. It answers the question: Which direction will the sorted data follow? The two most common orders are ascending, which means increasing from least to greatest, and descending, which means decreasing from greatest to least.



The criterion, on the other hand, refers to the basis used to determine how the data will be sorted. It answers the question: What are we sorting by? For example, are we sorting by exam scores, temperatures, prices, student names, book titles, or time?

In general, sorting criteria fall into two main categories: numeric data, which consists of numbers, and alphanumeric data, which typically includes letters, words, and characters.

Sorting numeric data is based on numerical value and may be arranged in ascending (e.g. 1, 2, 3...) or descending (e.g. 10, 9, 8...) order. Examples include exam scores, temperatures, prices, and time.

Alphanumeric data is sorted based on character sequences and may also be ordered in ascending (e.g. A, B, C...) or descending (e.g. Z, Y, X...) order. Examples of alphanumeric data include student names, book titles, file names, or ID codes.

Whether a dataset is sorted in a specific order or by a particular criterion depends on the nature of the data and the purpose of sorting.

EXAMPLE 4.1

The school's database contains data on all enrolled students. The table below shows a sample segment of that database with 10 entries. Even though these are just a few data items, it is still difficult and tedious for a person to extract information from it manually.

Name	Family	Absences	Average	Transport
Leila	Carter	1	85	Bus
Tarek	Najem	0	91	Walk
Sofia	Petrovic	10	51	Parents
Ahmed	Dandash	8	64	Bus
Ethan	Müller	0	94	Parents
Dana	Lankart	2	83	Parents
Isabel	Garcia	1	74	Parents
Roland	Bauer	13	45	Walk
Felix	Jensen	3	77	Bus
Ben	Zimmer	0	89	Bus

Let's say we wanted to find a particular family name, "Carter." We would need to check each family name starting from the top — a linear search approach. As we've previously discussed, linear search is not the most efficient algorithm but is necessary when the data is unsorted. However, we can sort this table by family name in ascending alphabetical order.

Name	Family	Absences	Average	Transport
Roland	Bauer	13	45	Walk
Leila	Carter	1	85	Bus
Ahmed	Dandash	8	64	Bus
Isabel	Garcia	1	74	Parents
Felix	Jensen	3	77	Bus
Dana	Lankart	2	83	Parents
Ethan	Müller	0	94	Parents
Tarek	Najem	0	91	Walk
Sofia	Petrovic	10	51	Parents
Ben	Zimmer	0	89	Bus

With a sorted list, finding a name becomes easier. It also allows us to apply more efficient searching algorithms, such as the binary search algorithm.

The same dataset can be sorted using different orders and rules. The choice of order and rule depends on the nature of the data and the purpose of sorting.

EXAMPLE 4.2

Let's say, for example, that this time we want to extract information about absences and attendance. If our purpose is to identify the students who were absent the greatest number of times, and the data under "Absences" contains numeric values, then we would sort the dataset by Absences in descending order of numeric value. With the data now sorted, identifying the students with the highest number of absences becomes much easier.

Name	Family	Absences	Average	Transport
Roland	Bauer	13	45	Walk
Sofia	Petrovic	10	51	Parents
Ahmed	Dandash	8	64	Bus
Felix	Jensen	3	77	Bus
Dana	Lankart	2	83	Parents
Leila	Carter	1	85	Bus
Isabel	Garcia	1	74	Parents
Tarek	Najem	0	91	Walk
Ethan	Müller	0	94	Parents
Ben	Zimmer	0	89	Bus

Not only does sorting make finding information easier—either by making it visually evident or by enabling the use of efficient search algorithms—but it also helps us spot trends and patterns in data, particularly in smaller datasets like this one.

One of the key benefits of sorting is that it makes patterns more visible. When data is arranged in a meaningful order—such as alphabetically or numerically—it becomes easier to scan, compare, and interpret. For example, sorting students by the number of absences in descending order quickly highlights those with the highest absenteeism, drawing attention to potential attendance issues.

Similarly, sorting exam scores from highest to lowest reveals the top performers in a class. But it can also help uncover broader trends.

EXAMPLE 4.3

If we examine the table—specifically the fields Absences and Average—we notice a pattern: students with the most absences tend to have the lowest averages, while those with the fewest absences tend to have the highest.

Name	Family	Absences	Average	Transport
Roland	Bauer	13	45	Walk
Sofia	Petrovic	10	51	Parents
Ahmed	Dandash	8	64	Bus
Felix	Jensen	3	77	Bus
Dana	Lankart	2	83	Parents
Leila	Carter	1	85	Bus
Isabel	Garcia	1	74	Parents
Tarek	Najem	0	91	Walk
Ethan	Müller	0	94	Parents
Ben	Zimmer	0	89	Bus

It is important to note that, although this observation does not confirm a relationship between attendance and academic performance, it raises a question that could be explored through further analysis.

As we've seen, sorting is a valuable strategy for organizing data in a way that makes it easier to search, interpret, and extract information. Whether we're searching for a specific name or analyzing attendance trends, sorting helps reveal patterns and supports more efficient decision-making. While sorting alone does not confirm relationships within data, it can highlight areas worth investigating.

The same dataset can be sorted in different ways depending on what we're looking for and why. Sorting by alphabetical order helps us locate items more quickly, while sorting numerically allows us to rank or compare values. As we continue exploring sorting strategies, we'll begin to see how even simple algorithms like Insertion Sort and Bubble Sort can support meaningful data analysis when applied thoughtfully.

SORTING ALGORITHMS

Sorting algorithms determine the step-by-step process of arranging data in a specific order. All sorting algorithms must be able to compare data items and change their positions so that the data becomes organized. While the goal is the same, different algorithms follow different strategies—each varying in which data items are compared, and how and when those items are moved to new positions.

In this section, we will explore two simple but effective sorting algorithms—Insertion Sort and Bubble Sort.

THE INSERTION SORT ALGORITHM

The insertion sort algorithm follows a strategy of compare, shift, and insert. Starting at the beginning of the list, it temporarily removes one data item at a time and compares it with the items that precede it. Once the correct position is found, any intervening out-of-order data items are shifted up, and the removed data item is inserted into the empty position closer to the beginning.

THE INSERTION SORT ALGORITHM FOLLOWS A SEQUENTIAL STRATEGY OF COMPARE, SHIFT, AND INSERT.

In a sense, out-of-order items are shifted toward the end, and data items are inserted one by one into their correct positions.

If the goal is to sort in ascending order, data items with lesser values are inserted in increasing order near the beginning of the list, while greater out-of-order values are shifted upward toward the end. If the goal is to sort in descending order, the opposite occurs: greater values are inserted in decreasing order near the beginning, while lesser out-of-order values are shifted upward.

EXAMPLE 4.4

In this example, we will sort a list of five numbers in ascending order. Starting at index 1, each data item is compared to the values before it. Greater out-of-order values are shifted upward, and the lesser value is inserted into its correct position near the beginning of the list. These steps are repeated in iterative loops until the last item in the list has been selected, compared, and inserted into its correct position.

“5, 3, 4, 1, 2”

To better visualize the list, each data item is shown in its respective position, labeled by its index.

DATA	5	3	4	1	2
INDEX	0	1	2	3	4

The First Iteration

Step 1 - Hold

Data item at Index 1 is temporarily held.

		3			
DATA	5		4	1	2
INDEX	0	1	2	3	4

Step 2 - Compare

Yes, “5” is greater than “3”.

		3			
DATA	5		4	1	2
INDEX	0	1	2	3	4

Step 3 - Shift

“5” is shifted to Index 1 instead of “3”.

		3			
DATA		5	4	1	2
INDEX	0	1	2	3	4

Step 4 - Insert

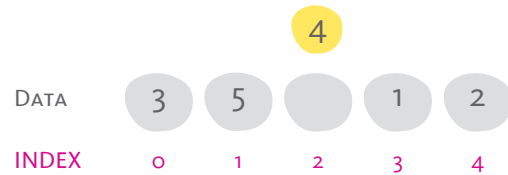
“3” is inserted at Index 0 instead of “5”.

DATA	3	5	4	1	2
INDEX	0	1	2	3	4

The Second Iteration

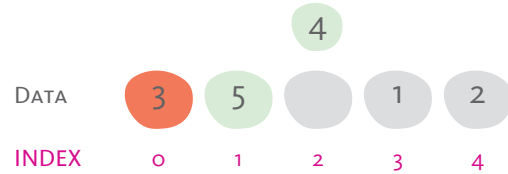
Step 1 - Hold

Data item at Index 2 is temporarily held.

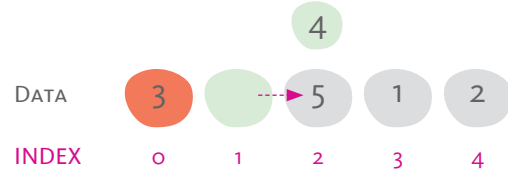
**Step 2 - Compare**

Yes, "5" is greater than "4".

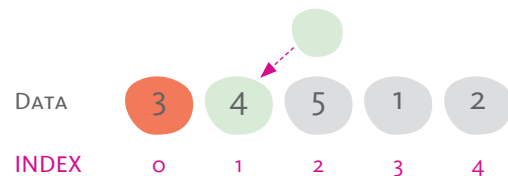
No, "3" is not greater than "4".

**Step 3 - Shift**

Only "5" is shifted to Index 2 instead of "4".

**Step 4 - Insert**

"4" is inserted at Index 1 instead of "5".



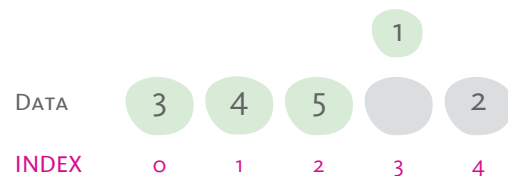
The Third Iteration

Step 1 - Hold

Data item at Index 3 is temporarily held.

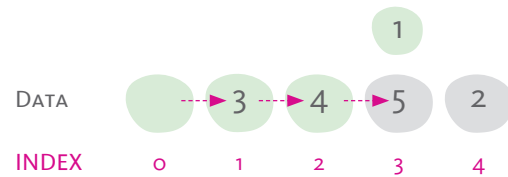
**Step 2 - Compare**

Yes, "5", "4", and "3" are greater than "1".



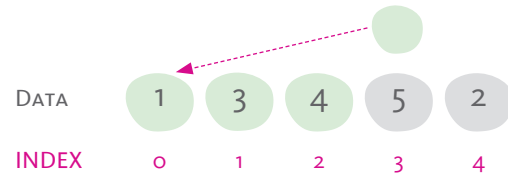
Step 3 - Shift

"5", "4", and "3" are shifted one index up.



Step 4 - Insert

"1" is inserted at Index 0 instead of "3".



The Fourth Iteration

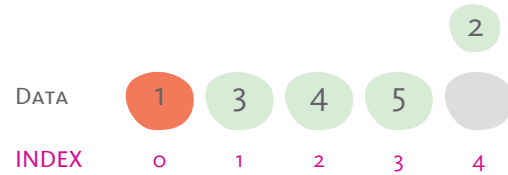
Step 1 - Hold

Data item at Index 4 is temporarily held.



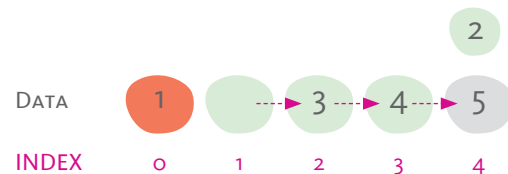
Step 2 - Compare

Yes, "5", "4", and "3" are greater than "2".
No, "1" is not greater than "2".



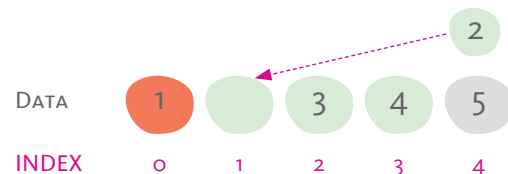
Step 3 - Shift

"5", "4", and "3" are shifted one index up.



Step 4 - Insert

"2" is inserted at Index 1 instead of "3".



Sorting Complete

The last index is reached.

All data is sorted in ascending order.



THE INSERTION SORT PROCESS

Now that we have demonstrated the mechanics of the algorithm, we need to establish the flow of steps that define the operations and conditions that make the insertion sort algorithm work.

VARIABLES & LABELS

Understanding how sorting works begins with recognizing how we name and refer to both data and positions within a dataset. In programming, we use variables as labels that help us hold and manipulate information.

A **VARIABLE** IS A LABEL WHOSE VALUE OR CONTENT CAN CHANGE.

In the insertion sort process, we use several key variables to make the algorithm readable and workable. These variables may represent either the data values themselves or their positions (called indices) within the list:

- **hold_index** refers to the index whose data is temporarily held for comparison.
- **hold_data** refers to the data at the hold_index.
- **compare_index** refers to the index that precedes the hold_index and against which the held data is compared.
- **compare_data** refers to the data at the compare_index.

It is important to remember the difference between data and index:

- Indices are fixed positions in the list—they do not change.
- Data can move from one index to another during sorting.

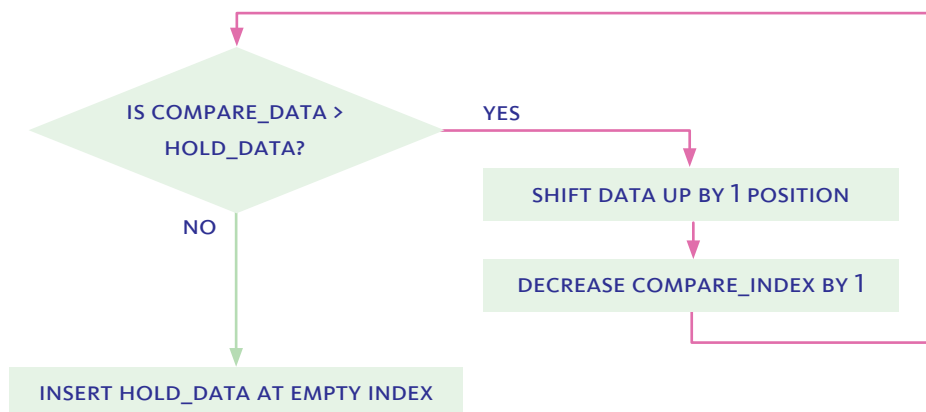
FLOW & CONTROL

Every algorithm has a flow or sequence of steps that are executed in order to achieve the required outcome—in this case, sorting a list of numbers in ascending order. This flow is controlled by certain conditions that determine the sequence in which instructions are carried out.

When a group of instructions needs to be repeated, they are collectively referred to as a loop. The insertion sort algorithm can generally be described as having two repeating sets of steps, which we will refer to as the outer loop and the inner loop.

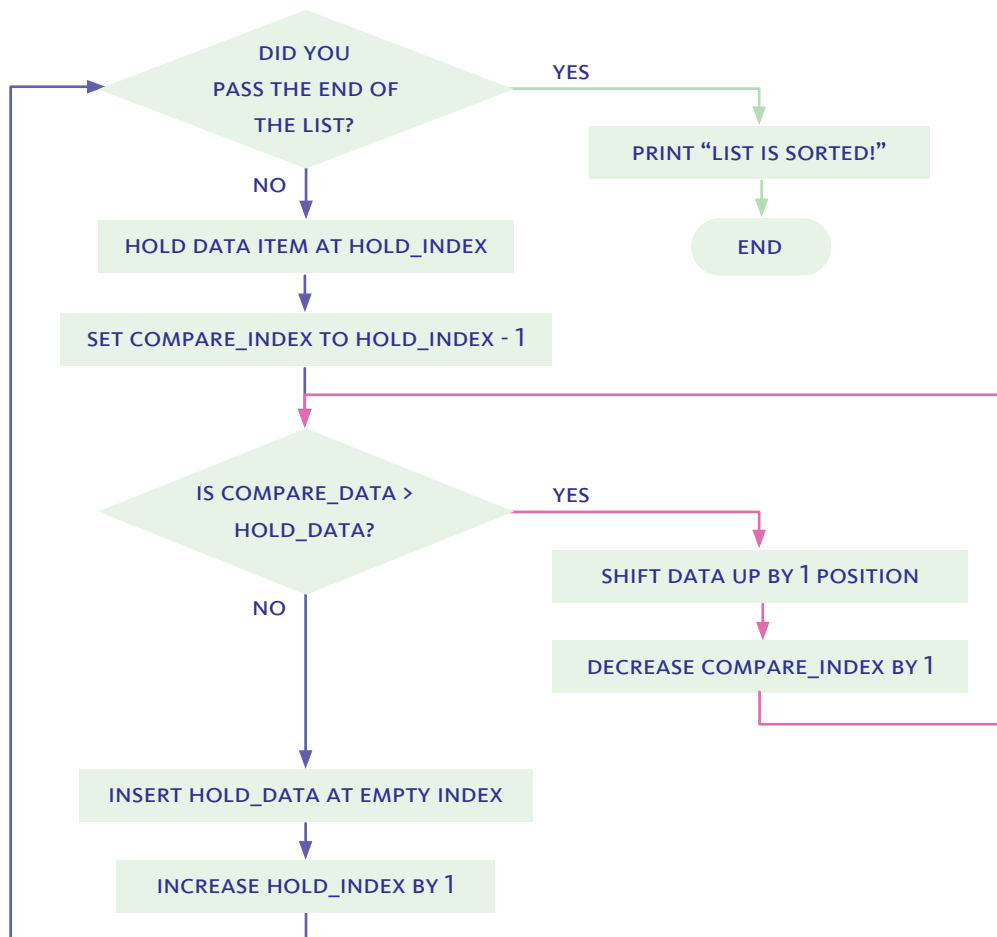
The **inner loop** performs a series of steps that compare the held data item with the items before it in the list. If the compared data is greater than the held data, it is shifted forward by one position. This process repeats as long as that condition is true.

Once the compared data is less than or equal to the held data, the loop ends, and the algorithm inserts the held data into the correct empty position toward the beginning of the list.



The inner loop compares and shifts data, moving one position at a time toward the beginning of the list. With each step, it shifts larger data items up by one index. The loop breaks when the compared data is less than or equal to the held data, at which point the held data is inserted into the empty index created by the shifting.

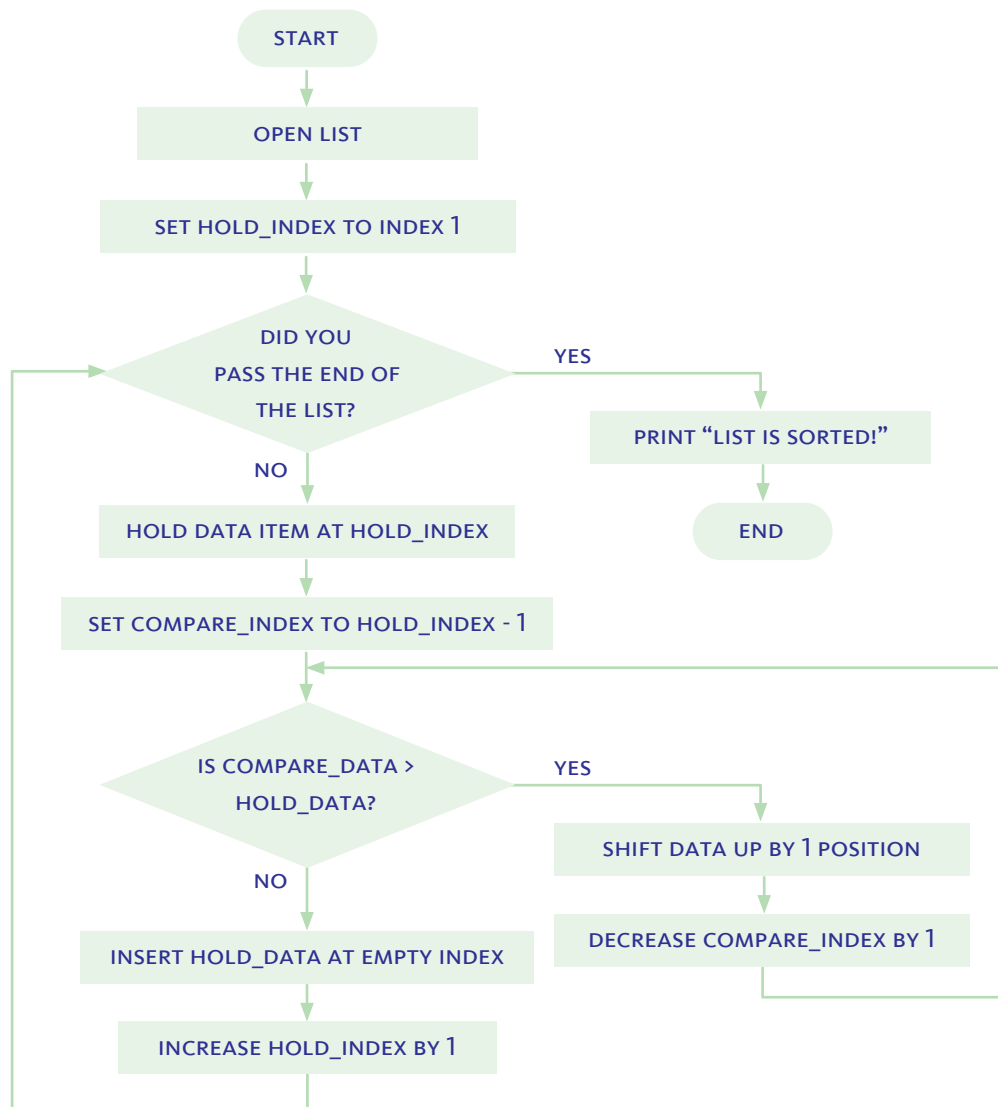
The **outer loop** is tasked with moving the inner loop sequentially through each index in the list. It starts at index 1, and at each step, it passes control to the inner loop to perform the comparisons, shifts, and insertion. When the inner loop completes its task, control is handed back to the outer loop, which moves the process forward by one index. The outer loop breaks when its condition is satisfied—that is, when it reaches the end of the list.



This flowchart highlights the two main components of the insertion sort process. The outer loop (blue) moves sequentially through each data item in the list using the `hold_index`. The inner loop (pink) repeatedly compares the held data with preceding items and shifts larger values upward until the correct position is found for insertion.

INITIALIZING

Initialization is the first part of the algorithm where we get everything ready before sorting begins. In this phase, we open the list we want to sort and set the starting point by choosing the first index we'll work with.



This flowchart shows the step-by-step process of the insertion sort algorithm. It begins with initialization, then uses an outer loop to move through the list and an inner loop to compare and shift data until each item is inserted into its correct position.

THE INSERTION SORT PROGRAM

Having understood the mechanics of the insertion sort algorithm and the flow of its process, it is now time to write the instructions for a program that implements this algorithm.

The program is divided into four blocks:

1. **Initialization** — this is where we begin by opening the list and declaring the variables needed for the sorting process. Before a variable can be used, it must first be declared and given an initial value. In this case, we declare `hold_index` and assign it the value 1, so that sorting starts with the second item in the list. We also declare `end_index` and assign it the value of the last index in the list, which sets the boundary for when the sorting process should stop.



```

START InsertionSort
OPEN list
SET hold_index (1)
SET end_index (last index of list)

REPEAT_UNTIL hold_index > end_index
  SET hold_data (data_item AT hold_index)
  SET compare_index (hold_index - 1)
  SET compare_data (data_item AT compare_index)

  AS_LONG_AS hold_data > compare_data
    SHIFT compare_data FROM compare_index TO compare_index + 1
    RESET compare_index (compare_index - 1)
    RESET compare_data (data_item AT compare_index)

  INSERT hold_data TO compare_index
  RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END InsertionSort

```

The pink brackets and green text indicate the initialization block of the insertion sort algorithm. This section sets up the list and defines key variables—`hold_index` and `end_index`—to prepare the algorithm before the sorting process begins.

- 2. Outer loop** — which moves one step at a time through the list, selecting the next data item to be sorted.

This loop is controlled by a **REPEAT_UNTIL** instruction, which allows the loop to continue until it reaches the end of the list—i.e., when the `hold_index` is greater than the `end_index`.

Inside this block, we initialize the `hold_data` to be the data item at `hold_index`, the `compare_index` to be one position before the `hold_index`, and the `compare_data` to be the value at the `compare_index`.

The block concludes with inserting the held data into the appropriate position, as determined by the inner loop, and then incrementing `hold_index` by 1 to prepare for the next outer loop cycle.

```
START InsertionSort
OPEN list
SET hold_index (1)
SET end_index (last index of list)

REPEAT_UNTIL hold_index > end_index
  SET hold_data (data_item AT hold_index)
  SET compare_index (hold_index - 1)
  SET compare_data (data_item AT compare_index)

  AS_LONG_AS hold_data > compare_data
    SHIFT compare_data FROM compare_index TO compare_index + 1
    RESET compare_index (compare_index - 1)
    RESET compare_data (data_item AT compare_index)

  INSERT hold_data TO compare_index
  RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END InsertionSort
```

The pink brackets and green text indicate the outer loop of the algorithm. This loop controls the progression through the list, selecting one data item at a time using the `hold_index`. For each pass, it prepares the held data and sets up comparisons before calling the inner loop. The loop continues until the `hold_index` has passed the `end_index`, meaning the entire list has been sorted.

3. **Inner loop** — which compares the held data with the values before it, shifts out-of-order data up the list, and determines where the held data should be inserted.

This loop uses an **AS_LONG_AS** instruction, which means that as long as the value of `hold_data` is greater than the value of `compare_data`, the algorithm continues shifting the `compare_data` up by one position and resetting `compare_index` one step back.

```
START InsertionSort
OPEN list
SET hold_index (1)
SET end_index (last index of list)

REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    AS_LONG_AS hold_data > compare_data
        SHIFT compare_data FROM compare_index TO compare_index + 1
        RESET compare_index (compare_index - 1)
        RESET compare_data (data_item AT compare_index)

    INSERT hold_data TO compare_index
    RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END InsertionSort
```

The pink brackets and green text indicate the inner loop, which repeatedly compares the held data with earlier items in the list. As long as the held data is smaller, larger values are shifted upward one position. The loop continues until the correct position is found for inserting the held data.

Notes

Program blocks can be arranged:

- Sequentially – one after another in order.
- Nested – with one block contained inside another.

- 4. Termination** — After all data items have been compared, shifted, and inserted into their correct positions, the program reaches its termination block.

This part is not part of any loop. It simply prints a message to indicate that the sorting is complete and then ends the program. It serves as a clear signal that the algorithm has finished running.

```
START InsertionSort
OPEN list
SET hold_index (1)
SET end_index (last index of list)

REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    AS_LONG_AS hold_data > compare_data
        SHIFT compare_data FROM compare_index TO compare_index + 1
        RESET compare_index (compare_index - 1)
        RESET compare_data (data_item AT compare_index)

    INSERT hold_data TO compare_index
    RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END InsertionSort
```

The pink brackets and green text indicate the termination block of the program. This block runs after the sorting process is complete. It prints a message to confirm that the list has been successfully sorted and then ends the program.

Notes

Indentation in a program shows which instructions belong to a block. Lines indented under a loop or condition are part of that block. Further indentation indicates nested blocks.

THE BUBBLE SORT ALGORITHM

The bubble sort algorithm follows a strategy of pair, compare, and swap. Starting at the beginning of the list, it repeatedly compares pairs of adjacent data items. If the two items are out of order, they are swapped so that the values begin to move toward their correct positions.

THE BUBBLE SORT ALGORITHM FOLLOWS A SEQUENTIAL STRATEGY OF PAIR, COMPARE, AND SWAP.

If the goal is to sort in ascending order, greater values are bubbled toward the end of the list with each full pass, while smaller values gradually settle into place near the beginning. This process is repeated through multiple passes until the entire list is sorted.

If the goal is to sort in descending order, each pass pushes the smallest unsorted value toward the end, while the greater values settle into place near the beginning.

EXAMPLE 4.5

Let's go back to the same list we used in Example 4.4, and this time use the bubble sort algorithm to sort its data in ascending order.

"5, 3, 4, 1, 2"

The bubble sort algorithm makes several passes through the list, each time sequentially comparing pairs of data items starting from the beginning. Like the insertion sort algorithm, bubble sort uses two nested loops. The outer loop controls the number of passes through the list, with each pass considered an iteration of the loop. The inner loop handles the individual pairing, comparing, and swapping of data items, and it iterates through each pair in the list.

The First Iteration

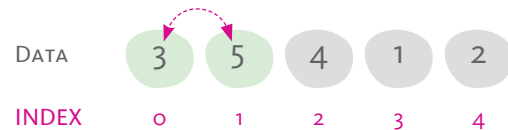
Step 1 - Pair & Compare

Index 0 and Index 1 are paired.

Yes, "5" is greater than "3"

Step 2 - Swap

"5" and "3" are swapped.

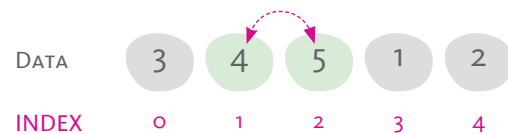
**Step 1 - Pair & Compare**

Index 1 and Index 2 are paired.

Yes, "5" is greater than "4"

Step 2 - Swap

"5" and "4" are swapped.

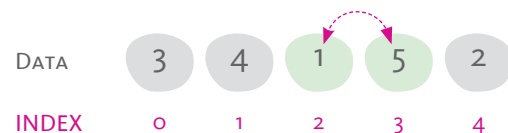
**Step 1 - Pair & Compare**

Index 2 and Index 3 are paired.

Yes, "5" is greater than "1"

Step 2 - Swap

"5" and "1" are swapped.

**Step 1 - Pair & Compare**

Index 3 and Index 4 are paired.

Yes, "5" is greater than "2"

Step 2 - Swap

"5" and "2" are swapped.



The Second Iteration

Step 1 - Pair & Compare

Index 0 and Index 1 are paired.

No, "3" is not greater than "4"

Step 2 - Swap

"3" and "4" are not swapped.

DATA	3	4	1	2	5
INDEX	0	1	2	3	4

DATA	3	4	1	2	5
INDEX	0	1	2	3	4

Step 1 - Pair & Compare

Index 1 and Index 2 are paired.

Yes, "4" is greater than "1"

Step 2 - Swap

"4" and "1" are swapped.

DATA	3	4	1	2	5
INDEX	0	1	2	3	4

DATA	3	1	4	2	5
INDEX	0	1	2	3	4

Step 1 - Pair & Compare

Index 2 and Index 3 are paired.

Yes, "4" is greater than "2"

Step 2 - Swap

"4" and "2" are swapped.

DATA	3	1	4	2	5
INDEX	0	1	2	3	4

DATA	3	1	2	4	5
INDEX	0	1	2	3	4

Step 1 - Pair & Compare

Index 3 and Index 4 are paired.

Yes, "4" is not greater than "5"

Step 2 - Swap

"4" and "5" are not swapped.

DATA	3	1	2	4	5
INDEX	0	1	2	3	4

DATA	3	1	2	4	5
INDEX	0	1	2	3	4

The Third Iteration

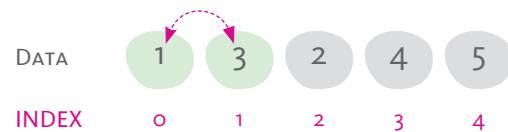
Step 1 - Pair & Compare

Index 0 and Index 1 are paired.

Yes, "3" is greater than "1"

**Step 2 - Swap**

"3" and "1" are swapped.

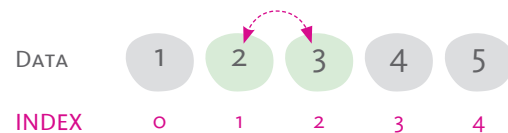
**Step 1 - Pair & Compare**

Index 1 and Index 2 are paired.

Yes, "3" is greater than "2"

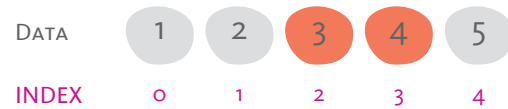
**Step 2 - Swap**

"3" and "2" are swapped.

**Step 1 - Pair & Compare**

Index 2 and Index 3 are paired.

No, "3" is not greater than "4"

**Step 2 - Swap**

"3" and "4" are not swapped.

**Step 1 - Pair & Compare**

Index 3 and Index 4 are paired.

No, "4" is not greater than "5"

**Step 2 - Swap**

"4" and "5" are not swapped.



The Fourth Iteration

Step 1 - Pair & Compare

Index 0 and Index 1 are paired.
No, "1" is not greater than "2"

DATA	1	2	3	4	5
INDEX	0	1	2	3	4

Step 2 - Swap

"1" and "2" are not swapped.

DATA	1	2	3	4	5
INDEX	0	1	2	3	4

Step 1 - Pair & Compare

Index 1 and Index 2 are paired.
No, "2" is not greater than "3"

DATA	1	2	3	4	5
INDEX	0	1	2	3	4

Step 2 - Swap

"2" and "3" are not swapped.

DATA	1	2	3	4	5
INDEX	0	1	2	3	4

Step 1 - Pair & Compare

Index 2 and Index 3 are paired.
No, "3" is not greater than "4"

DATA	1	2	3	4	5
INDEX	0	1	2	3	4

Step 2 - Swap

"3" and "4" are not swapped.

DATA	1	2	3	4	5
INDEX	0	1	2	3	4

Step 1 - Pair & Compare

Index 3 and Index 4 are paired.
No, "4" is not greater than "5"

DATA	1	2	3	4	5
INDEX	0	1	2	3	4

Step 2 - Swap

"4" and "5" are not swapped.

DATA	1	2	3	4	5
INDEX	0	1	2	3	4

THE BUBBLE SORT PROCESS

Having understood the mechanics and demonstrated the algorithm step by step, we can now take a closer look at the underlying operations and conditions that make the bubble sort process work. By identifying these key components, we can begin to organize the logic into clear instructions and structured loops that reflect how the algorithm actually functions.

DATA & INDICES

As in any algorithm, we must start by defining our variables. These variables must be declared before they are used, and each is assigned an initial value that may change as the algorithm runs.

- **hold_index** refers to the index whose data is temporarily held for comparison.
- **hold_data** refers to the data at the hold_index.
- **compare_index** refers to the index that precedes the hold_index, and against which the held data is compared.
- **compare_data** refers to the data at the compare_index.
- **end_index** refers to the last index in the list. In this case, it remains fixed throughout the sorting process.

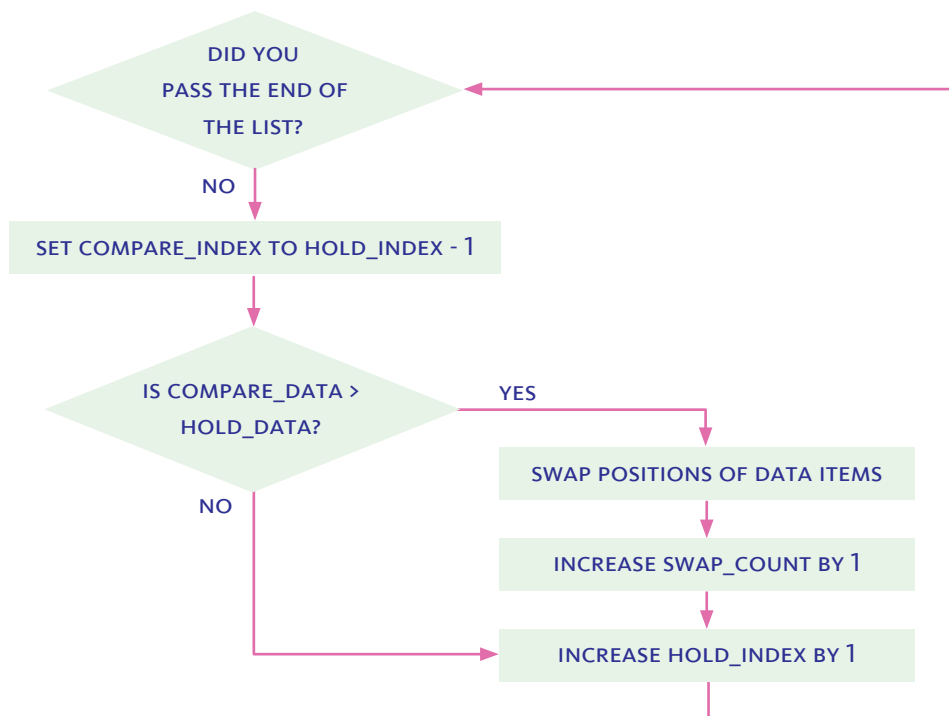
In addition to these variables—previously encountered in the insertion sort algorithm—the bubble sort algorithm requires an additional variable:

- **swap_count** is a variable that keeps track of the number of times a swap occurs during a single pass through the list.

LOOPS & CONDITIONS

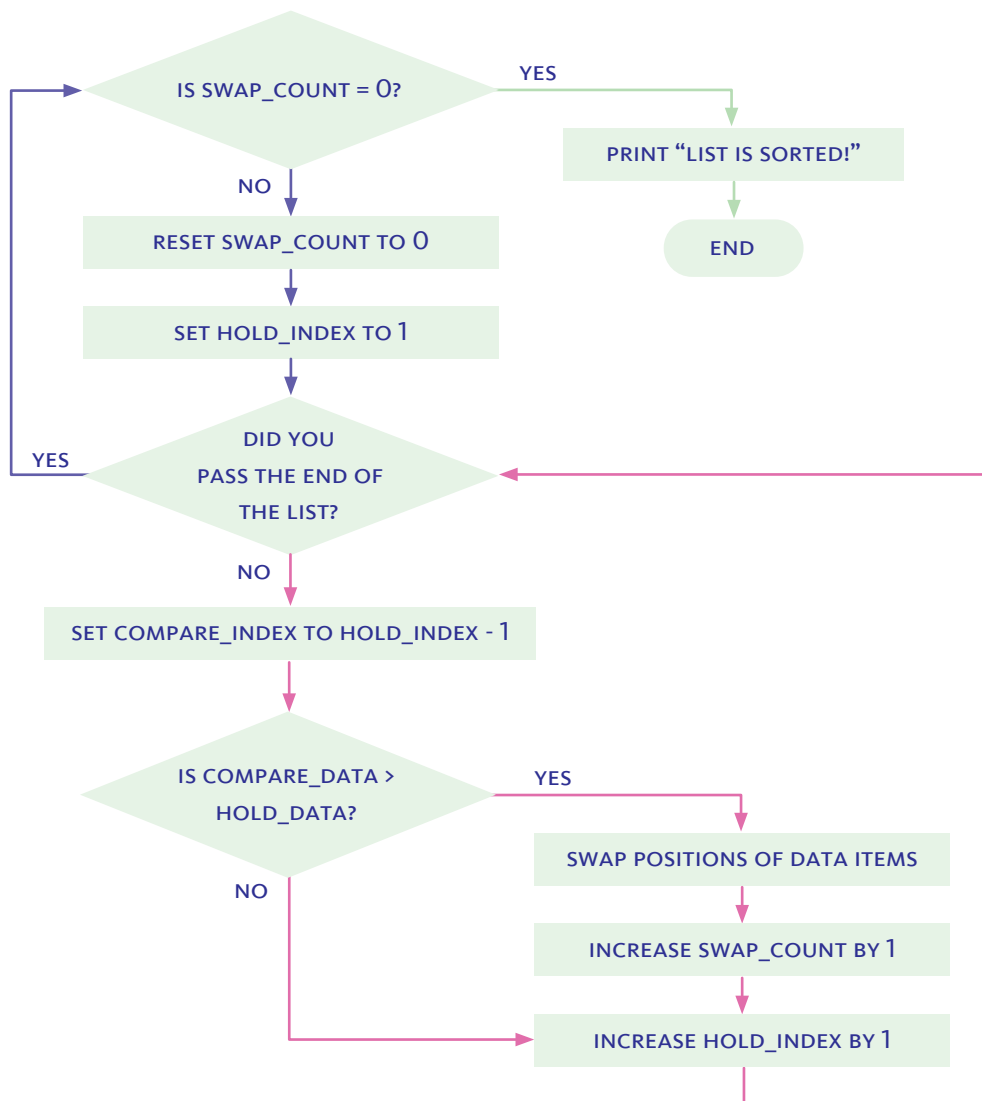
The bubble sort algorithm uses two loops: an inner loop and an outer loop. The inner loop performs a single pass through the list, pairing, comparing, and swapping adjacent data items. The outer loop repeats these passes until no swaps are needed, meaning the list is fully sorted.

The **inner loop** performs a series of steps that compare the held data item with the items before it in the list. If the compared data is greater than the held data, it is shifted forward by one position. This process repeats as long as that condition is true. Once the compared data is less than or equal to the held data, the condition is no longer satisfied. At that point, the loop ends, and the algorithm inserts the held data into the correct empty position toward the beginning of the list.



This flowchart illustrates the inner loop of the bubble sort process. It begins by setting the `compare_index` to one position before the `hold_index`, forming a pair of adjacent data items. The algorithm compares their values and swaps them if needed. Each time a swap occurs, the `swap_count` is increased. The `hold_index` then moves forward, and the loop continues until the end of the list is reached, completing a full pass.

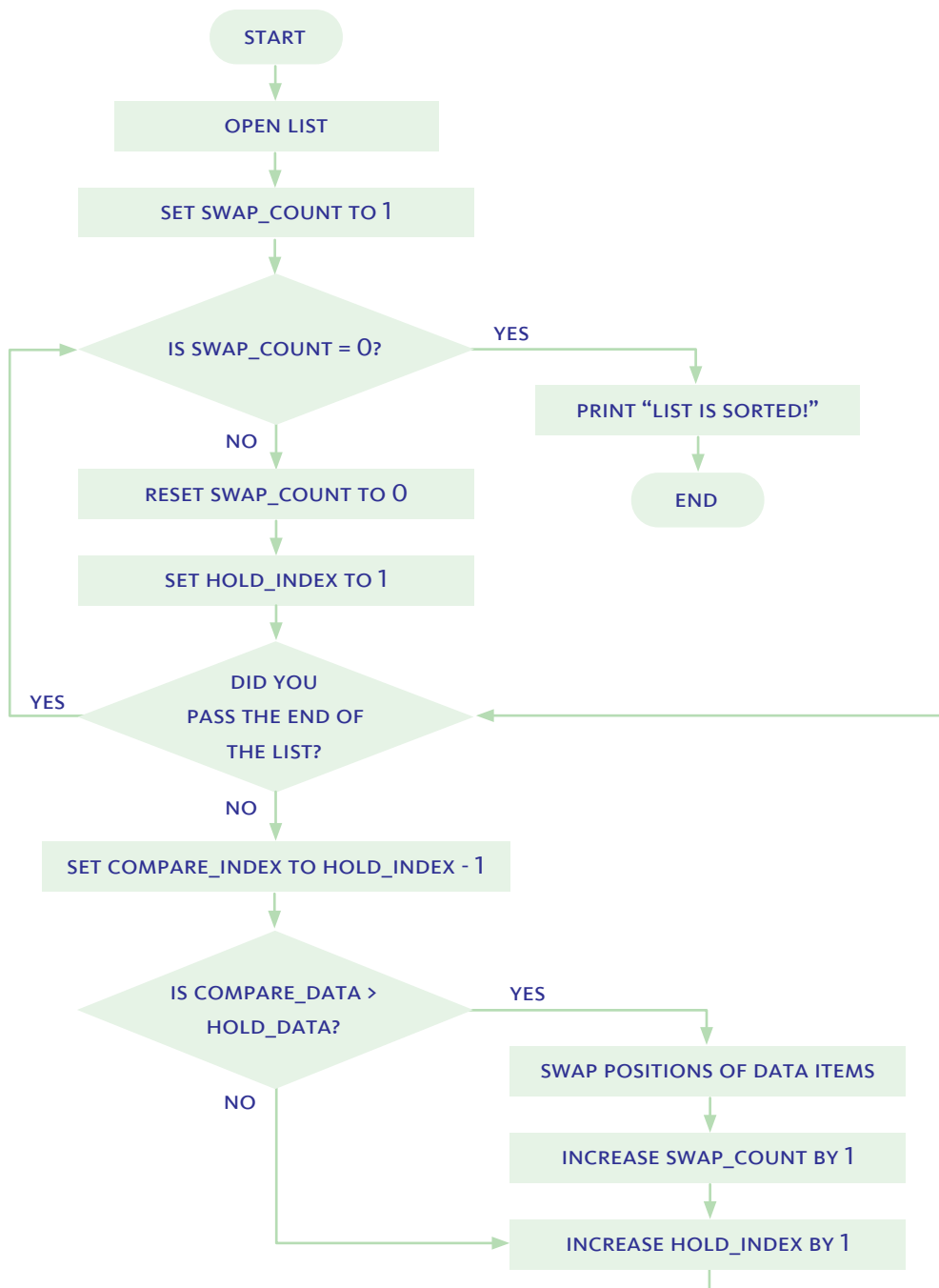
The **outer loop** is tasked with repeating the passes until the list is fully sorted. After each pass, it checks `swap_count`. If `swap_count` is 0, no swaps occurred, and the list is sorted. If not, the list is not completely sorted. The loop prepares for another pass by resetting `swap_count` to 0, moving `hold_index` back to the start, and running the pass again. This continues until a pass completes with no swaps.



The outer loop (blue) repeats passes through the list until no swaps occur. The inner loop (pink) performs a single pass, comparing and swapping adjacent data items as needed.

INITIALIZING

First, the list is opened, and the variable `swap_count` is set to 1. Since the loop stops when `swap_count` equals 0—meaning no swaps were made—we cannot start with it set to 0, or the algorithm would end before it begins. Setting it to 1 ensures that the sorting process is able to start.



THE BUBBLE SORT PROGRAM

Now that we understand how the bubble sort algorithm works and how its steps flow, we can begin writing the instructions for a program that implements it.

The program has four blocks:

1. **Initialization** — This is where we begin by opening the list and declaring the variables needed for the sorting process. Before a variable can be used, it must first be declared and given an initial value. In this case, we declare `end_index` and assign it the value of the last index in the list, setting the boundary for the sorting range. We also declare `swap_count` and assign it an initial value of 1 to ensure that the algorithm enters the outer loop for its first pass.



```
START BubbleSort
OPEN list
SET end_index (last index of list)
SET swap_count (1)

REPEAT_UNTIL swap_count (0)
  RESET swap_count (0)
  SET hold_index (1)

  REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    IF compare_data > hold_data THEN
      SWAP_POSITIONS (compare_data, hold_data)
      RESET swap_count (swap_count + 1)

    RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END BubbleSort
```

The pink brackets and green text indicate the initialization block of the bubble sort algorithm. These variables are essential to prepare the algorithm before the sorting process starts.

2. Outer loop — The outer loop controls the overall sorting process by repeating passes through the list until it is fully sorted.

It begins with a **REPEAT_UNTIL** instruction that continues looping until `swap_count` is 0, indicating no swaps occurred during the last pass. At the start of each pass, the algorithm resets `swap_count` to 0 and sets `hold_index` to 1, preparing to step through the list. This outer loop ensures that the sorting continues as long as necessary to bring all elements into order.

```
START BubbleSort
OPEN list
SET end_index (last index of list)
SET swap_count (1)

REPEAT_UNTIL swap_count (0)
  RESET swap_count (0)
  SET hold_index (1)

  REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    IF compare_data > hold_data THEN
      SWAP_POSITIONS (compare_data, hold_data)
      RESET swap_count (swap_count + 1)

  RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END BubbleSort
```

The pink bracket and green-highlighted text indicate the outer loop block of the bubble sort algorithm. This section controls how many passes the algorithm makes through the list, repeating the process until no swaps occur—signaling that the list is sorted.

- 3. Inner loop** — is responsible for performing a single pass through the list, comparing and swapping adjacent items if needed.

It begins with a **REPEAT_UNTIL** instruction that runs as long as `hold_index` is less than or equal to `end_index`. At each step, the algorithm sets `hold_data` to the value at `hold_index`, identifies `compare_index` as the position just before it, and retrieves `compare_data` from that position. If `compare_data` is greater than `hold_data`, the two values are swapped, and `swap_count` is increased by one. The `hold_index` is then incremented, and the loop continues through the rest of the list.

```
START BubbleSort
OPEN list
SET end_index (last index of list)
SET swap_count (1)

REPEAT_UNTIL swap_count (0)
  RESET swap_count (0)
  SET hold_index (1)

  REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    IF compare_data > hold_data THEN
      SWAP_POSITIONS (compare_data, hold_data)
      RESET swap_count (swap_count + 1)

    RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END BubbleSort
```

The pink bracket and green-highlighted text indicate the inner loop block of the bubble sort algorithm. This section performs comparisons and swaps between adjacent items as it moves through the list from left to right during a single pass.

- 4. Termination** — Once the sorting process is complete—meaning a full pass has been made with no swaps—the algorithm exits the outer loop. At this point, it prints the message “List is sorted!” to confirm that the list has been successfully ordered. The program then ends.

```
START BubbleSort
OPEN list
SET end_index (last index of list)
SET swap_count (1)

REPEAT_UNTIL swap_count (0)
  RESET swap_count (0)
  SET hold_index (1)

  REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    IF compare_data > hold_data THEN
      SWAP_POSITIONS (compare_data, hold_data)
      RESET swap_count (swap_count + 1)

  RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END BubbleSort
```

The pink bracket and green-highlighted text indicate the termination block of the bubble sort algorithm. This section confirms that the list is sorted and marks the end of the program.

Notes

AS_LONG_AS and REPEAT_UNTIL resemble natural language, making them ideal for beginners. They help students focus on how conditions control the flow of a program and highlight the key distinction between pre-check and post-check logic:

- AS_LONG_AS runs while a condition remains true.
- REPEAT_UNTIL runs until a condition becomes true.

PERFORMANCE

When more than one algorithm can solve the same problem, performance becomes an important way to compare them. Two measures of performance are **step count** and **execution time**.

STEP COUNT AND EXECUTION TIME ARE TWO MEASURES OF PERFORMANCE.

Step count refers to the number of steps required by the algorithm to complete the task—which, in this case, is sorting. This measure helps us understand the structure and logical efficiency of an algorithm. Fewer steps means the algorithm performs a smaller number of operations to complete its task.

Execution time refers to how long it takes for the program to run and sort a particular dataset. This measure tells us about performance in practice. Shorter execution time means the program finishes faster—but that also depends on how long each type of step takes.

Both measures are useful—and together, they give us a fuller picture of performance.

WORST-CASE SCENARIO

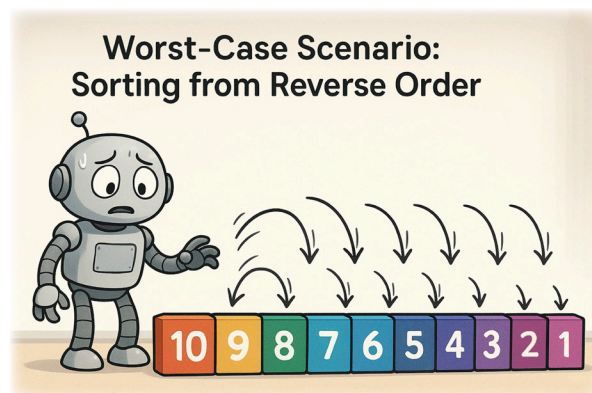
Algorithms and programs can perform differently depending on the dataset they are given. Since datasets can vary considerably, we typically use the worst-case scenario as a standard way to evaluate performance.

In the case of sorting, the worst-case scenario occurs when the list is arranged in the exact opposite order of the desired outcome. So, if we are trying to sort a list in ascending order, the worst case would be a list that starts in descending order. In this situation, the algorithm must shift or swap nearly every item to put the list in the correct order.

We use the worst case because:

1. It shows us the maximum number of steps an algorithm might take—this helps us understand its performance limits.
2. It prepares us for real-world situations where the data might be in the least helpful or most disorganized state.
3. It allows us to fairly compare algorithms by seeing how they handle the same challenging scenario.

In other words, if an algorithm performs well in the worst case, we can be confident that it will perform even better when the data is easier to work with.



INSERTION SORT

Let's explore the performance of insertion sort under the worst-case scenario: a list that is completely reversed.

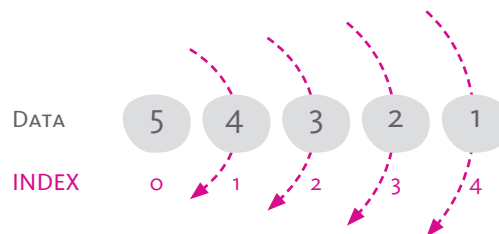
We will assume a list of 5 numbers that need to be sorted in ascending order. Since we want the worst-case, the list will start in descending order, meaning every number is in the opposite position of where it should be.

DATA	5	4	3	2	1
INDEX	0	1	2	3	4

STEP COUNT

When we examine the steps, we'll group them into two categories: **comparisons** and **moves**. The difference is that comparisons can be done with data in place, while moves involve the physical repositioning of data—either through a hold, shift, or insert.

The algorithm starts at index 1 and moves through the list one index at a time until the end. This process is controlled by the outer loop. For a list with 5 data items, the outer loop will run 4 times, and during each iteration, it will hold one item for sorting.



Each curved arrow represents one iteration of the outer loop in the insertion sort algorithm. At each index, the algorithm holds the current value and prepares to compare it with the values in all preceding indices. This process repeats from index 1 to the end of the list.

Now let's look at the inner loop. At each index, the inner loop compares the held data with the data at all preceding indices. Since this is a worst-case scenario, every comparison results in a shift.

- At index 1, there is 1 comparison and 1 shift.
- At index 2, there are 2 comparisons and 2 shifts.
- At index 3, there are 3 comparisons and 3 shifts.
- At index 4, there are 4 comparisons and 4 shifts.

Since this is a worst-case scenario, every comparison results in a shift. That means the number of shifts is equal to the number of comparisons, which again equals the number of preceding indices.

Finally, we are left with the insertions. These occur once during each iteration of the outer loop, giving us 4 insertions in total.

Index	Hold	Compare	Shift	Insert
1	1	1	1	1
2	1	2	2	1
3	1	3	3	1
4	1	4	4	1
Total	4	10	10	4

So for a 5-item dataset, under the worst-case scenario of a completely reversed list, the insertion sort algorithm requires a total step count of 4 (holds) + 10 (comparisons) + 10 (shifts) + 4 (insertions) = 28 steps.

EXECUTION TIME

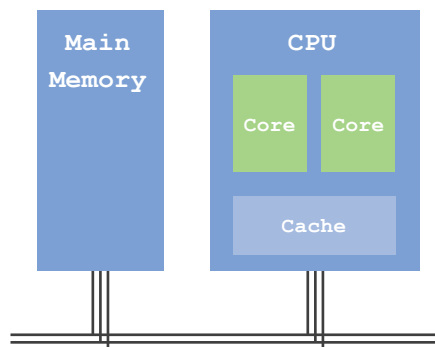
Execution time—ideally measured by actually running the program on a computer—can also be estimated using basic assumptions and an understanding of the steps the algorithm performs.

To keep our analysis realistic yet manageable, we categorize steps into two types:

1. Processor-only steps (like comparisons), which involve only the CPU cores and their internal registers.
2. Memory-related moves (like holds, shifts, and insertions), which involve moving data between memory locations, such as between cache and main memory (RAM).

In a real computer, memory access time can vary depending on whether data is stored in fast-access cache or slower main memory. However, since our

dataset is small, it's reasonable to assume that all memory-related steps occur within the processor's cache. Cache access is faster than accessing main memory, but still slower than internal CPU operations.



This figure is a simplified representation of the processor and memory components in a computer's architecture. The CPU contains multiple cores for executing instructions and a cache for fast, nearby access to frequently used data. The main memory (RAM) holds larger amounts of data for longer periods while programs run, but accessing it is slower than accessing the cache.

For the sake of estimation:

- We will assign 1 nanosecond (ns) to processor-only steps.
- We will assign 5 nanoseconds (ns) to memory-related steps, assuming cache-level latency.

This allows us to make performance comparisons based on an estimated execution time:

Notes

The time estimates used in these examples are intentionally simplified to support conceptual understanding. While actual timings depend on hardware architecture and system conditions, the goal is to help students recognize that different types of operations vary in cost—and that performance can be meaningfully estimated using reasonable assumptions.

- Each comparison = 1 ns
- Each hold, shift, or insert = 5 ns

By multiplying the number of each type of step by its estimated time cost, we can calculate the total execution time of the algorithm in nanoseconds.

This approach gives us a practical way to reason about performance, even without running the program on a real computer.

Index	Hold	Compare	Shift	Insert
1	1 (x 5)	1 (x 1)	1 (x 5)	1 (x 5)
2	1 (x 5)	2 (x 1)	2 (x 5)	1 (x 5)
3	1 (x 5)	3 (x 1)	3 (x 5)	1 (x 5)
4	1 (x 5)	4 (x 1)	4 (x 5)	1 (x 5)
Total	20 ns	10 ns	50 ns	20 ns

So for a 5-item dataset, under the worst-case scenario of a completely reversed list, the insertion sort program has an estimated execution time of 20 ns (holds) + 10 ns (comparisons) + 50 ns (shifts) + 20 ns (insertions) = 100 ns.

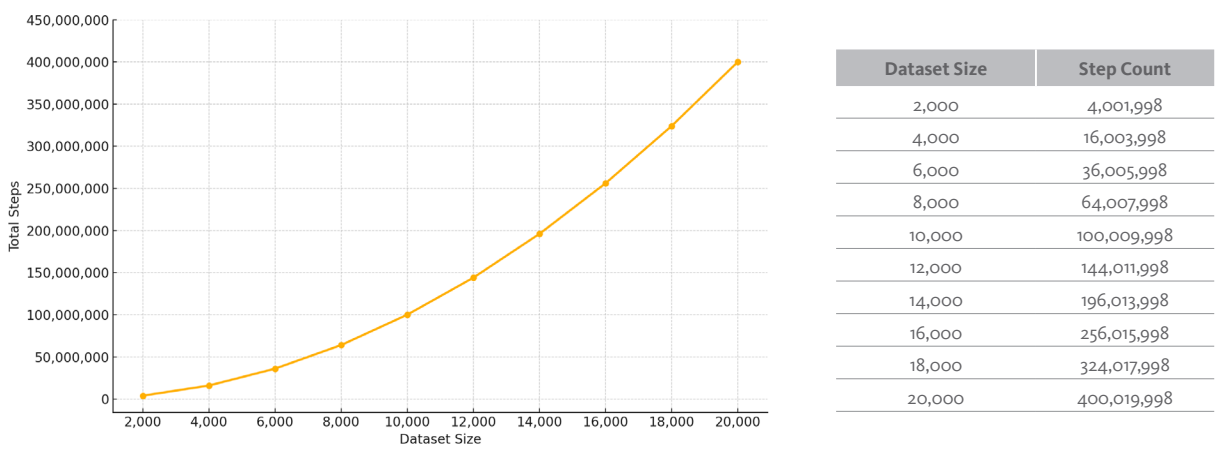
SCALABILITY

Now that we've explored how insertion sort performs on a small dataset, we can begin to explore what happens as the size of the dataset increases—a concept known as scalability.

As the dataset grows, both the step count and the execution time increase. This is especially true under the worst-case scenario, where the list is in completely reversed order.

As we scale up the size of the dataset, we can clearly see the impact on step count. With every new item added, the algorithm must compare it with all preceding items and shift them if necessary. This means that each additional item increases the total number of steps more than the one before it.

If we run simulations on datasets of increasing size—starting from 2,000 and going up to 20,000 items—we can demonstrate that, under the worst-case scenario, the step count grows rapidly. In fact, the amount of increase itself becomes greater with each step up in dataset size. This shows that the algorithm does not scale well and performs poorly as the size of the dataset increases.



This table and graph show how the total number of steps required by the insertion sort algorithm increases as the dataset size grows from 2,000 to 20,000 items under the worst-case scenario. The steep upward curve illustrates how step count accelerates with each additional item, highlighting the algorithm’s poor scalability for large datasets.

Notes

This is an example of quadratic growth. The total step count increases approximately with the square of the dataset size, which is why performance drops rapidly as the dataset grows.

Now let's examine how step count reflects on execution time. One might assume that execution time increases in the same way step count increases as the dataset grows—and while that's generally true, execution time behaves slightly differently. That's because it depends not only on the number of steps or operations, but also on how long each operation takes to execute.

The time required to execute an operation depends on where the data is stored. The closer the data is to the processor, the faster it can be accessed. Small datasets usually fit entirely in the CPU's cache, where access is very fast. But with large datasets, the processor may need to access main memory (RAM), which is much slower.

This introduces memory latency—the time it takes to fetch or store data from memory—which becomes a noticeable part of total execution time for large datasets.

EXECUTION TIME DEPENDS ON BOTH STEP COUNT AND MEMORY LATENCY.

Going back to our simulation, let's determine the memory latency for each operation. As the dataset increases in size, more memory is needed to hold it. This means the data is likely to be held further from the processor, moving from fast-access cache to slower main memory (RAM).

Because of this, we assign different latency values depending on the dataset size:

- For small datasets, the data is likely held in the CPU's cache, where access is very fast.
- For medium-sized datasets, the data may spill into L3 cache, which is slower.
- For larger datasets, the data is held in main memory (RAM), where access time is significantly longer.

Comparisons always occur within the CPU's registers and are very fast (typically around 1 nanosecond or less).

Memory-related moves—including holds, shifts, and insertions—require access to memory beyond the CPU and are more sensitive to dataset size and memory latency.

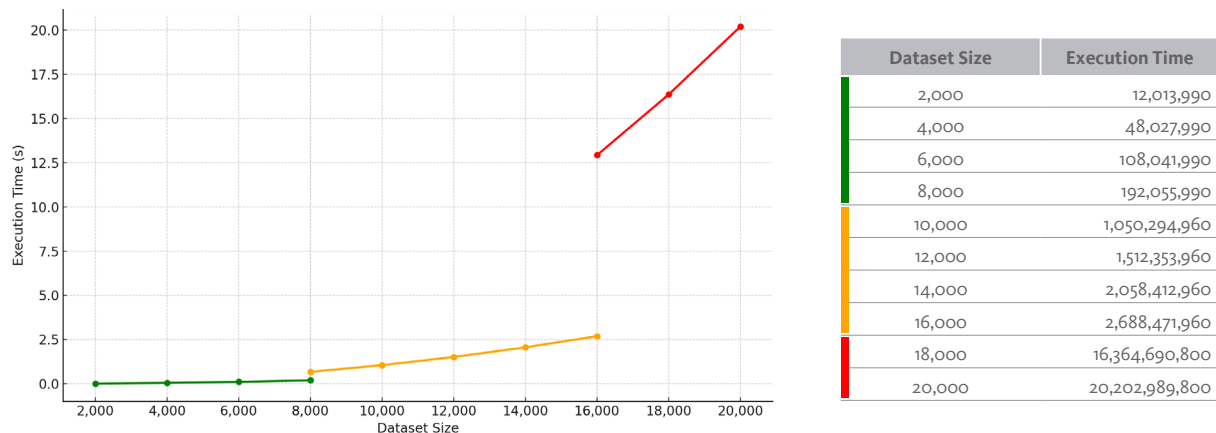
Dataset Size	Memory Location	Comparison (CPU)	Hold / Shift / Insert (Memory)
2,000–8,000	L1/L2 Cache	1 ns	5 ns
8,001–16,000	L3 Cache	1 ns	20 ns
16,001–20,000	Main Memory	1 ns	100 ns

According to these estimates, we can calculate execution times for each dataset size. As the dataset grows, it gradually begins spilling into higher memory tiers. Smaller datasets are held close to the processor in the L1 and L2 caches, but as the size exceeds the capacity of these caches, the data begins to spill into the L3 cache—which, in our simplified simulation, occurs at around 8,000 items. For even larger datasets that exceed the capacity of the L3 cache (around 16,000 items in the simulation), the data must be held in main memory (RAM).

Notes

In real systems, memory access doesn't shift abruptly between tiers. Instead, as the dataset grows, portions of it gradually move from fast-access cache to slower memory. Our simulation simplifies this behavior by assigning a dominant memory tier based on dataset size. This helps students understand the concept of memory latency without needing to model the complexities of real-world caching systems.

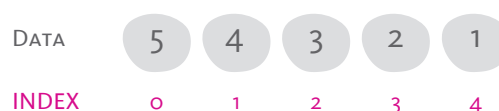
As a result, the algorithm slows down not only due to the increasing number of steps, but also due to memory latency—the added time required to access data stored in slower memory tiers.



This table and graph show how execution time increases as dataset size grows from 2,000 to 20,000 items using the insertion sort algorithm under the worst-case scenario. Execution time is shown in nanoseconds in the table and converted to seconds in the graph. The curve is divided into three color-coded segments based on the type of memory used—cache (green), L3 cache (orange), and main memory (red). The sharp rise in execution time highlights the combined effect of step count growth and increased memory latency, revealing the algorithm's poor scalability with larger datasets.

BUBBLE SORT

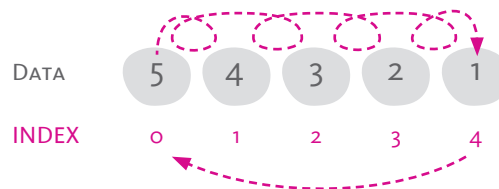
Let's explore the performance of the bubble sort algorithm in the same way we did for insertion sort. We'll start with a small list of 5 numbers that need to be sorted in ascending order. Since we're interested in the worst-case scenario, the list will begin in descending order—meaning that every number is in the opposite position of where it should be.



STEP COUNT

In the same way we examined step count for the insertion sort algorithm, we'll group the steps into two categories: comparisons and moves. The difference is that instead of holds, shifts, or insertions, bubble sort uses swaps to move data.

The bubble sort algorithm repeatedly compares adjacent pairs of data items and swaps them if they are in the wrong order. This process is controlled by the outer loop, which runs a fixed number of times regardless of whether the list becomes sorted earlier. For a list with 5 items, the outer loop runs 4 times, and during each pass, the inner loop performs 4 comparisons.



This diagram shows the structure of a single pass in the bubble sort algorithm. Each curved arrow represents a comparison between adjacent values. In the worst-case scenario, every comparison results in a swap. While the number of comparisons remains the same in every pass, the number of swaps decreases as larger values are moved to the end of the list.

Each pass in the bubble sort algorithm makes 4 comparisons (for a 5-item list), regardless of whether the list becomes partially sorted. The pattern of comparing adjacent pairs remains the same across all passes. However, the number of swaps decreases with each pass because the largest remaining value is moved into its correct position at the end of each round.

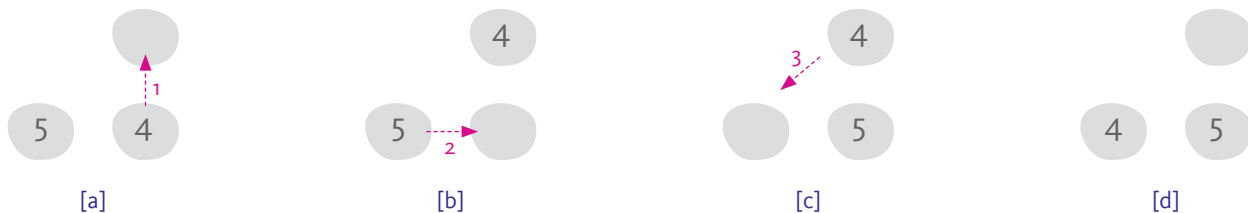
- In Pass 1, there are 4 comparisons and 4 swaps.
- In Pass 2, there are 4 comparisons and 3 swaps.
- In Pass 3, there are 4 comparisons and 2 swaps.
- In Pass 4, there are 4 comparisons and 1 swap.

So for a 5-item dataset, under the worst-case scenario of a completely reversed list, the bubble sort algorithm requires a total step count of 16 (comparisons) + 10 (swaps) = 26 steps.

Pass	Compare	Swaps
1	4	4
2	4	3
3	4	2
4	4	1
Total	16	10

However, it is important to examine the swap in detail as a type of move. A swap is not simply a single move of data from one place in memory to another; it actually requires three separate moves.

In real memory operations, one data item cannot simply overwrite a location already occupied by another. Instead, the existing value must first be moved to an empty temporary location, the target location is then overwritten with the new value, and finally, the temporarily held value is written into the second location. This sequence means that a single swap involves three distinct moves.



This figure shows the three moves involved in a swap operation during bubble sort. [a] The value 4 is moved to a temporary location. [b] The value 5 takes the place of 4. [c] The stored value 4 is moved from the temporary location into 5's original position. [d] The swap is complete, with 4 and 5 now in their new positions.

Accordingly, we must revise our step count to account for three moves per swap. This gives a total step count of 16 (comparisons) + 30 (moves) = 46 steps.

Pass	Compare	Swaps	Moves
1	4	4	12
2	4	3	9
3	4	2	6
4	4	1	3
Total	16	10	30

EXECUTION TIME

Having detailed the steps required for the bubble sort algorithm to sort a worst-case scenario list of 5 items, we can now discuss the time it would take to execute such a task. Using the same estimates of time as before—comparisons at 1 ns each and moves at 5 ns each—we can calculate the execution time.

- Comparisons: $16 \times 1 \text{ ns} = 16 \text{ ns}$
- Moves: $30 \times 5 \text{ ns} = 150 \text{ ns}$

This gives a total execution time of: $16 \text{ ns} + 150 \text{ ns} = 166 \text{ ns}$.

When we compare the results for a worst-case scenario list of 5 items, we see a clear difference between the two algorithms. Insertion sort required 28 steps and 100 ns to complete the task, while bubble sort required 46 steps and 166 ns.

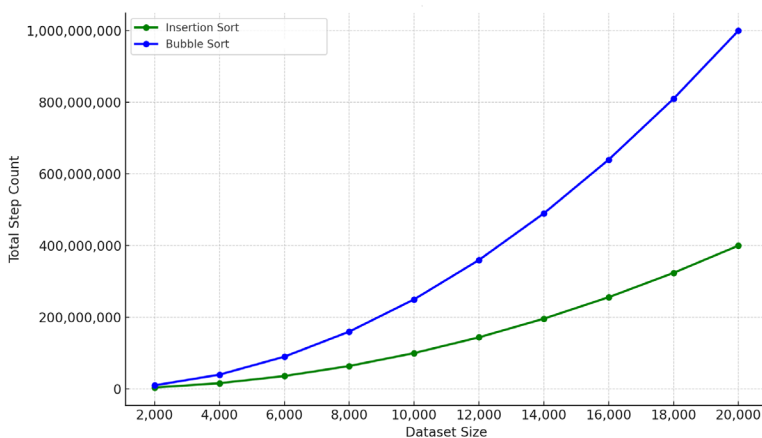
This means that bubble sort performed more operations and took longer to execute for the same dataset and conditions. Even at this small scale, the difference highlights bubble sort's lower efficiency compared to insertion sort in both step count and execution time.

SCALABILITY

While the difference in performance is already visible with a small dataset, it becomes even more pronounced as the dataset grows. Because bubble sort makes a fixed number of comparisons in every pass and performs swaps that require multiple moves, the total step count increases rapidly with larger lists.

As we saw with insertion sort, more steps mean more time—but for bubble sort, the higher number of moves per swap amplifies this effect. To understand the full impact, we need to examine how bubble sort scales with dataset size and how this affects execution time.

Let's return to our simulation using dataset sizes ranging from 2,000 to 20,000 data items. We will begin by comparing the insertion sort algorithm with the bubble sort algorithm, focusing on the scalability of their step counts.



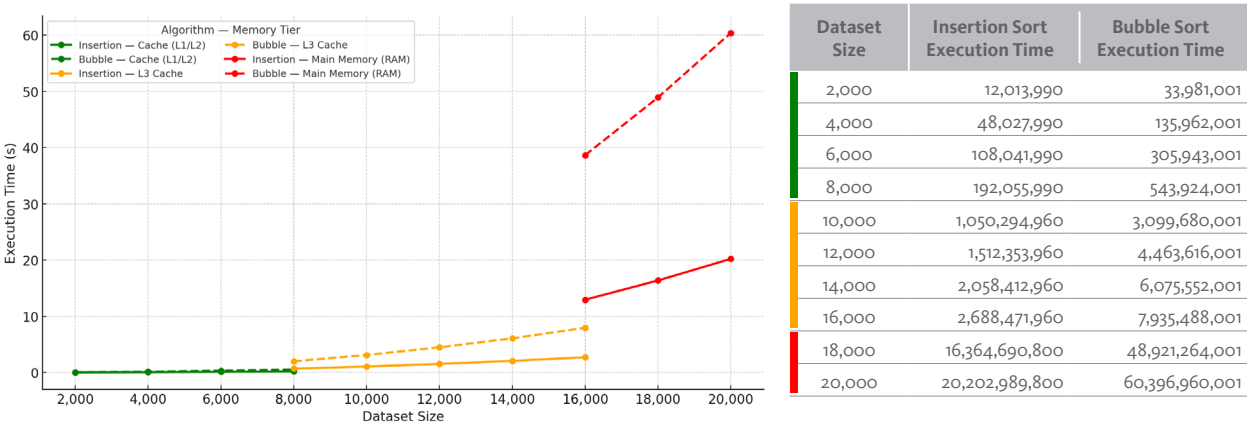
Dataset Size	Insertion Sort Step Count	Bubble Sort Step Count
2,000	4,001,998	9,993,001
4,000	16,003,998	39,986,001
6,000	36,005,998	89,979,001
8,000	64,007,998	159,972,001
10,000	100,009,998	249,965,001
12,000	144,011,998	359,958,001
14,000	196,013,998	489,951,001
16,000	256,015,998	639,944,001
18,000	324,017,998	809,937,001
20,000	400,019,998	999,930,001

This graph and table compare the total step counts for insertion sort and bubble sort under the worst-case scenario, for dataset sizes ranging from 2,000 to 20,000 items. Both algorithms exhibit quadratic growth, but bubble sort consistently requires about 2.5 times more steps. The gap between the two curves widens as dataset size increases, clearly illustrating bubble sort's poorer scalability.

Examining the graph and table, it becomes clear that bubble sort is far less efficient than insertion sort. The main reason is that bubble sort’s structure forces it to perform a fixed number of comparisons in each pass, regardless of how much of the list is already sorted.

In a worst-case scenario, every comparison results in a swap, and each swap involves three separate moves in memory. This extra work adds up quickly, making bubble sort require about 2.5 times more steps than insertion sort for the same dataset size. As the dataset grows, this difference becomes increasingly significant, highlighting how the mechanics of an algorithm can have a major impact on its performance.

These differences become even more pronounced when we calculate the execution time required for each dataset. As the dataset grows, it often exceeds the capacity of faster memory tiers (such as the CPU’s cache) and must be stored in slower tiers (such as L3 cache or main memory). This increase in memory latency further slows the execution of the sorting process, amplifying the performance gap between the two algorithms.



This graph and table compare execution times for insertion sort and bubble sort under the worst-case scenario for dataset sizes from 2,000 to 20,000 items. Execution time is shown in nanoseconds in the table and converted to seconds in the graph. Times are segmented by memory tier—cache (green), L3 cache (orange), and main memory (red). Bubble sort’s higher step count and costly swaps cause its execution time to grow much faster, especially as larger datasets move into slower memory.

Understanding algorithm performance goes beyond knowing whether an algorithm produces the correct result. How efficiently it solves a problem—measured in step count, execution time, and memory use—can make a significant difference, especially with large datasets or time-sensitive tasks.

As seen in our comparison of insertion sort and bubble sort, two algorithms with the same growth rate can perform very differently in practice due to their mechanics and the cost of individual operations. Choosing the right algorithm is therefore about matching the method to the problem, the dataset size, and the available computing resources.

SUMMARY

Sorting – the arrangement of data in a specific order based on a certain criterion.

- Order is the direction—ascending or descending.
- Criterion is what we sort by, such as exam scores, prices, names, or titles.
- Sorting helps organize data, making it easier to search, interpret, and analyze.

Algorithm & Program –

- An algorithm is a series of steps designed to perform a particular task.
- A program is the set of written instructions a computer can execute to run an algorithm.
- A variable is a label whose value or content can change. Variables in programming need to be declared, meaning their label is specified and an initial value is assigned.
- Flow is the sequence in which the steps of a program or algorithm are carried out.
- Control refers to the conditions that determine this sequence, such as when certain steps are executed, repeated, or skipped.
- Loops are a common control structure used to repeat a set of instructions, while conditional statements decide which instructions run based on specific criteria.

- A **block** is a segment of code that performs a particular task. A program is typically organized into three main blocks: initialization, main body, and termination. Blocks can be **sequential** (one after another) or **nested** (one inside another).
- Indentation is used to show which instructions belong to a loop block.

Insertion Sort – Builds a sorted section one item at a time.

- Outer Loop – Moves through each index in the list (starting at the second item) and hands control to the inner loop.
- Inner Loop – Compares the held item with preceding items, shifts items as needed, and inserts the held item into its correct position based on the sorting criterion.
- Strategy – Compare, shift, insert.

Bubble Sort – Repeatedly passes through the list, swapping adjacent items if out of order.

- Outer Loop – Repeats the inner loop for a fixed number of passes to ensure the list becomes fully sorted.
- Inner Loop – Compares adjacent pairs and swaps them if they are out of order, gradually moving items into their correct positions based on the sorting criterion.
- Strategy – Pair, compare, swap.

Performance –

- Step count and execution time are two key measures of performance.
- Worst-Case Scenario – The most challenging dataset arrangement for an algorithm (e.g., completely reversed list for ascending sort). Used for fair comparisons.
- Scalability refers to how an algorithm's performance changes as the dataset size increases.
- Memory latency is the delay between requesting data from memory and the moment it becomes available for use.
- Insertion sort is a more efficient algorithm than bubble sort, requiring fewer steps and less execution time. The difference widens as datasets grow, especially when memory latency becomes significant.

Activity 4.1

Read the following statements, and write on the line provided for each statement 'T' if it is true and 'F' if it is false.

T/F	STATEMENT
<u>T</u>	Sorting is the arrangement of data in a specific order based on a certain criterion.
<u>F</u>	In programming, a variable is a label whose value cannot change.
<u>F</u>	The order in sorting refers to the basis or criterion used to determine how data is arranged.
<u>T</u>	Flow refers to the sequence of steps in a program, while control refers to the conditions that decide which steps are executed, repeated, or skipped.
<u>T</u>	A program is typically organized into initialization, main body, and termination blocks.
<u>T</u>	In insertion sort, the inner loop compares the held item with preceding items, shifts items as needed, and inserts it in the correct position.
<u>F</u>	In bubble sort, each swap requires one move in memory.
<u>T</u>	Step count and execution time are two measures used to compare algorithm performance.
<u>F</u>	A worst-case scenario is the easiest dataset arrangement for an algorithm to sort.
<u>T</u>	As dataset size increases, the need for slower memory tiers can raise memory latency and increase execution time.

Activity 4.2

Answer the questions below.

1. What does “order” refer to in sorting?
 - a) The basis for sorting
 - b) The sequence in which algorithms are written
 - c) **The direction of arrangement**
 - d) The type of data being sorted

2. Which of the following is an example of a numeric sorting criterion?
 - a) Student names
 - b) **Exam scores**
 - c) Book titles
 - d) File names

3. Which statement about sorting criteria is correct?
 - a) They can only be numeric
 - b) **They can be numeric or alphanumeric**
 - c) They must be alphabetical
 - d) They must be in descending order

4. Which of the following is an example of alphanumeric data?
 - a) Temperatures
 - b) Prices
 - c) **ID codes**
 - d) Time

5. Why is sorting used?
 - a) To make programs run faster
 - b) **To organize data for easier search and analysis**
 - c) To reduce memory usage
 - d) To prevent data duplication

Activity 4.3

Answer the questions below.

1. What is an algorithm?
 - a) A program written in a specific language
 - b) A series of steps designed to perform a task
 - c) A list of variables
 - d) A method for compiling code
2. What is a program?
 - a) A set of instructions a computer can execute
 - b) A flowchart showing an algorithm
 - c) A memory location in a computer
 - d) A sorting method
3. What does declaring a variable involve?
 - a) Stating its label and assigning an initial value
 - b) Sorting its contents
 - c) Creating a loop for it
 - d) Printing it to the screen
4. Which of the following controls the order in which program steps run?
 - a) Sorting
 - b) Variables
 - c) Flow & control structures
 - d) Program termination
5. Which is NOT a typical program block?
 - a) Initialization
 - b) Main body
 - c) Termination
 - d) Compilation

Activity 4.4

Answer the questions below.

1. What is the basic strategy of insertion sort?
 - a) Pair, compare, swap
 - b) **Compare, shift, insert**
 - c) Merge, split, compare
 - d) Select, swap, compare

2. What does the outer loop in insertion sort do?
 - a) Compares adjacent pairs
 - b) Holds an item and inserts it in its correct position
 - c) **Moves through each index and hands control to the inner loop**
 - d) Swaps items until sorted

3. What does the inner loop in insertion sort do?
 - a) Pairs adjacent items for comparison
 - b) **Shifts items and inserts the held item in the correct position**
 - c) Selects the smallest value in the list
 - d) Divides the list into halves

4. Which step is NOT part of insertion sort?
 - a) Compare
 - b) Shift
 - c) Insert
 - d) **Swap**

5. In the worst-case scenario, how is the dataset arranged for insertion sort?
 - a) Already in the intended order
 - b) **In reverse of the intended order**
 - c) In random order
 - d) Grouped by type

Activity 4.5

Answer the questions below.

1. What is the basic strategy of bubble sort?
 - a) Compare, shift, insert
 - b) Merge, compare, shift
 - c) **Pair, compare, swap**
 - d) Select, swap, compare

2. What does the outer loop in bubble sort do?
 - a) Moves through the list starting at the second item
 - b) Compares each item with all previous items
 - c) **Repeats the inner loop until a pass completes with no swaps**
 - d) Inserts items into the sorted section

3. What does the inner loop in bubble sort do?
 - a) Shifts items forward
 - b) **Swaps adjacent items if out of order**
 - c) Holds an item for insertion
 - d) Divides the list in half

4. How many moves are required for one swap in bubble sort?
 - a) 1
 - b) 2
 - c) **3**
 - d) 4

5. In bubble sort, after each pass, which item is moved into its correct position?
 - a) The largest item
 - b) The smallest item
 - c) **The next item in the intended order**
 - d) The item with the highest index

Activity 4.6

Answer the questions below.

1. Which two measures are common for evaluating algorithm performance?
 - a) Order and criterion
 - b) Memory and variables
 - c) Flow and control
 - d) **Step count and execution time**

2. What is a worst-case scenario?
 - a) A program without loops
 - b) The least challenging arrangement of data
 - c) **The most challenging arrangement of data**
 - d) A dataset with missing values

3. What does scalability describe?
 - a) How well an algorithm works on small datasets
 - b) **How performance changes as dataset size increases**
 - c) How memory is allocated in a program
 - d) How variables are declared

4. What is memory latency?
 - a) The speed of the processor
 - b) **The delay between requesting data from memory and receiving it**
 - c) The time taken to write a program
 - d) The order in which data is stored

5. Why is insertion sort generally more efficient than bubble sort?
 - a) It requires more swaps
 - b) It works only on small datasets
 - c) **It requires fewer steps and less execution time**
 - d) It avoids using loops

Activity 4.7

In this activity, you will use a spreadsheet application to sort three different lists of numbers. You will then extract specific information from each list before sorting and after sorting, and reflect on the time and effort it took.

Download the spreadsheet here: bit.ly/Activity4_7

- a. List 1 – 50 Random 6-Digit Numbers (100,000–999,999)

The smallest number: 102372

The largest number: 955156

The duplicate number: 312944

Find the number specified in the spreadsheet.

- b. List 2 – 100 Random 6-Digit Numbers (100,000–999,999)

The smallest number: 102372

The largest number: 955156

The duplicate number: 704924

Find the number specified in the spreadsheet.

Did the larger dataset size make it easier or harder to find the data without sorting? Explain:

It harder to find values without sorting because there is more data to scan.

Sorting makes locating values much faster.

- c. List 3 – 100 Random 6-Digit Numbers (222,222–333,333)

The smallest number: 222354

The largest number: 331588

The duplicate number: 319738

Find the number specified in the spreadsheet.

Did the larger dataset size make it easier or harder to find the data without sorting? Explain:

Numbers look more alike, making numbers harder to spot.

Sorting makes locating values much faster.

Activity 4.8

In this activity, you will work with data presented in table form, each representing a real-world scenario. Sort the data and extract the information needed to answer the questions. Use sorting and functions such as COUNT and AVERAGE to investigate and answer.

Download the spreadsheet here: bit.ly/Activity4_8

- a. You are given an excerpt from a school database for Grades 7 and 8 with the following fields: Grade, Section, Student ID, and GPA.

1. How many students are in each grade?

Grade 7 — 32 ; Grade 8 — 35

2. If the school wants to recognize the top 3 students in each grade, who are they? List their student IDs.

Grade 7 — ID-2885, ID-2696, ID-7516

Grade 8 — ID-7309, ID-1489, ID-2598

3. If the school is offering support to students with a GPA below 2.00, how many students in each grade would qualify?

Grade 7 — 8 students

Grade 8 — 7 students

4. Looking at the number of students in each section, which section in each grade has the fewest students and which has the most?

Grade 7 — fewest A (9), most B (12)

Grade 8 — fewest B (10), most C (13)

5. If the school wants to recognize the teacher whose section has the highest average GPA, which section would win? Show the average GPA for each section to support your answer.

Grade 7 — A: 2.65, B: 2.60, C: 2.76 (highest)

Grade 8 — A: 2.59, B: 3.35 (highest), C: 2.88

- b. You are given an excerpt from a logistics company's delivery database showing: Order Number, Route, Day of Delivery, and Time En Route (minutes).

1. Which order was delivered the fastest, and which was the slowest?

Fastest: 700182 (29 min) ; Slowest: 700125 (62 min)

2. For each delivery day, which order was delivered the fastest?

Monday: 700115 (35 min) ; Wednesday: 700139 (35 min) ; Friday: 700182 (29 min)

3. Deliveries that require more than 50 minutes are considered slow. Compare delivery days and routes by the number of slow deliveries. What stands out?

By day — Monday: 15, Wednesday: 6, Friday: 5

By route — Route 1: 7, Route 2: 4, Route 3: 15

Monday and Route 3 have the most slow deliveries.

4. Which route has the lowest average delivery time? Show the average time for each route to support your answer.

Route 2 — 44.10 min (best) ; Route 1 — 45.63 min ; Route 3 — 47.83 min

5. For each delivery day, which route has the lowest average delivery time? Recommend the best route to take on each day and support your answer with the average times.

Monday: Route 2 — 38.10 min

Wednesday: Route 1 — 38.90 min

Friday: Route 3 — 38.30 min

Notes

Use the tables in this lesson to explain that databases store information in tables, with fields (column headings) and entries (rows of data). Point out that sorting, counting, and averaging are basic operations used in database queries, laying the groundwork for more formal database concepts in later lessons.

Activity 4.9

Use the Insertion Sort algorithm to arrange the two given lists in ascending order. For each step, record the number of comparisons, shifts, and inserts you perform, and write the updated list. Keep a running total of the comparisons, shifts, inserts, and total operations at the bottom of the table.

a. [C, D, A, B]

Index	Hold	Compare	Shift	Insert	List
1	1	1	0	0	C, D, A, B
2	1	2	2	1	A, C, D, B
3	1	3	2	1	A, B, C, D
Total	3	6	4	2	15

b. [1, 3, 0, 2, 4, 5]

Index	Hold	Compare	Shift	Insert	List
1	1	1	0	0	1, 3, 0, 2, 4, 5
2	1	2	2	1	0, 1, 3, 2, 4, 5
3	1	2	1	1	0, 1, 2, 3, 4, 5
4	1	1	0	0	0, 1, 2, 3, 4, 5
5	1	1	0	0	0, 1, 2, 3, 4, 5
Total	5	7	3	2	17

Activity 4.10

Use the Bubble Sort algorithm to arrange the two given lists in ascending order. For each pass through the list, record the number of comparisons and swaps you perform, and write the updated list. Keep a running total of the comparisons, swaps, and total operations at the bottom of the table.

a. [B, C, D, A]

Pass	Compare	Swap	List
1	3	1	B, C, A, D
2	3	1	B, A, C, D
3	3	1	A, B, C, D
Total	9	3	12

b. [2, 4, 0, 3, 5, 1]

Pass	Compare	Swap	List
1	5	3	2, 0, 3, 4, 1, 5
2	5	2	0, 2, 3, 1, 4, 5
3	5	1	0, 2, 1, 3, 4, 5
4	5	1	0, 1, 2, 3, 4, 5
5	5	0	0, 1, 2, 3, 4, 5
Total	25	7	32

Activity 4.11

Review the pseudocode program for Insertion Sort. Then answer the comprehension questions provided.

```
START InsertionSort
OPEN list
SET hold_index (1)
SET end_index (last index of list)

REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    AS_LONG_AS hold_data > compare_data
        SHIFT compare_data FROM compare_index TO compare_index + 1
        RESET compare_index (compare_index - 1)
        RESET compare_data (data_item AT compare_index)

    INSERT hold_data TO compare_index
    RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END InsertionSort
```

1. Which two variables are declared in the initialization block?

hold_index, end_index

2. What does this instruction do: RESET compare_index (compare_index - 1)?

This instruction moves the comparison one position to the left.

3. What does this instruction do: REPEAT_UNTIL hold_index > end_index?

Ensures the program ends when it reaches the end of the list.

4. Which instruction moves the sorting to the next index in the list?

RESET hold_index (hold_index + 1)

5. Which instruction would you change to sort the list in descending order?

Change the instruction to: AS_LONG_AS hold_data < compare_data

Activity 4.12

Review the pseudocode program for Bubble Sort. Then answer the comprehension questions provided.

```
START BubbleSort
OPEN list
SET end_index (last index of list)
SET swap_count (1)

REPEAT_UNTIL swap_count (0)
  RESET swap_count (0)
  SET hold_index (1)

  REPEAT_UNTIL hold_index > end_index
    SET hold_data (data_item AT hold_index)
    SET compare_index (hold_index - 1)
    SET compare_data (data_item AT compare_index)

    IF compare_data > hold_data THEN
      SWAP_POSITIONS (compare_data, hold_data)
      RESET swap_count (swap_count + 1)

    RESET hold_index (hold_index + 1)

PRINT "List is sorted!"
END BubbleSort
```

1. Which two variables are declared in the initialization block?

end_index, swap_count

2. What does this instruction do: REPEAT_UNTIL swap_count (0)

Repeats passes through the list until no swaps occur.

3. Which instruction moves the sorting to the next index in the list?

RESET hold_index (hold_index + 1)

4. What happens in RESET swap_count (swap_count + 1)?

It increases the swap counter by one each time a swap is made.

5. Which instruction would you change to sort the list in descending order?

Change the condition to IF compare_data < hold_data THEN.

Activity 4.13

Having learned the mechanics, performance, and logic behind insertion and bubble sort algorithms, it's time to explore them further. This activity allows students to examine these algorithms in action using provided Python programs. Follow the links to download the three programs for this activity or use the short link: bit.ly/Activity4_13.

1. [Insertion Sort Program](#): Creates a list from all integers in a user-defined range (maximum 20 items). The list can be generated in worst-case (descending) or random order. The program shows the list after each completed insert (labeled by index) and reports totals for comparisons, shifts, and inserts.
2. [Bubble Sort Program](#): Builds the same kind of list (worst-case or random). The program shows the list after each completed pass (labeled by pass number) and reports totals for comparisons and swaps.
3. [Combined Sorting Program](#): This program includes both insertion and bubble sort. It generates a dataset from all integers in a user-defined range and runs both algorithms on the same list.
 - If the list size is 10,000 items or fewer, the program runs both algorithms and reports operation counts and execution time.
 - If the list size is larger than 10,000, the program offers a Theory Mode that displays the worst-case (reverse-sorted) operation counts without running the algorithms.

Notes

Notice the similarity between the pseudocode and the Python programs provided for insertion and bubble sort. This shows how pseudocode serves as a bridge between algorithmic logic and actual coding. By following the same variable names and structure, students can clearly see how the logic of each step is translated into Python. Encourage students to use pseudocode as a guide to understand sorting processes, compare algorithm performance, and develop their programming skills.

4

ARTIFICIAL INTELLIGENCE

LESSON 1: MACHINE LEARNING

One of the fundamental attributes of an intelligent system is **learning**—the ability to acquire and use information. Learning involves creating internal links, or **associations**, between what is already known and new information. At its core, learning is the capacity to form new associations and to make use of them.

LEARNING IS THE ABILITY TO FORM AND USE NEW ASSOCIATIONS.

EXAMPLE 1.1

Consider a toddler. Through experience, she recognizes the shape of a cup and understands that it contains water to quench her thirst. These are associations already present in her mind: <cup as an object>, <water>, <thirst>. She learned them through repeated experience.

Now something new happens. Maya feels thirsty and points to the cup. Her mother says, "cup" and hands Maya the cup. The next day, the same event repeats: Maya points to the cup, her mother says, "cup," and gives it to her. After a few days of this repeating pattern, Maya feels thirsty but does not see the cup nearby. This time, she looks at her mother and says, "cup."



Through repetition, Maya recognized a pattern: whenever she heard the word 'cup', she received the cup itself. This formed an association in her mind between the word 'cup' and the object. She learned that the object's name is 'cup'.

Maya has learned. She created a new association in her mind: linking the object cup to the word ‘cup’. This learning occurred because she recognized a repeating pattern—the sound "cup" was always paired with the actual cup—and reinforced it through experience.

From this simple example, we can identify three fundamental attributes of learning:

1. It requires the creation of links or **associations** between things.
2. It depends on the ability to recognize **patterns**.
3. It usually requires **repetition** to strengthen those links.

MACHINE LEARNING

Artificial Intelligence (AI) is the field of computer science that focuses on building machines that can perform tasks that usually require human intelligence.

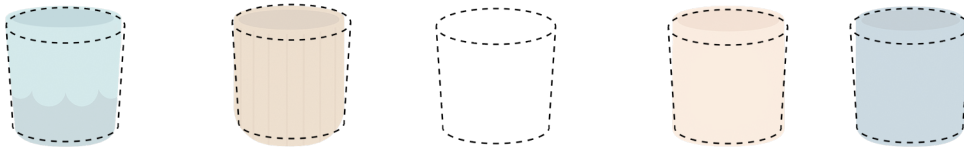
One branch of AI is called machine learning—this is when a computer is designed to have learning capabilities like Maya’s. Through repetition, the computer can identify patterns in data and form internal associations in memory. Once these associations are established and reinforced, the computer can make use of them.

EXAMPLE 1.2

An AI system can also be taught to identify a cup. To do this, it is given an image of a cup and is instructed to associate that image with the word ‘cup’. But to ensure it can recognize cups of all kinds, the computer is shown many images of cups in different sizes, colors, and forms.



By reviewing these images repeatedly, the computer begins to identify the general shape that defines a cup. This ability is called pattern recognition—the capacity to detect recurring regularities, in this case, the overall shape of a cup.



Shared Shape

These images show cups of different sizes, colors, and details. By seeing many different examples, the computer begins to identify the shared shape that makes an object a cup, even when the style, color, or design changes. The ability to identify this shared shape is called pattern recognition.

PATTERN RECOGNITION IS THE ABILITY TO DETECT RECURRING REGULARITIES.

TYPES OF LEARNING

AI systems, like humans, can learn in different ways. There are two main types of machine learning: supervised learning and unsupervised learning.

Supervised learning is like teaching. Maya was given a cup and told, “This is a cup.” This happened repeatedly until she learned to recognize the shape of a cup and associate it with the word ‘cup’.

For an AI system, supervised learning works in a similar way. The computer is given data that already has labels and must find the pattern that links the data to those labels. For example, just as in the previous case, the AI is shown many images of cups, each labeled ‘cup,’ and it learns to associate the recurring shape with the label.

On the other hand, **unsupervised learning** is more like exploring. The AI is not told what to look for. Instead, it analyzes the data, identifies patterns, and groups similar items together based on shared features. Labels are not used during learning, but they can be added afterward to name each group once the patterns are understood.

EXAMPLE 1.3

Imagine Maya walking around the house each day and grouping objects by similarity: cups in one pile, shoes in another, crayons in another. Later, someone tells her the names of these groups: “cups,” “shoes,” and “crayons.” In this way, she would have learned without being told the names in advance.



In unsupervised learning, similar objects are grouped together without being told their names. Later, labels such as ‘cups,’ ‘shoes,’ and ‘crayons’ can be added to each group.

In a similar way, an AI system may be instructed to review hundreds of unlabeled images of household objects and group them based on recurring patterns of shape, color, or design. In such a case, it is said that the AI has learned unsupervised. The labels can be added to those groups afterward.

TWO COMMON TYPES OF LEARNING ARE SUPERVISED LEARNING
AND UNSUPERVISED LEARNING.

BUILDING THE MODEL

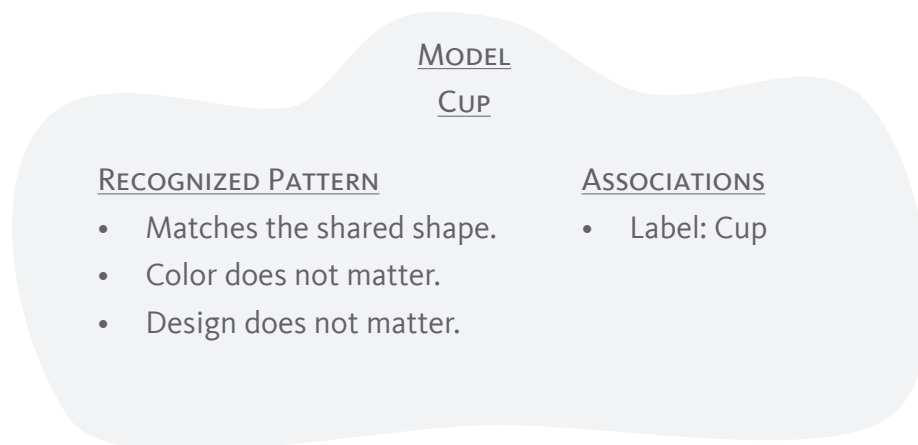
The process of learning results in the creation of a model. A model is the AI's internal representation of what it has learned — made up of the patterns it has recognized, the labels associated with those patterns, and the rules it has formed about which features are important and which are not.

A **MODEL** IS THE INTERNAL SYSTEM THAT REPRESENTS RECOGNIZED PATTERNS AND THE ASSOCIATIONS FORMED FROM THEM.

EXAMPLE 1.4

In this example, we will explore what would be contained in the AI's model of a cup. The most important element is the shared shape—the recurring pattern the AI has learned to identify. But the model also includes refinements, or rules, about features that do not affect whether something is a cup.

For instance, the style, color, or decorative designs are all noted but explicitly



This diagram shows a model of a cup. On the left are the recognized patterns that define a cup. The model identifies the shared shape as essential while explicitly instructing the AI that other variations—such as style, color, or design—do not matter for recognizing a cup. On the right are the associations linked to this pattern, namely the label 'cup'.

treated as irrelevant. As long as the object fits the shared shape, the model recognizes it as a cup.

Finally, the model includes the associations connected to this pattern, such as the label ‘cup’. This enables the computer to generalize, applying what it has learned to correctly identify cups it has never seen before.

TRAINING THE MODEL

Once a model is built, it must be tested to evaluate how well it performs. This testing is done on new data—examples the AI has not seen before. The goal is to determine whether the model can generalize what it has learned and accurately classify new inputs, such as recognizing a cup in an unfamiliar image.

Testing does not happen in isolation. As part of this process, the model’s predictions are compared to the correct answers. When it makes errors—known as misclassifications—those mistakes are analyzed. The model is then refined by adjusting how it interprets patterns and associations.

This ongoing process of learning from mistakes and improving performance by updating the model is known as training.

TRAINING IS THE PROCESS OF IMPROVING A MODEL USING NEW DATA AND CORRECTIONS.

Notes

While there is an official training phase that occurs after a model is first built and before it is released for use, training is not limited to that stage. In reality, training—or more accurately, refinement—is an ongoing process.

EXAMPLE 1.5

Now that the model for a cup has been built, it is tested on new images the AI has not seen before. The AI attempts to classify each image using the model it has created for 'cup'. It checks for the recurring pattern it has learned, disregards irrelevant features like color and design, and predicts which images contain cups based on that pattern.

The model correctly classified some images, but misclassified others. Upon review, it was found that the cups the model missed were those that had handles. The AI was informed that these examples were also cups and had been incorrectly excluded. It was then instructed to review those images, compare the patterns, and refine its model to correct the error.



Correctly Classified



Misclassified

This image shows how the model performed when tested on new data. On the left are cups that the model correctly classified based on the learned pattern. On the right are cups that were misclassified—these were not recognized because the model had not yet learned to accept handles as part of the pattern for 'cup'.

After reanalyzing the images, the model identified a new feature—the handle—and recognized that some cups have handles while others do not. It updated its internal representation accordingly, forming a new rule: “Handles do not matter.”

MODELCUPRECOGNIZED PATTERN

- Matches the shared shape.
- Color does not matter.
- Design does not matter.
- Include handles.

ASSOCIATIONS

- Label: Cup

After training, the model is updated to include cups with handles. It adds a rule to include handles as part of the recognized pattern, allowing the model to classify as ‘cup’ any object that matches the shared shape—whether it has a handle or not.

Now that the initial phase of training is complete, the model is ready for use. But learning does not stop here. While there is an official training phase that takes place after a model is first built and before it is released, training is not limited to that stage.

In practice, learning—or more precisely, refinement—is an ongoing process. As the model encounters new data, makes mistakes, or operates in changing conditions, it can continue to be adjusted and improved.

LEARNING IS A CONTINUOUS PROCESS.

Notes

Remind students: Learning doesn’t stop after the test — it’s a continuous process.

Just like an AI model, we don’t get everything right the first time. We learn through practice, mistakes, feedback, and adjustment. The more we apply what we’ve learned and reflect on our errors, the stronger our internal models become.

SUMMARY

Learning – the ability to form and use new associations.

- The three fundamental attributes of learning:
 1. the ability to recognize patterns.
 2. the ability to form associations.
 3. the need for repetition.
- **Pattern recognition** – the ability to detect recurring regularities.
- Two common types of learning are **supervised** (with labels) and **unsupervised** (without labels).
- **Model** – the internal system that represents recognized patterns and the associations formed from them.
- **Training** – the process of improving a model using new data and corrections.
- Learning is a continuous process.

Activity 1.1

Read the following statements, and write on the line provided for each statement 'T' if it is true and 'F' if it is false.

T/F	STATEMENT
<u>T</u>	Learning is the ability to form and use new associations.
<u>F</u>	Pattern recognition allows a system to detect and respond to random or unrelated features.
<u>T</u>	Supervised learning uses labeled data.
<u>F</u>	Unsupervised learning means the system receives labels in advance and uses them to form associations.
<u>T</u>	A model is the internal system that holds the patterns and associations a computer has learned.
<u>F</u>	Once a model is created, it can no longer be changed or improved.
<u>T</u>	Training is the process of improving a model by using new data and correcting its mistakes.
<u>F</u>	Learning is a one-time event that stops once the initial model is built.
<u>T</u>	A well-trained model can generalize its learning to identify patterns it has never seen before.
<u>T</u>	Repetition helps reinforce associations and improve pattern recognition.

Activity 1.2

Answer the questions below.

1. What is the best definition of learning?
 - a) The ability to remember everything perfectly
 - b) The ability to perform tasks without error
 - c) **The ability to form and use new associations**
 - d) The process of storing all inputs in memory

2. Which of the following is NOT a fundamental attribute of machine learning?
 - a) Recognizing patterns
 - b) Repeating examples
 - c) **Memorizing facts**
 - d) Creating associations

3. What does pattern recognition allow a system to do?
 - a) Respond only to labeled data
 - b) **Detect recurring regularities in inputs**
 - c) Ignore all differences in data
 - d) Randomly sort unrelated data

4. In pattern recognition, which kind of feature is most likely to be ignore?
 - a) The shared structure across examples
 - b) A recurring sound pattern
 - c) The repeated shape of an object
 - d) **An irrelevant design variation**

5. In supervised learning, what is given to the AI system to help it learn?
 - a) Data with missing pieces
 - b) **Data labeled with correct answers**
 - c) Completely random patterns
 - d) Unlabeled examples from multiple sources

 Activity 1.2 (continued)

Answer the questions below.

6. Which statement describes unsupervised learning?
 - a) **The AI finds patterns without labels**
 - b) The AI is told which output is correct
 - c) The AI memorizes all data examples
 - d) The AI ignores unknown patterns

7. What is a model in machine learning?
 - a) A database of all stored inputs
 - b) A system of memorized examples
 - c) A set of outputs from one training session
 - d) **An internal system of learned patterns and associations**

8. What does it mean when a model can generalize?
 - a) It copies exact answers from training data
 - b) It makes up patterns without learning
 - c) It erases data to free memory space
 - d) **It applies learned patterns to new, unseen data**

9. What happens when a model makes a misclassification?
 - a) It automatically deletes the data
 - b) The model is tested on easier data
 - c) **It is retrained or refined to improve**
 - d) The model shuts down for safety

10. Which statement best reflects the idea of continuous learning?
 - a) A model is fixed and never changes after being used
 - b) **Learning continues as the model encounters new data and errors**
 - c) Once trained, a model no longer needs updates
 - d) Learning stops once the system is deployed

Activity 1.3

Read the narrative and answer the questions below.

Plant Learning

A team of scientists is using machine learning to build a system that can identify different types of plants. At first, they collect thousands of plant images and label each one with its name (e.g., “rose,” “sunflower,” “tulip”). They use these labeled examples to teach the AI system what each plant looks like. Over time, the AI begins to recognize important patterns in the shape of the leaves and petals, while ignoring variations like background color or lighting.

Once the model is built, it is tested on a new set of plant images it has never seen before. It correctly identifies many of them but fails to recognize some roses that were photographed from an unusual angle. The scientists examine these mistakes and add more examples showing roses from different angles. The system updates its model and improves its accuracy.

1. What type of learning is being used to teach the AI system to identify plants, and how do you know?

Supervised learning, because the AI system was trained on labeled data (each image had the plant's name).

2. What features did the model learn to focus on, and which did it learn to ignore?

It learned to focus on shapes of leaves and petals (important features) and to ignore background color and lighting (irrelevant features).

3. Why is it important to have the model tested on images it had never seen before?

To check if it could generalize — apply what it learned to new situations.

4. Why did the model fail to recognize some of the roses? What was missing?

The model failed because it hadn't seen enough variety in angle or perspective during training, so it didn't recognize roses in unfamiliar views.

5. How did the model improve, and what process does this describe?

The model improved through training/refinement — it learned from its mistakes and updated its internal representation to become more accurate.

Activity 1.4

Read the narrative and answer the questions below.

Calls of the Wild

Scientists are studying animal sounds recorded in a rainforest. They have thousands of short audio clips, but most of them are not labeled — they don't know which species made each sound. Instead of guessing or labeling by hand, they use unsupervised learning to help them organize the data.

The AI analyzes the audio clips and looks for recurring patterns in pitch, rhythm, and frequency. Over time, it groups sounds that are similar into distinct clusters. Each cluster represents a sound pattern group, but the computer does not know what animal made which sound.

By grouping the data this way, the AI helps scientists discover structure in the sound recordings. This allows researchers to focus their time on analyzing each group and potentially identifying unknown species or behaviors.

1. What kind of learning is the AI using in this scenario, and why?

Unsupervised learning, because the AI is working with unlabeled sound data and grouping it based on similarity.

2. What features of the sounds is the AI analyzing to group them?

The AI analyzes features like pitch, rhythm, and frequency of the sounds.

3. Why doesn't the AI label the sounds during the learning process?

Because it was not given labels — it doesn't know which species made which sound.

4. What does the AI produce at the end — labels or something else?

The AI produces clusters or groups of similar sounds, not labeled categories.

5. How does this help scientists who are studying animals in the wild?

It helps scientists by organizing large amounts of unlabeled data so they can focus on patterns, identify unknown species, or find new behaviors.



Activity 1.5

In this activity, you'll explore how to build, test, and use a machine learning model using an online platform. You'll start by uploading sample images provided to you and then have the option to add your own examples.

Choose a Platform

Start by selecting a simple machine learning platform. Two recommended options are:

- Teachable Machine by Google
- Machine Learning for Kids

Both platforms allow you to upload data, train a model, and test it — but Machine Learning for Kids also supports model refinement and feedback.

Getting Started

- Choose a platform.
- Create a new image project.
- Upload the provided images to build at least two classes (e.g., “cat” and “dog”).
- Click Train Model.
- Test your model using new images.
- (Optional) If the model makes mistakes, add more examples and retrain.

Sample Image Sets

To help you get started, choose one of the following sets. Each contains enough examples to form two distinct classes:

- Cats and Dogs: bit.ly/cats_dogs_ML
- Flowers and Trees: bit.ly/flowers_trees_ML

You may also create your own dataset by taking or drawing pictures and uploading them to the platform.

LESSON 2: PREDICTION

In the previous lesson, we explored how an AI system learns using machine learning. It analyzes data, identifies patterns, and builds a model that helps it understand and make sense of new inputs. For example, after studying many examples, the AI forms an internal model of what a 'cup' looks like. This model becomes the basis for making predictions when it encounters a new object.

In this lesson, we will explore the basics of prediction and how Big Data helps AI make more accurate and reliable predictions.

PREDICTION

The answer to something is not always obvious. In many situations, the outcome isn't clear, and we have to make a guess. When we do, we rely on our experience and knowledge to make the best guess we can — even if we're not completely sure.

Prediction is the process by which an intelligent system — including humans — tries to estimate the most likely answer or outcome in a new situation.

PREDICTION IS WHEN AN AI USES ITS MODEL TO ESTIMATE THE MOST PROBABLE OUTPUT.

EXAMPLE 2.1

When Maya first learned to associate the object <cup> with the word 'cup', her mental model was very simple — it was based on just one cup. At first, she guessed correctly when she saw similar-looking cups. But when she saw cups that looked different — maybe taller, wider, or with a lid — she sometimes guessed wrong.

Each time she made a wrong prediction, her parents corrected her. Maya learned from her mistake and updated her model of what a cup could look like.

Over time, with more examples and feedback, her predictions improved. She became better at correctly classifying new objects as cup or not a cup — even if they looked different from the first one she saw.



Maya is learning to recognize cups. With each new example, her ability to recognize cups improves — increasing the probability that she will predict correctly, even when they look different.

In practice, prediction in AI takes different forms depending on the type of data it works with. In general, prediction applies to two kinds of data: categorical and continuous. In both cases, the AI draws on patterns it has learned from previous data to make its best guess — a prediction that reflects the most probable outcome given the information it has.

Whether it is predicting a category or estimating a numerical value, the model is performing the same essential task: using what it has learned to make a well-informed prediction in a new situation.

CATEGORICAL PREDICTION

Sometimes, the goal is to decide which category something belongs to. For example, the AI might try to recognize whether an object is a cat, determine the language of a spoken word, or match an animal sound to its species. These are all examples of categorical prediction, where the AI chooses the most likely label from a set of possible categories — much like answering the question, “What is this?”

EXAMPLE 2.2

At a recycling facility, a robotic system separates plastic bottles from other waste. A camera captures images of objects on a moving conveyor belt and sends them to a prediction model trained to recognize plastic bottles. The model analyzes each image and estimates the probability that the object is a plastic bottle. If the probability is high enough, the system triggers a robotic arm to remove the object and place it in the plastic recycling bin.



An AI-controlled robotic arm identifies and removes a plastic bottle from a moving conveyor belt at a recycling facility.

When the system began operating, it was tested with 100 items — 50 plastic bottles and 50 non-bottle items. During the test, the AI model made 30 mistakes.

- It failed to identify 20 plastic bottles, meaning it correctly classified only 30 out of 50 bottles.
- It mistakenly classified 10 non-bottle items as plastic bottles, meaning it correctly identified 40 non-bottle items as not bottles.

In total, the system got 70 out of 100 predictions correct, resulting in an accuracy of 70%.

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}} = \frac{30 + 40}{100} = \frac{70}{100} = 70\%$$

The items that were misclassified were collected and reviewed to identify the reasons for these errors, helping the team understand how the model could be improved.

- **Bottles Missed:** The model failed to recognize green bottles and crumpled bottles, which looked different from the training examples.
- **Objects Misclassified:** The model incorrectly classified these objects as plastic bottles due to similar shapes or features.



Original Data

Bottles it was trained on.



Bottles Missed

The model missed green bottles and crumpled bottles.



Objects Misclassified

The model misclassified objects that had similar shapes to bottles it was trained on.

ACCURACY IS ONE MEASURE OF A MODEL'S PERFORMANCE.

To improve the model's predictive accuracy and reduce its errors, the AI was retrained using a larger and more diverse set of images. This new training data included:

- Bottles in various colors (green, blue, and clear)
- Bottles that were crushed, dented, or partially obscured
- Images taken from different angles and lighting conditions
- More examples of non-bottle items with similar shapes or materials

By increasing the volume and variety of training data, the model learned to recognize a wider range of bottles and distinguish them more reliably from other objects.

The updated model was tested again using a new set of 100 items, with the same distribution as before: 50 plastic bottles and 50 non-bottles. This time, the model made only 10 mistakes:

- It missed 5 actual plastic bottles,
- And it misclassified 5 non-bottles as bottles.

That means it correctly identified 45 plastic bottles and 45 non-bottle items, giving a new accuracy of:

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}} = \frac{45 + 45}{100} = \frac{90}{100} = 90\%$$

GREATER VOLUME AND VARIETY OF DATA IMPROVE A MODEL'S PREDICTIVE ACCURACY.

CONTINUOUS PREDICTION

In the previous section, we explored categorical prediction, which applies to categorical data and involves assigning inputs to predefined categories. But in many cases, the task is not to classify, but to estimate a number. The AI might be asked to predict tomorrow's temperature, estimate how many points a student will score on a test, or forecast how much it will rain.

These are all examples of continuous prediction, where the goal is not to choose a category but to estimate a numerical value — answering questions like “How much?” or “What value?”

EXAMPLE 2.3

In a fictional neighborhood, an AI system is being developed to help estimate house prices based on their size. The only information the AI receives is the floor area of each house (in square meters) and the price it sold for. The goal is to train the AI to recognize the pattern between size and price so that it can predict the price of a new house based only on its size.

To begin, the AI is trained on a small dataset containing the sizes and sale prices of just six houses. Using this limited information, the AI attempts to learn the relationship between a house's size and its price by identifying a pattern in the data.



Size (square meters)	Price (\$1,000's)
107	291
115	292
107	302
115	305
93	227
82	173
100	274
120	340
130	316
90	234

This figure shows the data used to teach the AI model how house size relates to price. The scatter plot displays the actual data points, while the red dashed line represents the best-fit line created by the model. The table lists the size and sale price of the ten houses used to train the model.

The model derives the equation for the best-fit line and uses it to predict prices based on house size. This equation allows the AI to estimate the selling price of any house in the neighborhood, based solely on its size. In this case, the equation is:

$$\text{Price} = 2.53 \times \text{Size} + 18.45$$

Now the model must be tested. It is given the sizes of five houses it has not encountered before. Using its best-fit line equation, the model predicts the price of each house based solely on its size.

To determine how well the prediction model performs, the predicted prices are compared to the actual sales prices. The difference between each predicted and actual value is called the error.

$$\text{Error} = \text{Actual Price} - \text{Predicted Price}$$

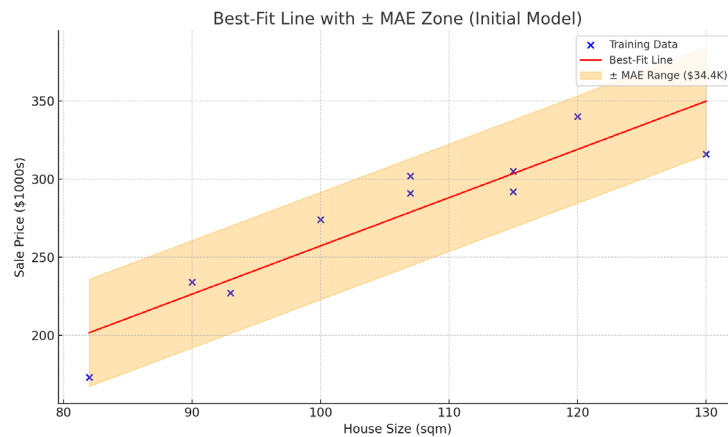
Size (sqm)	Actual Price (\$1,000)	Predicted Price (\$1,000)	Error (\$1,000)
111	291	124	-52
124	331	316	-15
157	433	406	-27
114	300	264	-36
146	399	357	-42

This table shows the actual and predicted prices for five houses that were not part of the training data. The error is calculated by subtracting the predicted price from the actual price. All values are negative, indicating that the model consistently underestimated the true prices.

A positive error means the model underestimated the price, while a negative error means it overestimated. To measure overall performance, we calculate the Mean Absolute Error (MAE) — the average size of the errors, using their absolute values to ignore direction. This tells us how far off the predictions are, on average.

$$\text{MAE} = \frac{\text{Sum of absolute value of errors}}{\text{Total Predictions}} = \frac{52 + 15 + 27 + 36 + 42}{5} = 34.4$$

The best-fit line shows what the model predicts, but the real-world value won't always land exactly on that line. The MAE gives us a typical margin of error, so we expect actual prices to fall within a band around the line — about plus or minus the MAE.



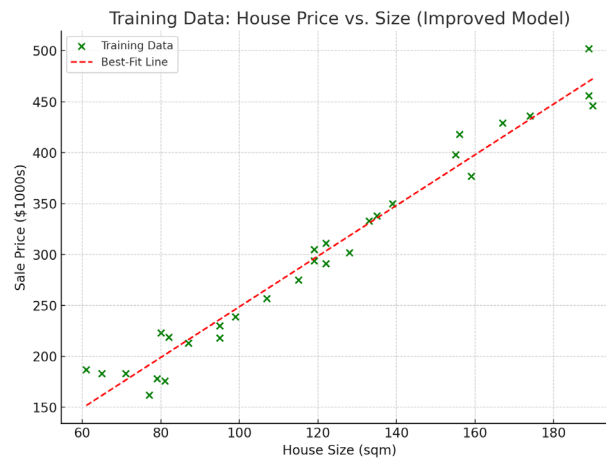
This graph shows the best-fit line produced by the model. The orange shaded area represents the full Mean Absolute Error (MAE) of \$34,400, illustrating the average range within which actual prices may fall around each predicted value.

THE MEAN ABSOLUTE ERROR IS ONE MEASURE OF A MODEL'S PERFORMANCE.

To improve the predictive performance of the model, it is trained on a larger volume and more varied set of data. Not only was it trained on a greater number of houses, but also on a wider range of sizes and prices. In the initial dataset, house sizes ranged from 82 to 130 square meters. Now, the model is exposed to houses both smaller and larger, with sizes ranging from 61 to 190 square meters.

The model recalculates the best-fit line equation using the expanded dataset and applies it to generate new predictions.

$$\text{Price} = 2.49 \times \text{Size} + 0.14$$



This graph shows the expanded training dataset, consisting of 30 houses with sizes ranging from 61 to 190 square meters. Each green point represents one house, and the red dashed line is the best-fit line generated by the model to represent the relationship between house size and price.

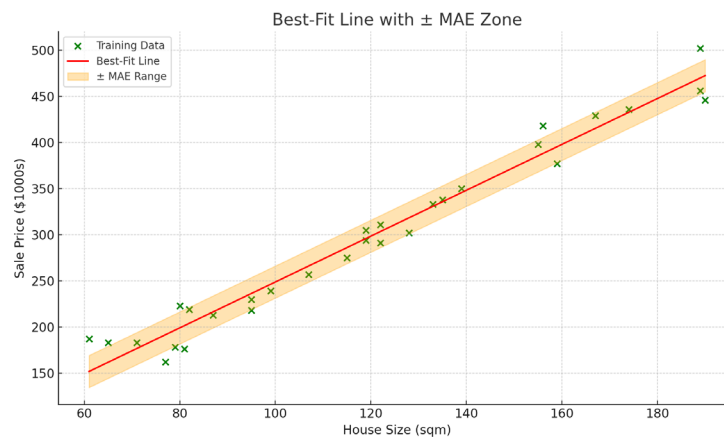
Using the new best-fit line, the model is tested again on the same five house sizes as before. For each, it predicts a price, which is then compared to the actual sale price. This allows us to calculate a new MAE, and evaluate whether the model's performance has improved.

Size (sqm)	Actual Price (\$1,000)	Predicted Price (\$1,000)	Error (\$1,000)
111	239	276	-37
124	316	308	8
157	406	390	16
114	264	284	-20
146	357	363	-6

This table shows the model's predictions after retraining on a larger dataset. For each house, the predicted price is compared to the actual sale price, and the error is calculated. Positive errors indicate underestimation; negative errors indicate overestimation.

$$MAE = \frac{\text{Sum of absolute value of errors}}{\text{Total Predictions}} = \frac{37 + 8 + 16 + 20 + 6}{5} = 17.4$$

As the volume and variety of training data increase, the MAE decreases. This means the zone of error around the best-fit line becomes narrower, making the model's predictions closer to the actual values.



This graph shows the best-fit line generated by the improved model, trained on a larger and more varied dataset. The orange shaded area represents the full Mean Absolute Error (MAE) of \$17,400 — a much narrower range than in the previous model, which had an MAE of \$34,400. The reduced error band indicates that the model's predictions are now, on average, closer to the actual sale prices.

Notes

Common Misconceptions

- An error means a bad prediction: All models make errors. The goal is to minimize them, not eliminate them — errors help the model improve.
- More data means a perfect model: More data helps, but only when it includes useful variety. Repetition or poor-quality data can make things worse.
- Models only learn from correct predictions: Models learn from all outcomes — including incorrect predictions — by using errors to adjust and improve over time.

CONCLUSION

In this lesson, we explored how AI makes predictions using two types of data: categorical and continuous. For categorical data, we measure how well the model performs using accuracy — the percentage of correct predictions. For continuous data, we use Mean Absolute Error (MAE) to calculate how far off the predictions are, on average.

We also saw that an AI model's ability to make accurate predictions depends on the data it learns from. As we increased the volume of training data and introduced greater variety, the model's predictions became more reliable. These two ideas — volume and variety — are part of what we call Big Data, which plays a key role in improving AI performance.

SUMMARY

Prediction is the process of estimating an outcome based on patterns learned from data.

AI can make predictions using:

- **Categorical** data (e.g., identifying whether an object is a bottle)
- **Continuous** data (e.g., estimating house prices or rainfall)

Accuracy is used to evaluate predictions on categorical data by measuring how many were correct out of all attempts.

Mean Absolute Error (MAE) is used for continuous data to measure how far off the predictions are, on average.

All models make errors — the goal is to reduce them, not eliminate them.

AI models improve as they are trained on more data. Specifically:

- **Volume** (more data) helps the model learn broader patterns.
- **Variety** (more types of examples) helps the model generalize better.

These concepts are part of what is called **Big Data**, which supports better learning and more accurate predictions in AI systems.

Activity 2.1

Read the following statements, and write on the line provided for each statement 'T' if it is true and 'F' if it is false.

T/F	STATEMENT
<u>T</u>	AI models can make predictions using either categorical or continuous data.
<u>F</u>	Accuracy is the best way to measure how close a numerical prediction is to the actual value.
<u>T</u>	Mean Absolute Error (MAE) tells us the average distance between predicted and actual values.
<u>F</u>	A model that makes prediction errors is considered broken.
<u>F</u>	More data always makes a model perfect.
<u>T</u>	Volume refers to the total amount of training data available to the model.
<u>F</u>	Variety means training a model on examples that are all the same type.
<u>T</u>	A prediction model for continuous data will produce a number, not a label.
<u>F</u>	If a model has high accuracy, that means its MAE is also low.
<u>T</u>	Big Data helps improve AI performance by increasing both volume and variety of training data.

 Activity 2.2

Answer the questions below.

1. What is the goal of prediction in AI?
 - a) To memorize all data exactly
 - b) To estimate the most likely outcome using learned patterns
 - c) To randomly guess based on probability
 - d) To classify only images
2. Which of the following is an example of a categorical prediction?
 - a) Estimating the temperature tomorrow
 - b) Predicting a student's exam score
 - c) Determining if an object is a plastic bottle
 - d) Calculating total sales for a store
3. Which metric can be used to evaluate continuous predictions like temperature?
 - a) Accuracy
 - b) Precision
 - c) Recall
 - d) Mean Absolute Error (MAE)
4. What does a high MAE value tell us about a model's predictions?
 - a) The model never makes mistakes
 - b) The model's predictions are far from actual values
 - c) The model's predictions are always correct
 - d) The model is overfitting
5. How is MAE calculated?
 - a) Percentage of correct predictions
 - b) Average of the absolute differences between predicted and actual values
 - c) Number of categories predicted correctly
 - d) The square root of total errors

Activity 2.2 (continued)

Answer the questions below.

6. What does the term "volume" refer to in the context of training data?
 - a) **The total amount of data the model learns from**
 - b) The range of model outputs
 - c) The number of times a model makes errors
 - d) The speed at which the data is processed

7. Why is "variety" in training data important for improving a model?
 - a) It increases the speed of training
 - b) It reduces the cost of data collection
 - c) It prevents the use of continuous prediction
 - d) **It helps the model generalize to different examples**

8. Which best describes the difference between accuracy and MAE?
 - a) MAE is used for text predictions; accuracy is used for numbers
 - b) Both are used for categorical predictions
 - c) **Accuracy is used for categorical data; MAE is used for continuous data**
 - d) Accuracy and MAE mean the same thing

9. What happens when a model is trained on a small, narrow dataset?
 - a) It performs well on all tasks
 - b) It automatically adjusts its error
 - c) It always predicts perfectly
 - d) **It struggles to make accurate predictions on new data**

10. Which of the following statements is true?
 - a) Models only learn from correct predictions
 - b) **Models learn from both correct and incorrect predictions**
 - c) Models stop learning after one training cycle
 - d) Models only learn from testing data

Activity 2.3

Read the narrative and answer the questions below.

A software company is developing an AI system that can automatically detect the language of short text messages. To train the model, engineers collect thousands of short sentences labeled with the language they are written in — such as English, Spanish, Arabic, and French.

The model learns to recognize specific features like vocabulary, accents, and sentence structure. Once trained, it is tested on new sentences submitted by users. It correctly identifies most languages, but it struggles with shorter messages or those that use informal spelling.

After reviewing the errors, the engineers realize that the AI hadn't been trained on enough casual or abbreviated text. They improve the model by adding more examples that include informal language, slang, and abbreviations. The system becomes more accurate and consistent in real-world use.

1. What type of learning was used to train the AI system to recognize languages, and how do you know?
Supervised learning — because the model was trained using labeled examples, where each sentence was tagged with its language.
2. What kind of prediction is this AI system performing?
Categorical prediction — the model is classifying each sentence into one of several languages.
3. Why did the AI model struggle with some user-submitted sentences?
Because the model hadn't been trained on enough informal or abbreviated text, so it couldn't recognize the language accurately in those cases.
4. How did the engineers improve the model?
They retrained it using more varied examples, including informal language and slang.
5. Why is it important to train the model with a wide variety of examples?
So the model can generalize and make accurate predictions, even when input appears in unfamiliar or casual forms.

Activity 2.4

Read the narrative and answer the questions below.

A transportation company is building an AI system that can estimate how long a trip will take by car. To train the model, engineers collect thousands of examples showing how long it took to travel different distances at different times of day. Each example includes the distance traveled and the actual travel time in minutes.

The AI is first trained on a small set of data — mostly from trips taken during off-peak hours. From this, it learns a basic pattern: longer distances usually mean longer travel times. But when tested on trips during rush hour, the model often predicts travel times that are too short, because it hasn't learned how traffic affects the results.

To fix this, the engineers expand the training data to include trips taken at different times of day, including busy periods with heavy traffic. The model adjusts its understanding and begins to make more accurate predictions.

1. What kind of prediction is the AI model making in this scenario?

Continuous prediction — it's estimating a numerical value (travel time in minutes).

2. Why were the model's early predictions inaccurate during rush hour?

Because the model had not been trained on examples that included traffic delays.

3. How did the engineers improve the model's performance?

They added more varied training data from different times of day.

4. What is one way to measure how far off the model's predictions are?

By calculating the Mean Absolute Error (MAE), which gives the average difference between predicted and actual travel times.

5. Why is it important to include both volume and variety in training data?

So the model can make accurate predictions across different conditions, such as heavy or light traffic.

Activity 2.5

Read the scenario below. Then answer the questions that follow. Use the provided spreadsheet. You will practice sorting, filtering, and using formulas—just like real data scientists do when they test AI models. Use the short link to access the spreadsheet: bit.ly/C4_AI_Activity2_5

- a. A grocery distribution center is testing an AI model that sorts fruit. The model decides if each item is an Apple or Not Apple. The center tests the model on a batch of 50 fruits. For each item, the AI makes a prediction. The spreadsheet shows both the actual label and the AI prediction.

1. How many predictions were correct?	36
2. How many predictions were incorrect?	14
3. What was the model's overall accuracy?	72%
4. How many apples were missed?	7
5. How many tomatoes were misclassified?	11

6. Which fruit did the AI confuse with apples? Why do you think this happened?

Tomatoes — confused with apples because they are round, similar in size, and often red like apples

- b. The grocery distribution center is considering using the AI model for full operations. On average, the center handles 100,000 apples every week, along with other fruits. When the AI makes a mistake, it costs the business money:

- Missing an apple = \$5 lost
- Mistaking a tomato for an apple = \$1 wasted

1. How many apples would be missed every week?	14,000
2. How many tomatoes would be misclassified every week?	22,000
3. What would the total weekly cost of missed apples be?	\$70,000
4. What would the total weekly cost of misclassified tomatoes be?	\$22,000
5. What is the combined weekly cost of using this AI model?	\$92,000

6. Based on these numbers, do you think the AI is ready for real-world use? Why?

The AI is not ready for operations — the cost of errors is extremely high. The model would need more training or a different approach before being deployed.

Activity 2.5 (continued)

Read the scenario below. Then answer the questions that follow. Use the provided spreadsheet. You will practice sorting, filtering, and using formulas—just like real data scientists do when they test AI models. Use the short link to access the spreadsheet: bit.ly/C4_AI_Activity2_5

- c. After reviewing the high cost of errors, the distribution center retrained its AI model to better distinguish between apples and tomatoes.

Refer to the sheet 'Activity 2.5c' to answer the following questions.

1. What is the accuracy of the new model?	90%
2. How many apples would it miss out of 100,000?	4,000
3. How many tomatoes would it misclassify out of 100,000?	6,000
4. What would the weekly cost of missed apples be?	\$20,000
5. What would the total weekly cost of misclassified tomatoes be?	\$6,000
6. What is the total weekly cost of errors with the new model?	\$26,000

7. How does this compare to the old model in Activity 2.5b? What is the improvement?

Old model cost = \$92,000 per week. New model cost = \$26,000 per week.

Savings = \$66,000 per week.

8. Think about another area of life. Where would AI errors be more serious than in a grocery store?

AI errors would be more serious in areas where lives are at stake. Examples:

Healthcare: A wrong diagnosis could harm a patient.

Traffic/Transportation: A self-driving car could cause an accident.

Airlines: An AI error in navigation or maintenance could risk hundreds of lives.

9. Imagine you are in charge of the distribution center. Would you use the 90% accurate AI, or keep humans sorting apples, or a combination of both AI and human? Explain your choice.

I would choose a combination of AI and human workers. The AI is fast and saves money, but at 90% accuracy it still makes mistakes. By adding human quality control, people can catch the errors the AI misses. In the real world, this is called 'Human-in-the-Loop AI'. AI and humans working together are more effective than either on their own.

Activity 2.6

Read the scenario below, then answer the questions that follow. Use the provided spreadsheet to apply formulas and calculations—just like real data scientists do when they test AI models. Use the short link to access the spreadsheet: bit.ly/C4_AI_Activity2_6

A company is testing an AI model to predict how many units of a product it will sell at different prices. Accurate predictions help the company plan inventory and revenue.

The AI studied data and built the following best-fit line:

$$\text{Sales (units)} = -12 \times \text{Price (\$)} + 220$$

The company then tested the model at different prices between \$8.00 and \$12.50.

- Complete the table by using the equation of the best-fit line to predict the number of units sold and the error for each prediction.

Price (\$)	Actual Units Sold (units)	Predicted Units Sold (units)	Error (units)	Error (Absolute value)
8.0	115	124	-9	9
9.0	125	112	13	13
10.0	95	100	-5	5
11.0	101	88	13	13
12.0	67	76	-9	9

- Calculate the Mean Absolute Error.

The MAE = 9.80

- If the AI makes one prediction every day, then over a 30-day month, by how many units will the company misestimate sales?

Total Error (units) = MAE x Days = 9.80 x 30 = 294 units

- If the company sells 10,000 units in a month, what percentage of sales does the total error (294 units) represent? What does this tell us about the model's usefulness?

294/10,000 x 100 = 2.94%

The model's error is about 3% of total sales. It is fairly accurate, but lowering the error could still save hundreds of units each month.

LESSON 3: LOGIC & REASONING

So far, we've seen that AI can learn from data and predict outcomes. But there is more to intelligence — reasoning. We use reasoning all the time. Much of our everyday thinking is reasoning. It helps us understand what we observe, solve problems, and decide what to do.

Reasoning, though we may not always notice it, is structured. There are rules to thinking that guide us toward conclusions. These conclusions, or outcomes of reasoning, may be an understanding, a new insight, a decision, or a plan. The rules that make reasoning work are referred to as logic.

In this lesson, we will explore reasoning, its forms, and the framework in which it operates: logic.

REASONING

Reasoning is a structured way of thinking — or processing — that helps us uncover new information, make decisions, and make sense of our world. In every case, reasoning follows a simple path: input, processing, and output. It begins with what is already known — information or knowledge that serves as the input. This is then processed, and the outcome is produced. In general, three outcomes are possible: an inference, a decision, or an explanation.

REASONING IS THE STRUCTURED WAY WE THINK TO DISCOVER,
DECIDE, OR UNDERSTAND.



INFERENCE

Inference means arriving at a new piece of information. It is the process of linking information we already have in order to reach a conclusion. The information we start with forms the basis or starting point of the reasoning process: these are called premises.

Premises may come from three sources:

1. facts we already know, which form general rules.
2. observations our senses are picking up.
3. predictions based on learned patterns.

A PREMISE IS THE STARTING POINT OF INFERENCE AND CAN BE A FACT, OBSERVATION, OR PREDICTION.

EXAMPLE 3.1 (HI - HUMAN INTELLIGENCE)

Layla is in science class, listening carefully as her teacher explains that all birds have feathers. A moment later, the teacher holds up a picture of a chicken and asks the class, “Is this a bird?” Layla thinks it through. She recalls what she has just learned: all birds have feathers. She observes that chickens have feathers. From this, she reasons that chickens must be birds.

LOGICAL FORM

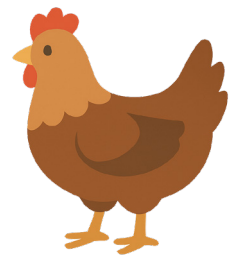
PREMISES

GENERAL RULE: ALL BIRDS HAVE FEATHERS.

OBSERVATION: CHICKENS HAVE FEATHERS.

INFERENCE

THEREFORE : CHICKENS ARE BIRDS.



INFERENCE IS THE PROCESS OF ARRIVING AT NEW INFORMATION.

EXAMPLE 3.1 (HI - ARTIFICIAL INTELLIGENCE)

A video app observes that a student has recently watched several basketball highlights. The AI model behind the app has learned a general rule from past data: *viewers who watch videos about a topic will probably enjoy other videos on that same topic.*

Using this knowledge, the system reasons that the student is likely interested in basketball more broadly. From this, it infers that the student would also enjoy basketball training videos and recommends them.



LOGICAL FORM

PREMISES

GENERAL RULE : VIEWERS WHO WATCH VIDEOS ABOUT A TOPIC WILL PROBABLY ENJOY OTHER VIDEOS ON THAT SAME TOPIC.

OBSERVATION : THE STUDENT WATCHED BASKETBALL HIGHLIGHTS.

INFERENCE

THEREFORE : PROBABLY, THE STUDENT WOULD ENJOY BASKETBALL TRAINING VIDEOS.

Both examples show how inference arrives at a new conclusion by linking what we already know. In the first case, the conclusion “*Chickens are birds*” followed from a general rule (*All birds have feathers*) combined with a specific observation (*Chickens have feathers*). Because this reasoning was grounded in a fact, the inference carried a high degree of certainty.

In the second case, the conclusion “*The student would probably enjoy basketball training videos*” followed from a general rule learned from data (*Viewers who watch videos on a topic will probably enjoy more videos on that topic*) and an observation (*The student watched basketball highlights*). Here the inference relied on a prediction, and since predictions come with uncertainty, the conclusion is expressed only as probable.

THE CERTAINTY OF AN INFERENCE DEPENDS ON THE CERTAINTY OF ITS PREMISES.

The certainty of an inference depends on the certainty of its premises. If the premises are facts, the conclusion is highly certain. If the premises are predictions, the conclusion is less certain and best understood as probably true rather than definitely true.

DECISION-MAKING

Decision-making means selecting the best choice from several possibilities. It is the process of looking at different choices, considering what will probably happen if we take each one, and then selecting the option with the highest utility — the one that is most useful, valuable, or best fits our goal.

DECISION-MAKING IS THE PROCESS OF SELECTING THE CHOICE WITH THE HIGHEST UTILITY.

Decisions are also guided by premises: facts, rules, observations, predictions, and even personal preferences or opinions. Which choice seems best depends on the context — sometimes facts dominate, other times preferences or predictions play the larger role.

EXAMPLE 3.2 (HI - HUMAN INTELLIGENCE)

Lina was getting ready for school and saw that the sky was cloudy. She thought about walking, riding her bike, or asking her father for a ride. She remembered that she didn't mind a little rain and that she really disliked depending on her father's schedule since he sometimes left very early. After weighing her options, she decided to take her bike.



LOGICAL FORM

PREMISES

PREFERENCE: I DON'T MIND A LITTLE RAIN.

PREDICTION: IT IS LIKELY THAT MY FATHER WILL LEAVE VERY EARLY.

CHOICES

CHOICE 1: IF I WALK, THEN PROBABLY I WILL BE SLOW AND MIGHT GET WET.

CHOICE 2: IF I BIKE, THEN PROBABLY I WILL ARRIVE FASTER AND RISK ONLY A LITTLE RAIN.

CHOICE 3: IF I RIDE WITH MY FATHER, THEN PROBABLY I WILL STAY DRY BUT ARRIVE TOO EARLY AT SCHOOL.

DECISION

THEREFORE: I WILL RIDE MY BIKE TO SCHOOL.

In Lina's case, her premises included a personal preference — *she didn't mind a little rain* — and a prediction — *her father will likely leave very early*. These premises shaped how she evaluated her choices.

She compared three choices: *walking*, *biking*, or *riding with her father*. Each choice came with possible outcomes, and she weighed the utility (how useful or valuable each outcome was to her).

1. Walking would take too long and might make her wet (low utility).

2. Riding with her father would keep her dry but could get her to school too early (low utility).
3. Biking was fast, comfortable enough, and matched her preference about rain (highest utility).

Her decision to ride her bike was based on comparing the outcomes against her premises and selecting the choice with the best overall value.

EXAMPLE 3.2 (AI - ARTIFICIAL INTELLIGENCE)

A self-driving car approaches an intersection. Its AI system considers three options: keep driving, slow down, or stop. Its sensors show that a pedestrian is stepping onto the crosswalk and a car is close behind. The car's decision-making is guided by two fundamental rules: never harm humans, and avoid collisions unless it contradicts the first rule. Based on this, the car decides to stop for safety.

LOGICAL FORM

PREMISES

RULE 1: NEVER HARM HUMANS.

RULE 2: AVOID COLLISIONS UNLESS IT CONTRADICTS RULE 1.

CHOICES

CHOICE 1: IF THE CAR KEEPS DRIVING, THEN THE PEDESTRIAN WILL BE HARMED.

CHOICE 2: IF THE CAR SLOWS DOWN, THEN PROBABLY IT WILL AVOID HARM BUT MAY RISK A REAR COLLISION.

CHOICE 3: IF THE CAR STOPS, THEN IT WILL NOT HARM THE PEDESTRIAN, BUT A REAR COLLISION WILL OCCUR.

DECISION

THEREFORE: STOP THE CAR.



For the self-driving car, its premises were programmed as rules: *never harm humans* and *avoid collisions unless that contradicts the first rule*. With these rules in place, the car evaluated three choices: *keep driving*, *slow down*, or *stop*. Each choice had different outcomes.

1. Continuing to drive would harm a pedestrian (very low utility).
2. Slowing down might possibly avoid harm but still risk a probable rear collision (medium utility).
3. Stopping would certainly keep the pedestrian safe even though it might cause a rear collision (highest utility, because protecting humans is the top rule).

The AI system was deciding under a “Definitely Do Not” condition set by its first rule. This condition gave that rule the highest utility or value, so it overruled the other possibilities. This shows how the car compared the possible outcomes against its guiding premises, then selected the option with the greatest utility — the one most consistent with its goals and priorities.

UTILITY IS A MEASURE OF HOW USEFUL, VALUABLE, OR GOAL-FITTING AN OUTCOME IS.

Although Lina and the self-driving car are very different, they both used the same reasoning pattern. Each began with premises — Lina’s were based on her preferences and a prediction, while the car’s were based on safety rules.

They considered several choices, weighed the possible outcomes, and compared the utility of each option. Lina’s decision was shaped by comfort, while the car’s was shaped by safety, but the structure of their reasoning was the same.

THE UTILITY OF A CHOICE DEPENDS ON THE PREMISES USED TO JUDGE IT.

EXPLANATION

Explanation is the process of reasoning backward from a given — something already accepted as true — in order to uncover its causes, contributing conditions, or relationships. In other words, explanation begins with an outcome and asks: “What conditions could have produced this?” Instead of starting with premises and moving forward, as in inference, explanation works in reverse — from an observation to possible reasons. In terms of directionality, explanation is therefore the opposite of inference.

A GIVEN IS THE STARTING POINT OF EXPLANATION, FROM WHICH WE REASON BACKWARD TO UNCOVER ITS CAUSES, CONTRIBUTORS, AND RELATIONSHIPS.

In terms of strategy, however, explanation is closer to decision-making: both involve a search process. Decision-making searches among possible actions to select the one with the highest utility, while explanation searches among possible reasons to identify the one with the greatest plausibility.

THE PLAUSIBILITY OF AN EXPLANATION DEPENDS ON THE PREMISES THAT SUPPORT IT.

Like inference and decision-making, explanation depends on a knowledge base — the rules, facts, or learned patterns that connect conditions to outcomes. Without this background knowledge, reasoning backward would not be possible. But unlike inference, which can sometimes deliver definite conclusions, explanation typically carries varying degrees of uncertainty. The strength of an explanation depends on the plausibility of its premises and how well they fit the observed outcome.

EXAMPLE 3.3 (HI - HUMAN INTELLIGENCE)

Omar steps outside one morning and notices that the front lawn is wet. He wonders what caused it. Two possibilities come to mind: maybe it rained during the night, or maybe the sprinkler was on. Omar thinks about what usually happens: when it rains, everything in the neighborhood gets wet, not just his lawn. But today, he notices that only the front lawn is wet while the rest of the ground is dry. Based on this observation, Omar reasons that the most likely cause is that the sprinkler was on.

LOGICAL FORM

GIVEN

OBSERVATION: THE FRONT LAWN IS WET.

PREMISES

GENERAL RULE 1: IF IT RAINS THEN EVERYTHING IS WET.

GENERAL RULE 2: IF THE SPRINKLER IS ON THEN ONLY THE FRONT LAWN GETS WET.

OPTIONS

OPTION 1: THE CAUSE MIGHT BE RAIN.

OPTION 2: THE CAUSE MIGHT BE THE SPRINKLER.

OBSERVATION: ONLY THE FRONT LAWN IS WET.

EXPLANATION

THEREFORE: SINCE ONLY THE FRONT LAWN IS WET, THE MOST PROBABLE EXPLANATION IS THE SPRINKLER WAS ON.

This example shows how explanation works by reasoning backward from an outcome. Omar begins with what is given — the lawn is wet — and considers different possible reasons. By comparing these options against additional evidence, he chooses the cause that best fits.

EXAMPLE 3.3 (AI - ARTIFICIAL INTELLIGENCE)

A medical AI system is fed patient information: the person has both a fever and a cough. The system compares this data to patterns in its knowledge base. It “knows” that the flu typically produces both fever and cough, while allergies can produce cough but not fever. Since the patient’s data includes fever along with cough, the AI rules out allergies as the cause and identifies the flu as the most probable explanation.



LOGICAL FORM

GIVEN

OBSERVATION: THE PATIENT HAS A FEVER AND COUGH.

PREMISES

GENERAL RULE 1: IF A PATIENT HAS THE FLU, THEN FEVER AND COUGH OCCUR TOGETHER.

GENERAL RULE 2: IF A PATIENT HAS ALLERGIES, THEN COUGH OCCURS BUT FEVER DOES NOT.

OPTIONS

OPTION 1: THE CAUSE MIGHT BE FLU.

OPTION 2: THE CAUSE MIGHT BE ALLERGIES.

OBSERVATION: THE PATIENT HAS BOTH FEVER AND COUGH.

EXPLANATION

THEREFORE: SINCE THE PATIENT HAS BOTH FEVER AND COUGH, THE MOST PROBABLE EXPLANATION IS FLU.

The AI follows the same reasoning process. It starts with the observed symptoms — fever and cough — and evaluates possible reasons using its knowledge base. By ruling out what doesn’t fit, it selects the most probable explanation.

Because different conditions can lead to the same outcome, explanation is not always certain. It is often probabilistic: the best explanation is the one that fits the evidence most strongly, though other explanations may remain possible.

EXAMPLE 3.4 (HI - HUMAN INTELLIGENCE)

Omar sees his front lawn is wet. He considers two possible causes: rain or the sprinkler. Since only the front lawn is wet, he concludes the sprinkler was most likely the cause.

Later, Omar discovers that his children had been playing with water. This new possibility was not part of his original reasoning. With this added information, the sprinkler explanation loses plausibility, and the children's play becomes the best explanation.



EXPLANATIONS CAN CHANGE WHEN NEW POSSIBILITIES ARISE

EXAMPLE 3.4 (AI - ARTIFICIAL INTELLIGENCE)

A medical AI is fed information about a patient who has both fever and cough. From its knowledge base, it considers flu and allergies as possible causes. Since allergies don't usually produce fever, the AI concludes flu is the most probable explanation.

But when the system receives additional information — the patient also has a rash — the plausibility of flu decreases. The new symptom suggests another possible condition, measles, and the explanation shifts to account for this more fitting cause.



CONCLUSION

Inference, decision-making, and explanation all share common ground: they are forms of reasoning that begin with premises — facts, observations, or predictions — and apply a knowledge base to reach conclusions. Each can operate with certainty or with varying degrees of probability.

Where they differ is in direction and purpose. Inference moves directly forward from premises to outcomes, helping us discover new information. Decision-making searches among possible actions to select the one that best meets a goal. Explanation searches backward from an outcome to identify the most plausible cause.

Reasoning takes many forms — whether discovering new information, choosing the best action, or searching for the most plausible cause — it is how we make sense of the world and act within it.

Notes

Common Misconceptions

- *Reasoning outcomes are always certain if the system is well-designed.*
Even with perfect logic, uncertainty arises from incomplete knowledge, noisy data, or ambiguous premises.
- *Decision-making is just picking the highest probability outcome.*
Decision-making is about expected utility, not just probability — the best choice balances both likelihood and value of outcomes.
- *Explanations must be causal.*
In AI, many explanations are associative or model-based rather than strictly causal.
- *One explanation is sufficient.*
Explanation is a search problem: multiple explanations may exist, and plausibility depends on background knowledge and available evidence.

SUMMARY

Reasoning – The structured way we think to discover, decide, or understand.

- Forms of reasoning – inference, decision, and explanation.
- Logic – the rules that guide reasoning.
- Knowledge Base – All forms depend on a knowledge base — rules, facts, or learned patterns that connect conditions to outcomes.

Inference – Arriving at a new piece of information by linking what we already know.

- Starting Point – Premises; can be:
 - Facts – general rules already known.
 - Observations – information detected through our senses.
 - Predictions – expectations based on learned patterns.
- Outcome – New information (a conclusion).
- Strategy – Direct reasoning from premises to outcome (no search).
- Directionality – Forward reasoning.
- Certainty – The certainty of an inference depends on the certainty of its premises.
 - If premises are facts, then inference is definite.
 - If premises are predictions, then inference is probable.

Decision-making – The process of selecting the option with the highest utility (the most useful, valuable, or goal-fitting).

- Starting Point – Premises (facts, rules, observations, predictions, preferences, opinions).
- Outcome – The choice with the highest utility.
- Strategy – Search among possible choices.
- Directionality – Forward reasoning.
- Utility – A measure of how useful, valuable, or goal-fitting an outcome is. It depends on the premises used to judge it.

Explanation – The process of reasoning from a outcome to uncover its causes, or contributing conditions.

- Starting Point – A given (an accepted observation or outcome)..
- Outcome – The reason with the highest plausibility.
- Strategy – Search among possible reasons.
- Directionality – Backward reasoning.
- Utility – A measure of how reasonable or believable an explanation is. It depends on the premises used to support it.

Activity 3.1

Read the following statements, and write on the line provided for each statement 'T' if it is true and 'F' if it is false.

T/F	STATEMENT
<u>T</u>	Reasoning is a structured way of thinking that helps us discover new information, choose actions, or make sense of outcomes.
<u>F</u>	Premises are always absolute facts; observations and predictions cannot be used as premises.
<u>T</u>	The certainty of an inference depends on the certainty of its premises.
<u>F</u>	Inference involves a search among different possibilities before reaching a conclusion.
<u>T</u>	Decision-making is the process of selecting the option with the highest utility.
<u>T</u>	Utility means the usefulness, value, or goal-fit of an outcome.
<u>T</u>	Decision-making and explanation both involve a search strategy, while inference does not.
<u>T</u>	Explanation is reasoning backward from a given outcome to uncover possible conditions or reasons.
<u>T</u>	The plausibility of an explanation depends on how well its premises fit the observed outcome.
<u>F</u>	Explanations are always certain, because once the cause is found it cannot be replaced.

Activity 3.2

Answer the questions below.

1. Reasoning can best be described as:
 - a) Guessing based on experience
 - b) Remembering facts from memory
 - c) **A structured way of thinking that follows rules to reach conclusions**
 - d) Choosing randomly when uncertain

2. In the reasoning process, the starting points we use are called:
 - a) Outcomes
 - b) **Premises**
 - c) Hypotheses
 - d) Causes

3. In reasoning, logic refers to:
 - a) The memory of facts and rules
 - b) The outcome of decision-making
 - c) The search among possible causes
 - d) **The rules that guide reasoning from premises to conclusions**

4. Why is logic important in reasoning?
 - a) It ensures all decisions are fast
 - b) **It provides structure so conclusions follow consistently from premises**
 - c) It allows us to guess without evidence
 - d) It reduces the number of premises needed

5. Which of the following can be premises in inference?
 - a) Facts
 - b) Observations
 - c) Predictions
 - d) **All of the above**

 Activity 3.2 (continued)

Answer the questions below.

6. The certainty of an inference depends on:
 - a) **The certainty of its premises**
 - b) The number of options available
 - c) The directionality of reasoning
 - d) The search process used

7. Decision-making is defined as:
 - a) Selecting the option with the highest plausibility
 - b) Reasoning backward from an outcome to causes
 - c) Directly linking premises to conclusions
 - d) **Selecting the option with the highest utility**

8. In decision-making, utility refers to:
 - a) How probable the outcome is
 - b) **How useful, valuable, or goal-fitting an outcome is**
 - c) How fast the decision can be made
 - d) How certain the premises are

9. Explanation differs from inference mainly because:
 - a) It moves forward from premises to outcomes
 - b) It depends only on facts
 - c) It guarantees certainty
 - d) **It reasons backward from a given outcome to possible conditions**

10. The starting point of explanation is called:
 - a) Premise
 - b) **Given**
 - c) Utility
 - d) Hypothesis

 Activity 3.2 (continued)

Answer the questions below.

11. The plausibility of an explanation depends on:
 - a) The number of possible outcomes
 - b) **The premises that support it and how well they fit the observed outcome**
 - c) The search method used
 - d) Whether the explanation is causal

12. Which of the following requires a search strategy?
 - a) Inference only
 - b) **Decision-making and explanation**
 - c) Explanation only
 - d) Inference, decision-making, and explanation

13. Which of the following best shows inference?
 - a) Deciding which bus to take to school
 - b) Explaining why the lawn is wet
 - c) **Linking a rule and an observation to reach a new conclusion**
 - d) Remembering a fact from memory

14. In decision-making, the best option is chosen by:
 - a) Selecting the one supported by the most facts
 - b) **Selecting the one with the highest utility**
 - c) Selecting the one with the highest probability of occurring
 - d) Selecting the one that requires the fewest premises

15. Why is uncertainty common in explanation?
 - a) Because explanations always require predictions
 - b) **Because multiple possible reasons can fit the same outcome**
 - c) Because causes never align with outcomes
 - d) Because it does not use a knowledge base

Activity 3.3

Read the narrative and answer the questions below.

- a.** In science class, students were learning about photosynthesis. The teacher explained that all plants require sunlight to survive. She then instructed them to start an experiment by placing one plant (A) near the window and another plant (B) inside the closet.

1. Infer the outcome for Plant (A).

Plant (A) will survive.

2. Infer the outcome for Plant (B).

Plant (B) will not survive.

3. What is the general rule you used to make this inference?

All plants require sunlight to survive.

4. State your logic for inferring the outcome for Plant (B).

Since all plants require sunlight to survive, and Plant (B) will not get any sunlight, therefore Plant (B) will not survive.

- b.** An AI model was trained on thousands of cat images to recognize features of cats. From the training data, it learned the general rule that cats usually have pointed ears and whiskers. The model is then shown a new image, which it classifies as a cat.

1. What form of reasoning did the AI model use to make this prediction?

Inference

2. What general rule did it acquire from its training?

If an animal has pointed ears and whiskers, then it is probably a cat.

3. Would you consider this general rule one of certainty?

No, it is not 100% certain. Some breeds of cats do not have pointed ears, so the rule is probabilistic rather than definite.

Activity 3.4

Read the narrative and answer the questions below.

- a. Amir is deciding how to spend his evening. He has two options: study for his math test tomorrow or watch a movie and go to bed early. He knows that studying will help him do better on tomorrow's test, but he also wants to relax after a long day.

1. What form of reasoning is Amir using?

Decision-making

2. What premises is Amir considering to make a choice?

His goal to do well on the test and his preference for relaxation.

3. What outcome does decision-making seek to produce?

The choice with the highest utility.

4. If Amir chooses to study, what does this reveal about his values?

He values doing well on the test more than relaxation.

- b. A delivery drone approaches a tall building on its route. It must decide whether to fly over the building, or around it. Its rules are: (1) minimize delivery time, and (2) conserve battery when possible. The drone maps the routes and calculates that flying over the building will take less time but will require more energy, while flying around it will require more time but less energy.

1. What premises guide the drone's reasoning?

Rules about minimizing delivery time and conserving battery.

2. What other information does the drone need to make its choice?

Which rule is more important in this situation.

3. If the drone chooses to fly around the building, what does this tell you about its programmed priorities?

It assigns higher utility to conserving battery than to minimizing time.

Activity 3.5

Read the narrative and answer the questions below.

- a. Sara walks into the kitchen and sees that the cookie jar is empty. She considers two possibilities: her brother ate the cookies, or her mother gave them to the neighbors. Sara looks around for clues, she notices cookie crumbs on the table.

1. What form of reasoning is Sara using?

Explanation

2. What is the given in this scenario?

The observation that the cookie jar is empty.

3. What possible reasons does Sara consider?

Her brother ate the cookies, or her mother gave them away.

4. Which explanation is most plausible, and why?

The explanation that her brother ate them is most plausible, because the crumbs on the table support this outcome.

- b. A smart home system detects that the living room lights are on in the middle of the night. From past data, it considers two possibilities: someone is awake and moving around, or the timer system malfunctioned. It also detects motion in the living room.

1. What is the given in this scenario?

The observation that the living room lights are on in the middle of the night.

2. What possible reasons does the AI consider?

That a person is awake, or that the timer system malfunctioned.

3. Which explanation is most plausible, and why?

The explanation that a person is awake is most plausible, because the motion sensor confirms activity.

Activity 3.6 - Practicing Reasoning with AI

In this activity, you will interact with an AI which will act as a mentor, providing questions and explanations to deepen your understanding of the topics of the lesson.

In the shared folder you will find five separate text documents, each containing the prompt you need for the activity, as well as the PDF of the lesson summary.

Access the folder from: bit.ly/C4_AI_Activity3_6

a. Reasoning Scenarios

In this part, you will practice three forms of reasoning:

- Inference
- Decision-Making
- Explanation

Each reasoning type has its own text file with a prompt.

Steps to Follow:

1. Download the file from the shared Google folder and choose the prompt for the reasoning type you want to practice.
2. Copy the full prompt and paste it into ChatGPT (or another LLM).
3. The prompt will guide the AI to give you three scenarios. For each scenario, carefully read the setup.
4. Type your answer directly into the chat, always including:
 - Your answer: the outcome, action, or cause.
 - Your reason: why your answer follows from the given rule, priorities, or causes.
5. The AI will confirm if your answer is correct and evaluate whether your reasoning is sound.
6. Continue through the scenarios one at a time until you complete all three.

Tips:

Think about the structure of reasoning:

- Is it predicting? (Inference)
- Is it choosing? (Decision-making)
- Is it explaining? (Explanation)

Always explain your reasoning, even if unsure — that is what the AI will evaluate.

Don't rush; reflect on why a certain answer follows from the premises.

b. Questions & Answers

In this part, you will test your understanding of the lesson using the PDF provided in the folder. You can choose between:

- True/False Questions
- Multiple-Choice Questions

Each activity has its own text file with a prompt.

Steps to Follow:

1. Open the shared folder and download the PDF of the lesson summary.
2. Attach the PDF in ChatGPT.
3. Copy the prompt for the activity you want to practice (True/False or Multiple Choice) and paste it into ChatGPT.
4. The AI will create 10 questions based on the summary:
 - If you chose True/False, each question will be answered with “True” or “False.”.
 - If you chose Multiple Choice, each question will have 4 options (A, B, C, D).
5. For each question:
 - Read carefully and submit your answer.
 - The AI will tell you if you are correct, provide the correct answer if needed, and give a short explanation.
6. After all questions:
 - The AI will list the questions you got wrong.
 - It will give you a short review of the key topics you struggled with.
 - It will suggest a follow-up exercise to strengthen your understanding.

Tips:

- For True/False: think about why the statement is true or false before answering.
- For Multiple Choice: read all four options carefully before choosing.
- Use the AI’s feedback to focus on the concepts you find most difficult.
- Repeat the activity for more practice — the AI can generate new sets of questions each time.

LESSON 4: AI IN THE REAL-WORLD

Artificial Intelligence (AI) is no longer just a futuristic idea from science fiction. It is already part of our everyday lives, often in ways we do not even notice. From recommending what video to watch next to helping doctors detect diseases earlier, AI systems are shaping how people live, work, and learn.

In this lesson, we will explore how AI is used in the real world, what challenges and limitations it faces, and why ethics and responsibility are central to its development. By the end, you should be able to explain examples of AI applications, recognize where AI might fail, and discuss the impact AI has on individuals and society.

APPLICATIONS OF AI

AI technologies are designed to extend the very processes of intelligence we studied in earlier lessons. They can recognize patterns in data, form associations between pieces of information, and learn from past examples. With these abilities, AI systems can make predictions, classify new situations, and even carry out forms of reasoning such as inference, decision-making, and explanation. Because these are the same foundations of human intelligence, AI can be applied across many areas of daily life, industry, and science.

Most of today's systems fall under Narrow AI, which means they are designed with a narrow focus. They can perform a single task or a limited set of tasks very effectively. Examples include navigation apps that suggest the fastest route or chatbots that respond to customer service questions. Although they may appear intelligent, their abilities are restricted to their specific domain and cannot be transferred to entirely different tasks.

NARROW AI SYSTEMS ARE DESIGNED TO PERFORM ONE SPECIFIC TASK OR A LIMITED SET OF TASKS VERY EFFECTIVELY.

By contrast, researchers often speak of General AI (Artificial General Intelligence, AGI). General AI is still theoretical—it has not yet been created. In principle, a General AI would have the ability to learn and apply intelligence across many tasks, just as humans can. It could transfer knowledge from one situation to another, adapt to unfamiliar challenges, and reason flexibly in new contexts. In other words, it would be capable of applying its intelligence to virtually any domain.

GENERAL AI COULD LEARN AND APPLY INTELLIGENCE FLEXIBLY
ACROSS MANY DIFFERENT TASKS.

Narrow AI applications, on the other hand, are typically classified according to the kind of cognitive function they mimic. The five most widely recognized categories are:

1. Perception
2. Language
3. Reasoning and Decision-Making
4. Prediction and Forecasting
5. Control and Robotics

PERCEPTION

Perception is the stage where AI systems handle raw input from the world. Just as humans use their senses to see and hear, AI uses sensors, cameras, and microphones to detect signals. The goal is to transform pixels, sound waves, or other raw data into information the system can work with.

PERCEPTION IS THE ABILITY OF AI TO INTERPRET SENSORY INPUT,
ESPECIALLY VISION AND SOUND.

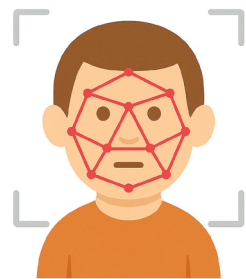
Computer vision

Computer vision allows AI to recognize and analyze visual information. This includes identifying faces in photos, detecting objects in security footage, and even reading handwriting.

For example, in medicine, AI systems are trained to analyze X-rays, CT scans, and MRI images, highlighting areas that might contain signs of disease. Farmers use drones with computer vision to monitor crop health, spotting areas affected by pests or lack of water. Retailers rely on AI-powered cameras to track how shoppers move through stores, helping them redesign layouts to improve shopping experiences. Many smartphones now use facial recognition as a security feature.

Examples of real-world applications:

- Identifying faces in photos
- Detecting objects in security footage
- Reading handwriting
- Analyzing X-rays, CT scans, and MRI images in medicine
- Monitoring crop health with drones in agriculture
- Tracking shopper movement in retail stores
- Smartphone facial recognition for security



Speech recognition

Speech recognition is a branch of AI perception. It allows AI to convert spoken language into digital signals that can later be processed as text. This technology makes it possible for computers to “hear” human speech. Speech recognition itself is not responsible for understanding or acting on the speech, but it provides the raw input that other AI systems require.

For example, systems like Siri, Alexa, and Google Assistant rely on speech recognition to convert spoken words into text before carrying out commands.

Automatic captioning tools also depend on speech recognition to generate subtitles. In some applications, speech recognition works together with language processing, such as in real-time translation between languages.

Examples of real-world applications:

- Virtual assistants (Siri, Alexa, Google Assistant)
- Automatic captioning tools for videos
- Real-time translation apps

**Limitations & Challenges**

Although AI perception has made impressive progress, it does not “see” or “hear” in the way humans do. Instead, it processes raw data such as pixels in an image or sound waves in an audio file. This allows AI systems to recognize patterns quickly and sometimes with remarkable accuracy, but there are important limitations:

- **Task-specific training:** An AI trained to recognize cats in photos cannot automatically recognize dogs without specific training, just as a speech recognition system trained on one language cannot understand another language without retraining. Each new skill requires its own dataset and training process.
- **Lack of common sense:** AI may detect shapes or sounds but does not understand their meaning in context—for example, it might confuse a toy cat with a real one.
- **Data dependence:** Performance depends heavily on the quality and diversity of the data. If the training data is incomplete or biased, the system may make mistakes in real-world situations.
- **Fragility:** Small changes in input, such as unusual lighting in an image or background noise in audio, can cause AI systems to fail where humans would still succeed.

LANGUAGE

Language is one of the most complex human abilities, involving not only words but also meaning, context, and culture. AI has made remarkable progress in this field through Natural Language Processing (NLP), which allows computers to process and generate human language in both written and spoken forms.

NATURAL LANGUAGE PROCESSING ANALYZES, TRANSLATES, AND GENERATES HUMAN LANGUAGE IN TEXT OR SPEECH.

Real-world applications of AI in language include:

- Translation tools (e.g., Google Translate) that enable instant communication across languages.
- Chatbots that provide customer service by analyzing questions and generating responses.
- Large language models (e.g., ChatGPT, Gemini) that can summarize documents, draft essays, or even write computer code.
- Content filtering on social media platforms to detect spam, offensive, or inappropriate material.
- Sentiment analysis that helps businesses understand customer opinions by scanning reviews and posts.
- Educational tools that give feedback on student writing, correcting grammar and suggesting improvements.



In all these applications, Natural Language Processing (NLP) specifically handles the processing of language—analyzing, interpreting, and generating text. These systems often work alongside speech recognition, which first converts spoken words into text before NLP can take over.

Examples of real-world applications:

- Translation tools
- Chatbots
- Large language models
- Content filtering on social media platforms
- Sentiment analysis
- Educational tools

**Limitations & Challenges**

AI language systems can translate text, summarize information, or generate conversations, but they face several challenges:

- Shallow understanding: AI does not truly “understand” meaning; it relies on statistical patterns in data, which can lead to irrelevant or confusing answers.
- Bias in training data: Language models can reproduce stereotypes or offensive language if such patterns are present in the data.
- Difficulty with context: Long conversations, sarcasm, humor, and cultural references are often misunderstood.
- Errors in translation: While real-time translation is powerful, idioms and cultural expressions are frequently mistranslated.

These issues show that AI language tools are valuable helpers but cannot replace human judgment or cultural awareness. They are best used as support systems that speed up tasks, provide suggestions, or make communication easier, while leaving interpretation and final decisions to people. Understanding their limitations helps users apply them wisely and avoid overreliance. In this way, AI language tools can enhance human abilities rather than attempt to replace them.

REASONING AND DECISION-MAKING

Reasoning involves drawing conclusions from information and making choices based on those conclusions. In AI, reasoning and decision-making represent the choice stage, where the system evaluates options and selects an action. While humans rely on experience and common sense, AI applies rules, models, or learned patterns to make choices within a specific domain. Although limited in scope, AI reasoning can be highly effective when focused on a well-defined task.

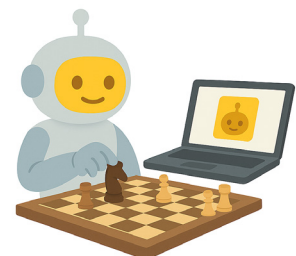
REASONING AND DECISION-MAKING ARE AI'S ABILITY TO EVALUATE OPTIONS AND CHOOSE ACTIONS WITHIN A SPECIFIC DOMAIN.

AI reasoning appears in many areas of daily life. Recommender systems are one of the most common examples. Platforms like YouTube, Netflix, or Spotify analyze a user's past behavior and suggest what to watch or listen to next. Online retailers such as Amazon use similar methods to recommend products based on browsing and purchase history.

In logistics and planning, delivery companies apply AI to calculate the most efficient routes for trucks, balancing distance, fuel, and time. Airlines rely on AI systems to adjust ticket prices depending on demand. Another striking example is in game-playing, where AIs that defeated world champions in chess and Go used reasoning strategies to explore possible moves and select the most promising path.

Examples of real-world applications:

- Recommender systems to suggest videos and music
- Online retailers to recommend products
- Logistics systems to calculate the most efficient route
- Airlines to adjust ticket prices
- Game-playing AIs to choose the best moves



In conclusion, AI reasoning is powerful when applied to narrow, clearly defined problems. It can increase efficiency, improve recommendations, and solve complex challenges in ways that humans alone may find difficult.

Limitations & Challenges

Despite its strengths, AI reasoning faces several important limitations:

- **Domain restriction:** AI can only reason within the data and rules it has been trained on. It cannot transfer decision-making skills to unrelated tasks.
- **Lack of common sense:** Algorithms may choose mathematically optimal solutions that are impractical or unsafe in real life.
- **Hidden bias:** Decisions made by AI in areas such as hiring or lending may reflect biases in the training data, leading to unfair outcomes.
- **Transparency:** Many decision-making systems are “black boxes,” making it difficult to explain how or why a conclusion was reached.

These challenges raise concerns about trust and accountability, especially in areas like finance, healthcare, and law enforcement. In these fields, even small mistakes or hidden biases can cause serious harm. If people cannot see how AI makes its decisions, it is hard to question or fix unfair outcomes. This makes it important to keep AI systems fair, clear, and supervised by humans.

Notes

Common Misconceptions

- *AI can think like humans.*

AI does not understand or think — it recognizes patterns and follows instructions.

- *AI is always right.*

AI can make mistakes, especially if the data is biased, incomplete, or poor quality.

PREDICTION AND FORECASTING

Prediction is one of AI's greatest strengths. By analyzing large amounts of past data, AI can identify patterns and extend them into the future.

PREDICTION AND FORECASTING USE PAST AND CURRENT DATA TO IDENTIFY PATTERNS AND PROJECT FUTURE OUTCOMES.

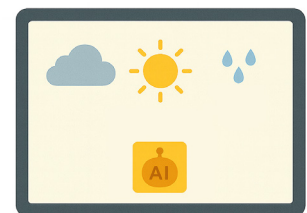
Some applications are now highly established. In finance, AI forecasts stock trends and detects suspicious transactions that may indicate fraud. In healthcare, predictive systems estimate the likelihood of disease outbreaks and assess patient risks. In meteorology, AI improves weather forecasts by analyzing satellite images and atmospheric data. And in retail and e-commerce, predictive tools anticipate customer demand, manage inventory, and reduce waste.

Other applications are still emerging. In sports, teams use AI to analyze player performance and suggest training strategies. In education, predictive analytics helps teachers identify students who may be at risk of falling behind. Governments are also beginning to apply predictive AI in urban planning, such as analyzing traffic flows or planning energy use.

These applications are powerful, yet they also have limits. Predictions are never perfect; their accuracy depends on the quality of the data. For example,

Examples of real-world applications:

- Finance to forecast stock trends
- Healthcare to predict disease outbreak
- Meteorology to generate weather forecasts
- Retail and e-commerce to anticipate demand
- Sports to analyze player performance
- Education to identify students at risk of falling behind
- Governments to predict traffic and energy needs



an AI trained only on sunny weather cannot perform well in a rainy climate. And without human oversight, relying too heavily on predictions can lead to costly mistakes.

Limitations & Challenges

AI prediction relies on patterns in past data, which creates important challenges:

- **Dependence on data quality:** Predictions are only as good as the data used to train the system. If the data is incomplete, biased, or inaccurate, the predictions will also be flawed.
- **Inability to handle the unexpected:** AI struggles with rare or new events that never appeared in its training data, so it may fail in unusual situations.
- **Overconfidence:** Predictions often look exact, but even small errors can grow into big problems, such as in financial trading or weather forecasting.
- **Ethical risks:** Predictive systems in areas like policing or education may unfairly label people, which can limit their opportunities and create long-term harm.

Predictions are powerful tools that can guide decisions, but they should always be checked and supervised by humans.

Notes

Common Misconceptions

- *AI will replace humans completely..*

AI can automate tasks, but it still requires human oversight, creativity, and judgment.

- *Robots and AI are the same thing.*

AI is software (algorithms and models), while robots are machines that may use AI to act.

CONTROL AND ROBOTICS

Control refers to the ability of AI to direct machines that act in the physical world. Robotics combines perception, reasoning, and prediction into coordinated action.

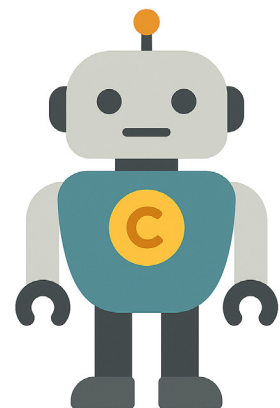
CONTROL AND ROBOTICS IS THE USE OF AI TO DIRECT MACHINES THAT ACT IN THE PHYSICAL WORLD.

One of the most familiar examples of AI in control is the self-driving car. These vehicles use cameras and sensors to take in information about their surroundings. AI perception helps them “see” the road, traffic signs, and obstacles. Prediction is used to guess how other drivers or pedestrians might move, and reasoning helps the car decide when to change speed, stop, or turn.

Robots in industry are another important use of AI. In factories, robotic arms assemble products with speed and precision, often working faster and more accurately than humans on repetitive tasks. Delivery robots and drones also rely on AI to plan safe routes and avoid obstacles, whether they are navigating sidewalks, warehouses, or the air.

Examples of real-world applications:

- Self-driving cars to navigate roads safely
- Industrial robots to assemble products in factories
- Delivery robots to carry goods through warehouses
- Drones to deliver packages
- Robotic surgery to assist doctors with operations
- Agricultural robots to harvest crops and sort produce
- Disaster-response robots to search for survivors in dangerous areas



AI-controlled robots are also becoming more common in other fields. In healthcare, robotic surgery uses AI to guide surgical instruments with great precision, which can shorten recovery times for patients. In agriculture, robots can harvest crops, sort produce, or spray fields, helping farmers reduce waste and save time. In disaster response, AI-driven robots are being tested to search for survivors in collapsed buildings, flooded areas, or other dangerous environments where sending humans would be too risky.

In all these cases, AI is not just analyzing information but also acting in the physical world, turning data into movement and decisions. This shows how AI control and robotics bring together perception, prediction, and reasoning to create systems that can work alongside people in many different settings.

Limitations & Challenges

AI systems that act in the physical world combine perception, reasoning, and prediction, but they are far from perfect:

- **Limited adaptability:** Robots trained for one task, such as picking strawberries, cannot easily adapt to another, such as harvesting apples.
- **Environmental challenges:** Self-driving cars may work well in one city but struggle in different weather, traffic, or road conditions.
- **Mechanical and safety risks:** Hardware failures or incorrect decisions can lead to dangerous accidents.
- **High costs:** Building and maintaining advanced robots is expensive, which limits widespread adoption.

These challenges remind us that while robots and AI-controlled machines can perform repetitive or risky tasks, they still require human supervision and intervention.

ETHICS AND RESPONSIBILITY

As AI becomes more common in daily life, questions of ethics—what is right or wrong—become increasingly important. Ethical considerations guide how AI should be designed, used, and supervised so that it benefits society while avoiding harm.

WHY ETHICS MATTERS

AI is powerful because it extends core human abilities—perception, language, reasoning, prediction, and control—into machines. But power comes with responsibility. Without ethical guidelines, AI could be misused, cause harm, or increase inequality.

For example, if a healthcare AI is trained only on data from adults, it may give poor advice when used with children, creating unfair outcomes. A predictive policing system that relies on biased crime data may target certain communities more than others, reinforcing social injustice.

Ethics ensures that AI systems are designed and used in ways that respect human rights and dignity. It reminds us to ask questions such as: *Who benefits from this technology? Who might be harmed? How can risks be reduced?*

Ethical considerations also help build trust. If people believe that AI is fair, transparent, and accountable, they are more likely to accept and use it. If they do not, society may reject valuable technologies out of fear or suspicion.

In short, ethics matters because AI does not exist in isolation—it affects real people, influences real decisions, and shapes the future of communities worldwide. By applying ethical principles, we can guide AI to support fairness, safety, and positive social change.



UNETHICAL CASE STUDIES

- ***Bias and Fairness***



Case Study: A university used an AI system to help rank student applications. Because the system was trained mostly on past data from high-income schools, it gave unfair preference to wealthy students while disadvantaging equally qualified students from less privileged backgrounds. This shows how bias in data can lead to unfair results.

- ***Privacy***



Case Study: A major social media company secretly used AI to scan private messages in order to target users with ads. Many people felt their personal conversations had been monitored without consent, which caused public backlash and loss of trust. This highlights why privacy is a key ethical concern in AI.

- ***Transparency and Accountability***



Case Study: In the U.S., some courts used an AI tool called COMPAS to predict how likely defendants were to reoffend. Judges used these scores in bail, parole, and sentencing decisions. Researchers later found the system gave harsher scores to Black defendants than to white defendants, and because its reasoning was hidden, unfair outcomes were hard to challenge.

- ***Dependence and Displacement***



Case Study: A factory introduced AI-controlled robots to increase production speed. While profits rose, hundreds of workers lost their jobs without retraining opportunities. This created unemployment and hardship in the local community, showing how AI can displace workers if changes are not managed responsibly.

SUMMARY

Narrow AI vs. General AI

- **Narrow AI:** Current real-world AI; designed for specific tasks (e.g., translation, face recognition). Cannot transfer skills to new tasks.
- **General AI:** A future goal; would learn and apply intelligence across many tasks like a human. Not yet achieved.

Five Cognitive Categories of AI

- **Perception (Input stage):** Converts raw data (images, sounds) into usable information. Examples: computer vision, speech recognition.
- **Language (Processing stage):** Interprets and generates human language. Examples: translation tools, chatbots, content filtering.
- **Reasoning & Decision-Making (Choice stage):** Evaluates options and selects actions. Examples: recommender systems, logistics planning, dynamic pricing.
- **Prediction & Forecasting (Future stage):** Uses past data to estimate future outcomes. Examples: stock forecasts, weather models, education risk alerts.
- **Control & Robotics (Action stage):** Directs machines to act in the physical world. Examples: self-driving cars, industrial robots, robotic surgery.

Limitations & Challenges

- **Perception** – Task-specific training, lack of common sense, data dependence, fragility
- **Language** – Shallow understanding, bias in data, context difficulty, translation errors
- **Reasoning and Decision-Making** – Domain restriction, lack of common sense, hidden bias, transparency issues
- **Prediction and Forecasting** – Data dependence, unexpected events, overconfidence, ethical risks
- **Control and Robotics** – Limited adaptability, environmental challenges, safety risks, high costs

Ethics and Responsibility

- **Bias and Fairness:** AI can reproduce social inequalities if trained on biased data.
- **Privacy:** AI may misuse personal data without consent.
- **Transparency and Accountability:** “Black box” systems make it hard to explain or challenge decisions.
- **Dependence and Displacement:** Overuse of AI can replace jobs and reduce human skills.
- **Why Ethics Matters:** Ensures AI respects human rights, builds trust, and supports fairness, safety, and positive social change.

Activity 4.1

Read the following statements, and write on the line provided for each statement 'T' if it is true and 'F' if it is false.

T/F	STATEMENT
<u>T</u>	Narrow AI can only perform specific tasks and cannot transfer skills to other tasks.
<u>F</u>	General AI has already been achieved and is widely used in real-world applications.
<u>T</u>	Perception in AI refers to converting raw data like images and sounds into usable information.
<u>F</u>	Language AI is mainly about understanding human emotions through language.
<u>T</u>	Reasoning and Decision-Making AI evaluates options and selects actions, such as in recommender systems and logistics planning.
<u>T</u>	Prediction and Forecasting AI uses past data to estimate future outcomes, such as stock trends or weather forecasts.
<u>F</u>	Control and Robotics represents the input stage of AI, focusing on data collection from sensors.
<u>T</u>	One limitation of AI perception is fragility, meaning small changes in lighting or noise can cause errors.
<u>F</u>	AI systems are always transparent and easy to explain, which ensures accountability.
<u>T</u>	Ethical concerns include bias and fairness, privacy, transparency, and the risk of job displacement.

Activity 4.2

Answer the questions below.

1. Which of the following best describes Narrow AI?
 - a) AI that can learn any task like a human
 - b) **AI that can only perform specific tasks**
 - c) AI that always understands meaning
 - d) AI that replaces human judgment completely
2. What is General AI?
 - a) Current real-world AI used in translation tools
 - b) **A future AI that could learn and apply skills across many tasks**
 - c) An AI limited to perception and prediction only
 - d) An AI already used in self-driving cars
3. Which cognitive category involves converting raw data like pixels and sound waves into usable information?
 - a) **Perception**
 - b) Prediction and Forecasting
 - c) Reasoning and Decision-Making
 - d) Language
4. Which is an example of AI in Language processing?
 - a) Stock forecasting
 - b) Self-driving cars
 - c) **Translation tools**
 - d) Robotic surgery
5. AI used in recommender systems, logistics planning, and dynamic pricing belongs to which cognitive category?
 - a) Perception
 - b) **Reasoning and Decision-Making**
 - c) Control and Robotics
 - d) Language

Activity 4.2 (continued)

Answer the questions below.

6. What does Prediction and Forecasting AI mainly do?
 - a) Uses past data to estimate future outcomes
 - b) Summarizes large documents
 - c) Converts sound waves into text
 - d) Decides which move to play in a game
7. Which example belongs to Control and Robotics?
 - a) Detecting faces in photos
 - b) Chatbots for customer service
 - c) Weather forecasting
 - d) Self-driving cars
8. Which is a limitation of AI perception?
 - a) Errors in translation
 - b) Fragility to small changes
 - c) Hidden bias in hiring
 - d) Overconfidence in forecasts
9. Why is transparency and accountability an ethical concern?
 - a) AI always explains its decisions
 - b) AI systems are inexpensive and adaptable
 - c) AI never makes mistakes if data is good
 - d) “Black box” systems make it hard to explain or challenge outcomes
10. Which of the following is NOT listed as an ethical concern of AI?
 - a) Bias and fairness
 - b) Privacy
 - c) Job displacement
 - d) Faster internet speeds

Activity 4.3

Read the narrative and answer the questions below.

- a. A new security system at a train station uses AI cameras to detect suspicious items. One morning, a child leaves behind a toy water gun. The AI system quickly flags it as a potential weapon, causing the station to be evacuated. Later, officials realize it was only a harmless toy.

1. Which cognitive function of AI is being used in this scenario?

Perception – analyzing visual input from cameras.

2. Why did the system mistake the toy water gun for a real weapon?

It recognized the shape but not the context.

3. What limitation of AI perception does this scenario highlight?

AI perception lacks common sense.

4. How could human oversight have prevented the unnecessary evacuation?

A human officer could have reviewed the flagged item and recognized it was only a toy before ordering an evacuation.

- b. An international company uses an AI translation tool for customer support. One customer in America writes, “It’s raining cats and dogs here, will that make my delivery late?” The AI translates the phrase literally into Mandarin. The customer support person reads it and is confused: “How could it rain cats and dogs?!”

1. Which cognitive function of AI is being used in this scenario?

Language – processing and translating text.

2. Why did the translation confuse the customer support person?

The AI translated the idiom word-for-word.

3. What limitation of AI language does this scenario highlight?

AI language has trouble with context and idiomatic expressions.

4. How could the company improve the accuracy of its translations?

By using human review or improving the system with more cultural/linguistic data.

Activity 4.3 (continued)

Read the narrative and answer the questions below.

- c. A food delivery company uses AI to assign drivers to orders. Drivers notice that short, high-paying deliveries mostly go to certain neighborhoods, while other areas get fewer and less profitable orders. The company later admits the AI was trained on past data that favored wealthier neighborhoods.

1. Which cognitive function of AI is being used in this scenario?

Reasoning and Decision-Making – assigning deliveries.

2. What unfair pattern did the drivers notice in the AI's decisions?

Wealthier neighborhoods got more profitable orders.

3. What limitation of AI reasoning does this scenario highlight?

Hidden bias in the training data.

4. How might the company fix this problem?

Retrain with fairer data and set rules for equal distribution.

- d. A city uses AI to forecast traffic for road repairs. The system predicts light traffic for a weekend, even though a major sports event is scheduled. Because the AI had never seen such an event before, it ignored it. Human planners assumed the AI accounted for everything, so construction went ahead. On game day, thousands of fans arrived, and the city was gridlocked.

1. Which cognitive function of AI is being used in this scenario?

Prediction and Forecasting – estimating future traffic.

2. Why did the prediction fail, even though the AI had data about the sports event?

It had no prior examples of such an event, so it ignored the data.

3. What limitation of AI prediction does this highlight?

Inability to handle the unexpected.

4. How could humans have helped avoid the problem?

By being aware of AI's limits, the planners should have reviewed the forecast critically and applied human judgment for unusual events.

Activity 4.3 (continued)

Read the narrative and answer the questions below.

- e. A city tests self-driving buses on public roads. Most of the time, the buses run smoothly. One day, however, a pedestrian suddenly crosses the street outside a crosswalk. The bus hesitates for a moment before stopping, narrowly avoiding an accident. City officials realize the AI system was not fully prepared for unpredictable human behavior.

1. Which cognitive function of AI is being used in this scenario?

Control and Robotics – managing self-driving buses.

2. What nearly caused the accident?

The AI hesitated when faced with unexpected pedestrian behavior.

3. What limitation of AI control and robotics does this highlight?

Safety risks – AI-controlled machines may fail in unpredictable real-world situations.

4. How could humans help reduce this type of risk?

By being aware of AI's limits and ensuring human oversight or emergency controls are in place.

- f. A tech company announces a new AI that can drive a car, write novels, prepare food, and give medical advice—all without retraining. Many people line up to buy this amazing new technology. Would you buy it, or would you be skeptical?

1. What type of AI is being claimed in this scenario?

General AI – an AI that can perform many different tasks across domains.

2. Why might people be excited about such a system?

Because it promises to combine many human-like abilities into one system.

3. Why should we be skeptical of this claim today?

We should be skeptical because General AI has not yet been achieved; all existing AI is still Narrow AI.



Activity 4.4 - Exploring AI in the Real World

In this activity, you will interact with an AI which will act as a mentor, providing examples, questions, and explanations to deepen your understanding of AI applications, limitations, and ethics. In the shared folder you will find separate text documents, each containing the prompt you need for the activity, as well as the PDF of the lesson summary.

Access the folder from: bit.ly/C4_AI_Activity4_4

a. Scenario Exploration

In this part, you will practice identifying AI cognitive functions and their limitations through real-world scenarios.

- Perception
- Language
- Reasoning & Decision-Making
- Prediction & Forecasting
- Control & Robotics

Steps to Follow:

1. Open the shared folder and download the PDF of the lesson summary.
2. Attach the PDF in ChatGPT (or another LLM).
3. Open the Scenario Exploration file, then copy the full prompt and paste it into ChatGPT.
4. You will receive one short story scenario at a time, each showing a real-world application of AI in one of the five cognitive functions.
5. Each scenario hides a limitation or challenge for you to uncover.
6. After the scenario, you will be asked four comprehension questions, one at a time. Answer each question, and the AI will tell you if you are correct, explain briefly, and correct you if necessary.
7. At the end of each scenario, the AI will review the questions you got wrong, highlight key topics you struggled with, and suggest one follow-up exercise.
8. The next scenario will move on to a different cognitive function (no repeats). Continue until all five functions are covered.

Always explain your reasoning, even if unsure — that is what the AI will evaluate.

Don't rush.

b. Questions & Answers

In this part, you will test your understanding of the lesson using the PDF provided in the folder. You can choose between:

- True/False Questions
- Multiple-Choice Questions

Each activity has its own text file with a prompt.

Steps to Follow:

1. Open the shared folder and download the PDF of the lesson summary.
2. Attach the PDF in ChatGPT.
3. Copy the prompt for the activity you want to practice (True/False or Multiple Choice) and paste it into ChatGPT.
4. The AI will create 10 questions based on the summary:
 - If you chose True/False, each question will be answered with “True” or “False.”.
 - If you chose Multiple Choice, each question will have 4 options (A, B, C, D).
5. For each question:
 - Read carefully and submit your answer.
 - The AI will tell you if you are correct, provide the correct answer if needed, and give a short explanation.
6. After all questions:
 - The AI will list the questions you got wrong.
 - It will give you a short review of the key topics you struggled with.
 - It will suggest a follow-up exercise to strengthen your understanding.

Tips:

- For True/False: think about why the statement is true or false before answering.
- For Multiple Choice: read all four options carefully before choosing.
- Use the AI’s feedback to focus on the concepts you find most difficult.
- Repeat the activity for more practice — the AI can generate new sets of questions each time.

c. Ethics Debate

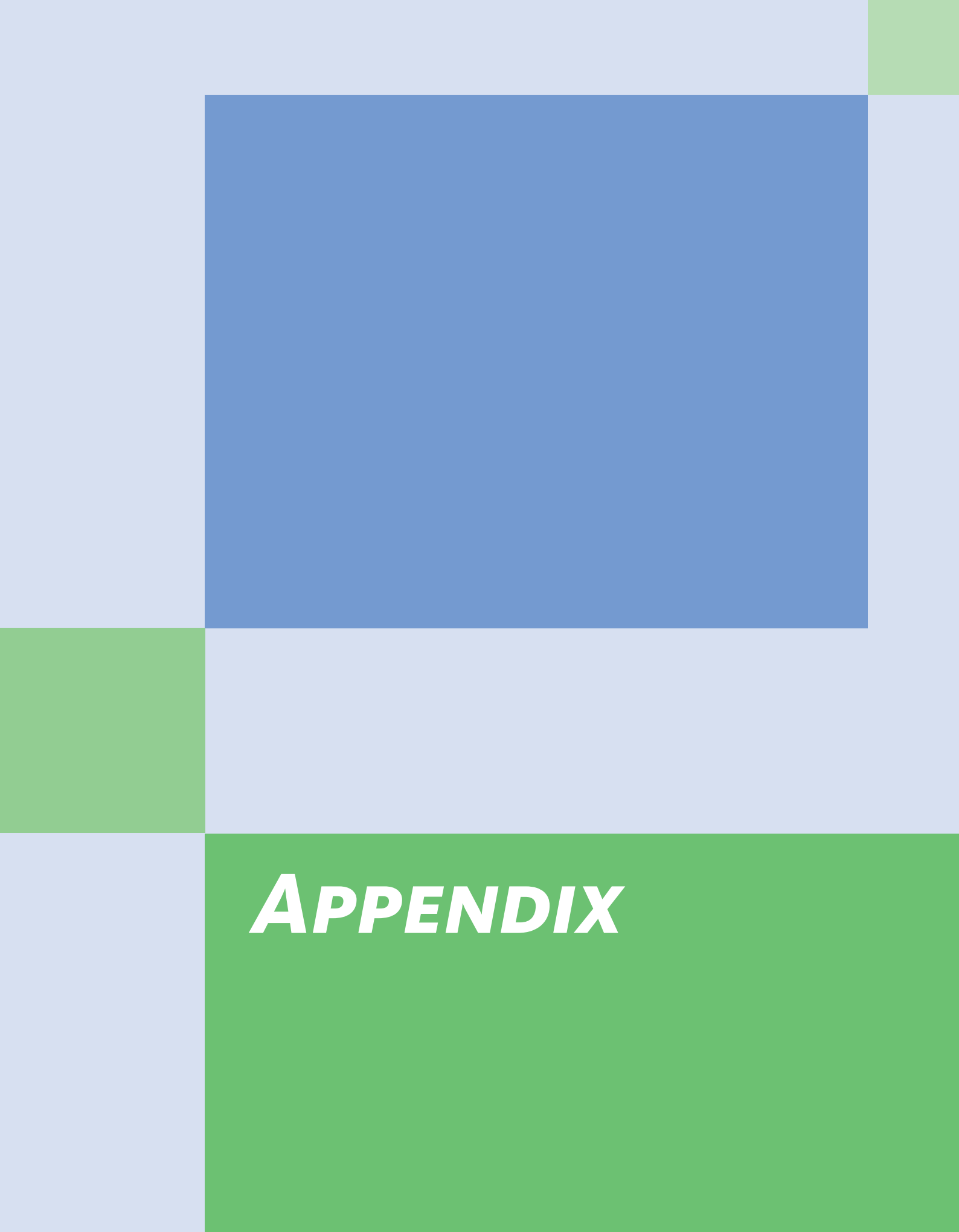
This optional part lets you practice thinking critically about AI and society.

Steps to Follow:

1. Copy the Ethics Debate Prompt into ChatGPT.
2. The AI will first list the available debate themes for you to choose from: Bias, Privacy, Transparency, Job Displacement.
3. After you select a theme, the AI will give you one ethical scenario for it.
4. You will then be asked 3–4 debate questions, one at a time. After each answer, the AI will tell you if your response is strong or weak, challenge you with a counterargument, and suggest improvements.
5. At the end of the debate, the AI will give you a short review of the ethical concepts and suggest one follow-up exercise.
6. Finally, the AI will ask if you want to try another theme. You may continue until you've practiced as many themes as you like.

Tips:

- There's no single "right" answer. Focus on reasoning clearly.
- Use evidence from the lesson (bias, fairness, human oversight).

The background of the page is a light blue grid. A large blue rectangle is positioned in the upper left quadrant. A green rectangle is in the upper right corner. A green rectangle is on the left side, below the blue rectangle. A large green rectangle occupies the bottom right portion of the page, containing the word 'APPENDIX' in white.

APPENDIX

GLOSSARY

TERM	DEFINITION	PAGE
Accuracy	A measure of how often a model's predictions are correct, calculated as the percentage of correct predictions out of the total.	p. 177
Actionable Information	Information that has been interpreted and can be directly used to make informed decisions.	p. 21
Adjacency List	A way to represent a graph by listing each node and the nodes it connects to, along with edge information.	p. 32
Algorithm	A step-by-step set of instructions that tells a computer how to solve a problem or perform a task. Algorithms can manage data, search through it, or arrange it in order.	p. 33, 68, 139
Array	A linear data structure where items are stored one after another in a single line. Arrays have a fixed size and usually hold items of the same type, so new items cannot be added or removed once created.	p. 12, 33
Artificial General Intelligence (AGI / General AI)	A theoretical form of AI that could learn and apply intelligence across many different tasks, like a human.	p. 217
Artificial Intelligence (AI)	The field of computer science focused on building machines that can perform tasks that normally require human intelligence.	p. 159
Ascending Order	Sorting data from smallest to largest or from A to Z.	p. 95
Association	A link formed between two ideas, such as connecting the image of a cup with the word "cup."	p. 158
Bar Chart	A chart that represents values using vertical or horizontal bars, useful for comparing quantities across categories.	p. 15
Best-Case Scenario	The situation where an algorithm finishes its task in the fewest possible steps (e.g., finding the item on the first try).	p. 73
Bias	An unfair pattern in AI predictions or decisions caused by biased training data or flawed design.	p. 228
Big Data	Large and varied sets of data that help AI models learn better and make more reliable predictions.	p. 184
Binary Search	A search algorithm for sorted datasets that repeatedly divides the list in half to find the target more quickly.	p. 75
Black Box	A system, often in AI, where the decision-making process is hidden and hard to explain or challenge.	p. 223
Block (in programming)	A section of code grouped together to perform a task; examples include initialization, main body, and termination.	p. 140
Bubble Sort	A sorting algorithm that repeatedly compares and swaps adjacent items until the list is sorted.	p. 111

Categorical Data	Another term for qualitative data; data that can be grouped and labeled but not measured.	p. 18
Categorical Prediction	A type of prediction where the AI assigns inputs to categories, such as identifying if an object is a bottle or not.	p. 175
Chart	A visual method of presenting data so patterns and trends can be easily seen (e.g., line, bar, pie, scatter).	p. 14
Child Node	A node in a tree that descends from another node (its parent).	p. 46
Cleaning Data	The process of removing errors or incorrect entries from a dataset so the data is accurate.	p. 12
Compare, Shift, Insert	The basic strategy used by insertion sort: compare items, shift larger ones, and insert the held item in the right place.	p. 99
Continuous Data	Quantitative data that can take any value within a range, such as temperature or time.	p. 16
Continuous Learning	The idea that a model keeps improving over time as it encounters new data and situations.	p. 165
Continuous Prediction	A type of prediction where the AI estimates a numerical value, such as house prices or temperatures.	p. 178
Control & Robotics	The use of AI to direct machines in the physical world, such as self-driving cars or delivery robots.	p. 226
Criterion (Sorting Criterion)	The rule or property used to decide how to sort data, such as by score, name, or price.	p. 95
Data	Raw input that has not yet been processed to add meaning.	p. 8
Data Structure	The specific way in which data is arranged and stored in a computer.	p. 12
Decision-Making	The process of selecting the best choice from several options by comparing their outcomes and choosing the one with the highest utility.	p. 197
Dependence & Displacement	When overreliance on AI reduces human roles or causes job loss without proper support or retraining.	p. 228
Descending Order	Sorting data from largest to smallest or from Z to A.	p. 95
Discrete Data	Quantitative data that is countable and can only take whole number values, such as number of students.	p. 17
Dynamic Size	A feature of some data structures (like lists, stacks, and queues) that allows them to grow or shrink as items are added or removed.	p. 34
Edge	In a graph, a link between two nodes that can carry extra information, such as distance or time.	p. 50
Error	The difference between the predicted value and the actual value.	p. 180
Ethics (in AI)	Principles that guide AI to be fair, safe, and beneficial to society while avoiding harm.	p. 228

Execution Time	The amount of time it takes for a program or algorithm to complete its run. It is often used to compare how efficiently different algorithms perform.	p. 72, 124
Explanation	A form of reasoning that works backward from an outcome (a given) to find the most plausible cause.	p. 201
FIFO (First In, First Out)	A principle where the first item added is the first one removed, used in queues.	p. 42
Fixed Size	A feature of some data structures (like arrays and tuples) that means their size cannot change once created.	p. 33
Flow (of a program)	The sequence in which steps of an algorithm or program are carried out.	p. 139
Flowchart	A diagram that shows the steps of a process using symbols and arrows to indicate decisions and actions.	p. 69
General AI	See Artificial General Intelligence (AGI).	p. 217
Generalization	The ability of a model to apply what it has learned to correctly identify or handle new, unseen examples.	p. 163, 180
Given	The starting point of explanation, an accepted observation or outcome that needs to be explained.	p. 201
Graph	A non-linear data structure where nodes are connected by edges, allowing many-to-many relationships.	p. 32
Hierarchical Structure	An arrangement where items are organized in levels, like in a tree with parent and child nodes.	p. 31
Indentation	The spacing used in code to show which instructions belong to a block or loop.	p. 140
Index	The position number of an item in a list or array, usually starting at 0. It helps locate and access items in a linear data structure.	p. 35, 70
Inference	A form of reasoning that links known facts, observations, or predictions to reach a new conclusion.	p. 195
Information	Data that has been processed and given meaning, making it useful for understanding or decision-making.	p. 8
Inner Loop	The loop inside another loop; in sorting, it compares and shifts or swaps data items repeatedly during each pass.	p. 104, 117
Insertion Sort	A sorting algorithm that builds a sorted section one item at a time by comparing, shifting, and inserting.	p. 99
Interpreting Data	The step of explaining what data means, asking “why” it happened and “what should we do.”	p. 20–21
Knowledge Base	The collection of rules, facts, or learned patterns that reasoning relies on to connect conditions to outcomes.	p. 206
Labeling Data	Adding descriptive tags or identifiers (like “temperature in °C”) to clarify what the data represents.	p. 11

Language (AI)	AI systems that process, understand, or generate human language in text or speech.	p. 220
Leaf Node	A node in a tree that has no children.	p. 46
Learning	The ability to form new associations and make use of them through repetition and pattern recognition.	p. 158
LIFO (Last In, First Out)	A principle where the last item added is the first one removed, used in stacks.	p. 39
Line Chart	A chart where data points are connected by a line, often used to show trends over time.	p. 14
Linear Data Structure	A type of data structure where items are arranged sequentially, like arrays, lists, stacks, and queues.	p. 30
Linear Search	A simple search algorithm that checks each item in a list one by one until it finds the target or reaches the end.	p. 68
List	A linear data structure with a dynamic size that can store mixed data types and allows insertion and removal at any position.	p. 36
Logarithmic Growth	A type of growth where progress increases quickly at first but slows as the dataset gets larger (common in binary search).	p. 80–81
Logic	The rules that guide reasoning and ensure conclusions follow consistently from premises.	p. 194
Loop	A control structure that repeats a set of instructions until a condition is met.	p. 104
Machine Learning	A branch of AI where a computer learns from data by recognizing patterns and forming associations.	p. 159
Mean Absolute Error (MAE)	A way to measure how accurate a model's numerical predictions are, by calculating the average size of the errors (ignoring whether they are positive or negative).	p. 180–181
Memory Latency	The delay between requesting data from memory and receiving it.	p. 140
Misclassification	A prediction error where a model gives the wrong output, such as failing to recognize a cup with a handle or labeling a tomato as an apple.	p. 163, 177
Model	The AI's internal system that stores recognized patterns and associations, used to make predictions or classifications.	p. 162
Narrow AI	AI designed for a specific task or limited set of tasks, such as translation or facial recognition.	p. 216
Natural Language Processing (NLP)	A branch of AI that analyzes, interprets, and generates human language.	p. 220
Node	A unit of data in tree or graph structures that can connect to other nodes.	p. 46
Non-Linear Data Structure	A type of data structure where data items are connected in more complex ways, like trees and graphs.	p. 31

Organizing Data	Arranging data in ways such as cleaning, structuring, or sorting so it becomes easier to analyze.	p. 11
Outer Loop	The main loop that controls how many passes through the list an algorithm makes.	p. 105, 118
Pair, Compare, Swap	The basic strategy used by bubble sort: compare pairs of adjacent items and swap them if needed.	p. 111
Parent Node	A node in a tree that has one or more child nodes.	p. 46
Pattern Recognition	The ability of a system to detect recurring regularities, such as shapes or sounds that repeat.	p. 160
Perception (AI)	The ability of AI to interpret sensory input, like vision or sound, using sensors, cameras, or microphones.	p. 217
Performance	A measure of how efficiently an algorithm runs, often judged by the number of steps it takes and the time it needs to finish.	p. 72, 124
Pie Chart	A circular chart divided into slices that represent proportions of a whole.	p. 15
Plausibility	A measure of how believable or reasonable an explanation is, based on how well its premises fit the outcome.	p. 201
Prediction	When an AI system uses its model to estimate the most probable outcome in a new situation.	p. 174
Prediction & Forecasting (AI)	Using past and current data to identify patterns and estimate future outcomes.	p. 224
Premise	The starting point of reasoning, which can be a fact, observation, or prediction.	p. 195
Privacy	The right to keep personal data protected from misuse or unauthorized access.	p. 228
Qualitative Data	Descriptive data that can be categorized into groups (like colors or gender) but not measured.	p. 18
Quantitative Data	Numerical data that can be measured or counted.	p. 16
Queue	A linear data structure with a dynamic size where items are added at the end and removed from the beginning, following FIFO.	p. 42
Reasoning	A structured way of thinking that helps us discover, decide, or explain by following logical rules.	p. 194
Reasoning & Decision-Making (AI)	AI systems that evaluate options and choose actions within a defined area, like recommender systems.	p. 222
Root Node	The top node in a tree, from which all other nodes descend.	p. 31, 46
Scalability	The ability of an algorithm or AI system to keep performing well as the dataset grows larger or the tasks become more complex.	p. 130, 225
Scatter Plot	A chart that shows the relationship between two continuous variables as individual points.	p. 14
Search Index	The current position in a dataset that an algorithm is checking.	p. 78

Search Range	The section of a dataset that an algorithm is currently looking at during a search.	p. 78
Sentiment Analysis	An AI tool that scans reviews, posts, or text to detect opinions, such as positive, negative, or neutral.	p. 220
Sorted List	A list in which items are arranged in order (such as numbers from smallest to largest or words alphabetically).	p. 75
Sorting	The process of arranging data in a specific order based on a chosen criterion.	p. 94
Sorting Data	Re-arranging data according to a criterion such as alphabetical order, numerical order, or date.	p. 13
Speech Recognition	An AI capability that converts spoken words into text.	p. 218
Stack	A linear data structure with a dynamic size where items can only be added or removed from the end, following LIFO.	p. 39
Step Count	The number of individual operations an algorithm performs while completing its task.	p. 124
Structuring Data	Storing data in an organized way, such as in lists (arrays) or tables.	p. 12
Supervised Learning	A type of machine learning where the system is trained with labeled data (e.g., images tagged as “cup”).	p. 160
Swap	An operation where two data items exchange positions, usually requiring three moves in memory.	p. 135
Table	A way of structuring data using rows and columns to handle multiple sets of related data.	p. 12–13
Target	The specific item an algorithm is trying to find in a dataset.	p. 70
Termination	The end block of a program or algorithm that signals completion of the task.	p. 110, 123
Training	The process of improving a model using new data and correcting mistakes.	p. 163
Training Data	The examples used to teach an AI model, helping it learn patterns for making predictions.	p. 178
Transparency & Accountability	The principle that AI systems should explain their decisions clearly so they can be reviewed and corrected if needed.	p. 228
Tree	A non-linear data structure that organizes items hierarchically in nodes connected as parents and children.	p. 31
Tuple	A linear data structure with a fixed size and format that can hold different types of data, often pairing related data together.	p. 37
Unsupervised Learning	A type of machine learning where the system groups data based on similarities without using labels.	p. 161
Utility	A measure of how useful, valuable, or goal-fitting an outcome is, used in decision-making.	p. 200
Variable	A label in programming that holds data and whose value can change.	p. 103

Variety (in data)	Having many different types of examples in the training data, which helps the model make better predictions in new situations.	p. 184
Visualizing Data	Presenting data in a visual format like charts or graphs to make trends and patterns clearer.	p. 14
Volume (in data)	The total amount of training data available to the model, which helps it learn broader patterns.	p. 184
Weight	Extra information stored on a graph edge, such as the distance or time between two nodes.	p. 51
Worst-Case Scenario	The situation where an algorithm takes the maximum possible steps to complete a task (e.g., checking the whole list).	p. 73

INDEX

Accuracy – Measure of performance for categorical predictions; percentage of correct predictions out of total attempts (p.176–178).

Adjacency list – A way of representing a graph by listing each node along with the nodes it connects to and the weight of each edge (p.32, 50–52).

Algorithm – A series of steps designed to perform a specific task, such as sorting a list (p.139–140).

- Algorithm performance – Evaluation of how efficiently an algorithm runs, often measured by execution time in best-case and worst-case scenarios (p.72–74).
- Algorithms (data structures) – Sets of instructions that control data structures, defining rules for size, data type, and operations (p.33).

Artificial General Intelligence (AGI / General AI)

– A theoretical AI system that could learn and apply intelligence flexibly across many tasks, similar to humans. Not yet achieved (p.217, 236).

Associations – Links formed between known and new information; essential for learning (p.158–159).

Array (List) – A linear data structure that stores items sequentially in memory. Arrays have a fixed size and homogeneous data type: items can be updated but not inserted or removed (p.12–13, 33–35).

Bar chart – A chart in which categories are represented by bars; useful for comparing quantities across categories. Can be vertical or horizontal (p.15, 17, 19).

Bias – An ethical concern where AI reproduces unfair patterns from biased training data, leading to unequal treatment (p.228–229).

Best-case scenario – The situation in which an algorithm finds the target immediately or completes its task with the fewest possible steps. For example, in search, it may succeed on the very first item checked (p.73, 85).

Big Data – Large datasets characterized by volume and variety; improve AI models by helping them generalize

and make more accurate predictions (p.184).

Binary search – Search algorithm for sorted datasets that repeatedly divides the search range in half until the target is found or eliminated (p.75–79).

- Flowchart – Diagram showing binary search process (p.77).
- Pseudocode – Step-by-step outline of binary search (p.79).
- Execution time – Grows logarithmically with dataset size, modeled with $\log_2(n)$ (p.80–82).
- Requirements – Dataset must be sorted (p.75, 86).

Black box problem – A limitation of AI reasoning and decision-making where systems lack transparency, making it difficult to explain or challenge outcomes (p.223).

Block – A segment of code that performs a task, usually part of initialization, main body, or termination (p.140).

Bubble sort – Sorting algorithm using the strategy of pair, compare, and swap. Repeatedly passes through the list, swapping adjacent items if out of order (p.111–113, 117–123).

- Process – Uses outer loop (repeats passes) and inner loop (compares/swaps adjacent pairs) (p.117–123).
- Pseudocode – Instructions for bubble sort (p.120–123, 153).
- Performance – Requires more steps than insertion sort; poor scalability; inefficient for large datasets (p.134–138).

Categorical data – Another name for qualitative data, which can be grouped and labeled but not measured (p.18).

Categorical prediction – Prediction task where the AI classifies inputs into categories (e.g., cat or not a cat, language detection) (p.175–176).

Charts – Visual tools to present data. Four common types: line chart, scatter plot, bar chart, and pie chart

(p.14–15).

Child node – A node that descends from a parent node in a tree structure (p.46).

Cleaning data – Removing erroneous or implausible data values before processing (p.12).

Continuous data – Quantitative data that can take any value within a range (e.g., temperature, height, time). Best visualized with line charts or scatter plots (p.16–17).

Continuous learning – The idea that learning does not stop after training; models can keep improving with new data and feedback (p.165).

Continuous prediction – Prediction task where the AI estimates a numerical value (e.g., house prices, rainfall, travel times) (p.178–180).

Control and Robotics – The use of AI to direct machines acting in the physical world. Applications include self-driving cars, industrial robots, delivery drones, robotic surgery, agricultural robots, and disaster-response robots (p.226–227).

Criterion (sorting) – The basis for sorting, such as numeric (scores, prices, time) or alphanumeric (names, book titles) (p.95).

Data – Raw values or signals that can be processed into meaningful information (p.8).

Data structures – Methods of organizing data in computers, such as lists (arrays), tables, stacks, queues, trees, and graphs (p.12–13).

- Dynamic size – Property of structures like lists, stacks, and queues that allows them to grow or shrink as items are added or removed (p.34, 39, 42).

Decision-making – The process of selecting the best choice from several options by weighing their outcomes and choosing the one with the highest utility (p.197–200).

Discrete data – Quantitative data that can only take whole-number values (e.g., number of students). Often visualized using bar charts (p.17).

Edge – A link between two nodes in a graph, often carrying additional information such as distance or latency (p.32, 50–52).

Error – The difference between predicted and actual values; can be positive (underestimation) or negative (overestimation) (p.180).

Ethics and responsibility – Principles ensuring AI respects human rights, fairness, safety, privacy, and accountability, while reducing risks of harm or inequality (p.228).

Execution time – The time required for a program to complete its run.

- In search – Calculated as time per attempt $\times$ number of attempts (p.72, 80–82).
- In sorting – Influenced by step count and memory latency (p.124–125, 131–133, 138).

Explanation – Reasoning backward from an observed outcome to uncover its most plausible causes, contributors, or relationships (p.201–204).

FIFO (First In, First Out) – Principle governing queues; the first item added is the first removed (p.42–44).

Flow & control – Flow is the sequence of steps in a program; control refers to conditions that determine which steps are executed, repeated, or skipped (p.139).

General AI – see Artificial General Intelligence (AGI).

Given – An accepted observation or outcome that serves as the starting point for explanation (p.201–202).

Graph – A non-linear data structure where nodes can connect freely to represent complex relationships. Edges may store additional information (p.32, 49–54).

Hierarchy – Organizational model in which data items are arranged at levels, such as in a tree structure (p.31, 45–47).

Hold index / hold data – Variables in sorting algorithms representing the index and the value temporarily stored for comparison (p.103, 116).

Information – Data that has been processed into meaningful form, typically through identification, organization, visualization, and interpretation (p.8–11, 21).

- Interpreting data – The final step; answering “why” and “what should we do?” to make data actionable (p.20–21).

- Labeling data – Adding descriptive identifiers (e.g., “temperature, outdoor, Celsius”) to clarify meaning and context (p.11).
- Organizing data – Arranging data to make it usable, through cleaning, structuring, or sorting (p.11–13).

Inference – Arriving at a new piece of information by linking premises (facts, observations, or predictions) into a conclusion (p.195–197).

Initialization – The first block of a program, where variables are declared and initial values assigned (p.106–107, 119–120).

Insertion sort – Sorting algorithm using the strategy of compare, shift, and insert. Builds a sorted section one item at a time (p.99–106, 139–140).

- Process – Uses outer loop (moves through each index) and inner loop (compares/shifts until correct position is found) (p.103–106).
- Pseudocode – Instructions for insertion sort (p.106–110, 152).
- Performance – Fewer steps and faster than bubble sort; still scales poorly for very large datasets (p.124–130).

Job displacement – Risk of AI replacing human workers without retraining opportunities, leading to unemployment and inequality (p.229).

Knowledge base – The collection of rules, facts, and learned patterns that reasoning processes rely on to connect conditions to outcomes (p.201).

Labeling data – see Information.

Language (AI cognitive function) – Natural Language Processing (NLP) enables AI to analyze, interpret, and generate human language. Applications include translation tools, chatbots, large language models, sentiment analysis, and educational tools (p.220–221).

Leaf node – A node in a tree with no child nodes (p.46).

Learning – The ability to form and use new associations; requires pattern recognition, association, and repetition (p.158–159, 166).

LIFO (Last In, First Out) – Principle governing stacks; the last item added is the first removed (p.39–41).

Linear search – A simple search algorithm that checks each item in order until the target is found or the end is reached (p.68–71).

- Flowchart – Diagram showing linear search process (p.69–70).
- Pseudocode – Step-by-step outline of linear search (p.71).
- Execution time – Grows linearly with dataset size (p.72–73).
- Applicability – Works on both sorted and unsorted datasets (p.75, 84).

Linear structures – Data structures where items are stored sequentially: array, list, tuple, stack, queue (p.33–34).

List – A linear data structure with dynamic size, accepts mixed data types, and allows insertion or removal at any position (p.36–37).

Logic – The rules that guide reasoning from premises to conclusions; the framework for inference, decision-making, and explanation (p.194).

Logarithmic growth – Growth pattern of binary search execution time; efficiency improves as dataset size increases (p.80–82).

Loop – Control structure used to repeat a set of instructions. Sorting algorithms use outer and inner loops (p.104–106, 117–122).

Machine learning – A branch of AI where computers are designed with learning capabilities to identify patterns, form associations, and improve through repetition (p.159).

Mean Absolute Error (MAE) – A measure of performance for continuous predictions; the average size of errors regardless of direction (p.180–183).

Memory latency – Delay between requesting data from memory and receiving it; becomes significant for large datasets stored outside cache (p.131–133, 138–139).

Misclassification – An error where a model incorrectly labels or classifies input data (p.163–164, 176–177).

Model – The internal system that represents recognized patterns, labels, and rules about what features matter or don’t (p.162–163).

Natural Language Processing (NLP) – AI technology that processes, translates, and generates human language in text or speech (p.220).

Narrow AI – Current form of AI designed to perform a specific task or a small set of tasks very effectively (p.216–217).

Node – A data construct used in trees and graphs to store both data and links to other nodes (p.46–48, 50).

Order (sorting) – Direction in which data is arranged, e.g., ascending (least to greatest) or descending (greatest to least) (p.95).

Parent node – A node with child nodes branching from it in a tree structure (p.46).

Pattern recognition – The ability to detect recurring regularities, such as the shared shape of cups regardless of color or design (p.160).

Pie chart – A circular chart showing categories as slices of a whole, indicating proportions or percentages (p.15, 20).

Plausibility – A measure of how reasonable or believable an explanation is, based on how well its premises fit the observed outcome (p.201, 204).

Prediction – Process by which AI (or humans) estimates the most probable outcome based on learned patterns (p.174–175).

Prediction & Forecasting – AI's ability to analyze past data to estimate future outcomes. Applications include finance, healthcare, meteorology, retail, education, sports, and urban planning (p.224–225).

Premises – The starting points of reasoning; can be facts, observations, predictions, preferences, or rules (p.195, 197–198).

Privacy – An ethical issue involving AI misuse of personal or sensitive data without consent (p.229).

Program – A set of written instructions a computer can execute to run an algorithm (p.139).

Qualitative data – Descriptive, categorical data that can be observed but not measured (e.g., gender, grade levels, types of cars). Must often be converted into quantitative form for visualization (p.18–19).

Quantitative data – Data that can be measured (continuous) or counted (discrete). Best visualized with line charts, scatter plots, or bar charts (p.16–17).

Queue – A linear data structure that follows FIFO. Items are added at the end and removed from the beginning (p.42–44).

Reasoning – A structured way of thinking to discover, decide, or explain. Outcomes may be inference, decision, or explanation (p.194, 206).

Reasoning and Decision-Making (AI) – AI's ability to evaluate options and select actions in a specific domain. Examples include recommender systems, logistics planning, dynamic pricing, and game-playing AIs (p.222–223).

Repetition – Strengthens associations and improves pattern recognition in both humans and AI (p.158–159, 166).

Root node – The topmost node in a tree structure from which all other nodes descend (p.46).

Scalability – How an algorithm's performance changes as dataset size increases. Sorting algorithms like insertion and bubble sort scale poorly due to quadratic growth (p.129–130, 137–138).

Scatter plot – A chart showing individual data points, often used to highlight relationships or correlations between two variables (p.14, 16–17).

Search index – Position within a dataset being checked during a search algorithm (p.71, 78).

Search range – Portion of a dataset still under consideration in a binary search; updated each iteration (p.78).

Searching – Fundamental operation required to locate a specific item in a dataset before modification, extraction, or processing (p.68, 83).

Sorting – The process of arranging data in a specific order based on a criterion, such as chronological order, increasing or decreasing value, or categories. Sorting makes it easier to search, interpret, and analyze data (p.13–14, 94–98).

Stack – A linear data structure that follows LIFO. Items can only be added or removed from the end (p.39–41).

Step count – The number of operations an algorithm performs. Used alongside execution time to measure performance (p.124, 126–127, 135).

Supervised learning – A type of machine learning that uses labeled data; the system is trained with examples

where the correct answer is provided (p.160).

Swap – Operation in bubble sort where two adjacent items exchange positions. Each swap requires three moves in memory (p.135).

Table – An arrangement of data in rows and columns; well-suited for organizing multiple sets of related data (p.12–13).

Termination – The final block of a program where a message indicates the task is complete and the program ends (p.110, 123).

Training – The process of improving a model using new data and corrections. It involves analyzing mistakes, refining rules, and updating the model to improve accuracy and generalization. Effective training depends on both the volume (amount) and variety (diversity) of training data (p.163–165, 178, 183–184).

Transparency and accountability – Ethical principle requiring AI decisions to be explainable and challengeable. Lack of transparency creates risks of unfair or harmful outcomes (p.229).

Tree – A non-linear hierarchical data structure with nodes connected along branches, including root, parent, and child nodes (p.31, 45–48).

Tuple – A linear data structure with fixed size and format, can store multiple data types together as a unit (p.37–38).

Uncertainty – The degree to which reasoning outcomes may not be definite, especially when based on predictions or incomplete premises (p.197, 204–205).

Underestimation/Overestimation – When predictions consistently fall below or above actual values (p.180–182).

Unsupervised learning – A type of machine learning that uses unlabeled data; the system identifies patterns and groups similar items without predefined labels (p.161).

Utility – A measure of how useful, valuable, or goal-fitting an outcome is; used to evaluate decisions (p.200).

Visualization – Presenting data in visual form (charts, graphs) to highlight patterns and trends (p.14–15).

Worst-case scenario – Situation where an algorithm must check all possible items before concluding, or in sorting, when data requires the maximum steps (e.g., reversed order for ascending sort). Represents the longest possible execution time (p.73–74, 124–127).

